



SALPHOENIX®1A100 系列 FPGA

PCI Express 用户手册

上海安路信息科技股份有限公司

UG713 (v1.2) 2023 年 2 月



目 录

目 录	1
1 SALPHOENIX®1A100 系列 FPGA PCI Express 控制器概述	8
1.1 PCIe 控制器支持的 PCIe 协议版本	9
1.2 PCIe 控制器支持的功能	9
1.3 PCIe 控制器设备类型	10
1.4 PCIe 控制器应用领域	11
1.5 PCIe 控制器用户侧数据位宽和时钟频率	12
1.6 PCIe 控制器内部框图	12
1.7 PCIe 控制器 BAR 空间	13
1.8 寄存器配置空间 (Configuration Space)	14
1.8.1 配置空间	14
1.8.2 PCIe 能力寄存器列表	15
1.8.3 PCIe 扩展能力寄存器	15
1.8.4 PCIe 寄存器的本地地址映射	16
2 PH1A100 系列 FPGA PCI Express 控制器信号接口	19
2.1 时钟和复位接口	23
2.1.1 功能简介	23
2.1.2 端口	23
2.1.3 Link Training 和 LTSSM 相关用法	25
2.1.4 时序图	27
2.2 系统信息接口 (SII)	27
2.2.1 功能简介	28
2.2.2 端口	28
2.3 发送接口 XALI0 and XALI1	30



2.3.1 功能简介.....	30
2.3.2 端口.....	30
2.3.3 PCIe 控制器发送 Request 用法.....	35
2.3.4 发送 CPL TLP 用法.....	43
2.4 接收目标端口 TRGT1.....	47
2.4.1 功能简介.....	47
2.4.2 端口.....	47
2.4.3 用法.....	53
2.4.4 时序图.....	55
2.5 目标完成包查找表 (Target Completion LUT) 接口.....	58
2.5.1 功能简介.....	58
2.5.2 端口.....	58
2.6 接收旁路 BYPASS 接口 (RBYP).....	59
2.6.1 功能简介.....	59
2.6.2 端口.....	59
2.6.3 用法.....	63
2.6.4 时序图.....	65
2.7 DBI 接口.....	66
2.7.1 功能简介.....	66
2.7.2 端口.....	66
2.7.3 用法.....	68
2.7.4 时序图.....	68
2.8 ELBI 接口.....	69
2.8.1 功能简介.....	69
2.8.2 端口.....	69
2.8.3 用法.....	71
2.8.4 时序图.....	71



2.9 SII: Legacy Interrupt 接口	72
2.9.1 功能简介	72
2.9.2 端口	72
2.9.3 用法	73
2.9.4 时序图	73
2.10 MSI 接口	74
2.10.1 功能简介	74
2.10.2 端口	74
2.10.3 用法	75
2.10.4 时序图	75
2.11 MSI-X 接口	76
2.11.1 功能简介	76
2.11.2 端口	76
2.11.3 用法	78
2.11.4 时序图	78
2.12 物理层 (PHY) 部分接口和数据链路层 (Data Link) 部分接口	79
2.12.1 功能简介	79
2.12.2 端口	79
2.13 VMI 接口	83
2.13.1 功能简介	83
2.13.2 端口	83
2.13.3 用法	85
2.13.4 时序图	85
2.14 FLR 接口	85
2.14.1 功能简介	85
2.14.2 端口	86
2.14.3 用法	86



2.14.4 时序图.....	86
2.15 CRI (Control Register Interface) 接口	87
2.15.1 功能简介.....	87
2.15.2 端口.....	87
2.15.3 用法.....	87
3 版本信息	88
免责声明	错误!未定义书签。

图片目录

图 1-1 PH1A100 (SFG676 封装) 中的 PCI Express 控制器和 PHY 资源示意图	8
图 1-2 PCI Express 系统结构拓扑图示例	11
图 1-3 PCI Express 控制器内部框图	12
图 1-4 Memory/IO 事务访问路径	14
图 1-5 PCI Express 扩展配置空间地址分配	15
图 2-1 PCIe 控制器的主要模块和接口	19
图 2-2 PCIe 控制器主要接口用法示例	22
图 2-3 CFG Request TLP 路由	22
图 2-4 MEM 和 IO 操作路由图	23
图 2-5 PCIe 初始化过程	27
图 2-6 用 app_ltssm_enable 延时 LTSSM	27
图 2-7 Client0/1 接口 (XALIO/1) 发送 Request TLP 相关信号	35
图 2-8 地址对齐关闭典型时序图	37
图 2-9 地址对齐开启典型时序图	38
图 2-10 地址对齐开启和关闭对数据总线周期数的影响	39
图 2-11 Client1 接口上的 MRd Request TLP	40



图 2-12 Client1 接口上的 MWr Request TLP	40
图 2-13 Client1 接口上的背靠背 Back to Back MRd 和 MWr TLP 示例	41
图 2-14 带有具体包头信号的 Memory Write Transaction.....	41
图 2-15 带有具体包头信号的 Memory Read Transaction.....	42
图 2-16 带有具体包头信号的 Atomic Transaction.....	42
图 2-17 带有具体包头信号的 Message Transaction.....	43
图 2-18 带有具体包头信号的 I/O Transaction.....	43
图 2-19 Client0/1 接口 (XALI0/1) 发送 CPL 报文相关信号	44
图 2-20 Client0 接口发送 CPL TLP 示例波形	47
图 2-21 TRGT1 接口信号.....	53
图 2-22 接收队列 (Receive Queue) 和输出队列 (Output Queue) 暂停 (Halting) 操作....	54
图 2-23 用 Grant/Halt 方式控制接收队列释放的示例.....	55
图 2-24 TRGT1 接口上的单个 MWr Request TLP	55
图 2-25 TRGT1 接口用户通过 halt 信号进行反压	56
图 2-26 带有具体 Header 信号的多个 MWr Request TLP.....	56
图 2-27 带有具体 Header 信号的多个 MRd Request TLP.....	57
图 2-28 带有具体 Header 信号的多个 Atomic 报文接收	57
图 2-29 RBYP 接口信号.....	64
图 2-30 RBYP 接口上的完成包 Completion	65
图 2-31 RBYP 接口上的完成包超时 Completion Timeout	66
图 2-32 DBI 写寄存器时序图.....	68
图 2-33 DBI 读寄存器时序图.....	69
图 2-34 ELBI 访问时序图.....	71
图 2-35 Legacy 中断请求时序图	74
图 2-36 MSI 中断请求时序波形	76
图 2-37 MSI-X 中断请求时序图.....	79
图 2-38 VMI 接口时序图.....	85



图 2-39 FLR 时序图.....	87
---------------------	----

表格目录

表 1-1 PCI Express 控制器模式.....	11
表 1-2 PCIe 控制器带宽、用户侧数据位宽和时钟频率对照表.....	12
表 1-3 BAR0, 2, 4 的特性列表.....	14
表 1-4 PCIe 控制器能力寄存器配置空间.....	15
表 1-5 PCIe 控制器支持的 PCIe 能力寄存器列表.....	15
表 1-6 Extended Capability 寄存器.....	15
表 1-7 PF 本地地址映射表.....	16
表 1-8 VF 本地地址映射表.....	17
表 2-1 时钟和复位信号列表.....	24
表 2-2 SII 的部分端口列表.....	28
表 2-3 XALI0 和 XALI1 接口信号列表.....	31
表 2-4 TLP Request Header 格式和 client 接口信号对应表.....	35
表 2-5 Traslation Request Header 格式和 client 接口信号对应表.....	36
表 2-6 Completion TLP Header 格式和 client 接口信号对应表.....	45
表 2-7 TRGT1 接口信号列表.....	47
表 2-8 TLP Request Header 格式和 trgt1 接口信号对应表.....	54
表 2-9 目标完成包查找表信号.....	58
表 2-10 RBYP 接口信号列表.....	60
表 2-11 Completion TLP Header 格式和 bypass 接口信号对应表.....	64
表 2-12 DBI 接口信号列表.....	66
表 2-13 ELBI 接口信号列表.....	69
表 2-14 SII: Legacy Interrupt 接口信号列表.....	72
表 2-15 MSI 接口信号列表.....	74



表 2-16 MSI-X 接口信号列表.....	76
表 2-17 Link Training 相关信号列表.....	79
表 2-18 VMI 接口信号列表.....	83
表 2-19 FLR 接口信号列表.....	86
表 2-20 CRI 端口列表.....	87



1 SALPHOENIX®1A100 系列 FPGA PCI Express 控制器概述

SALPHOENIX®1（以下简称 PH1A100）系列 FPGA 集成的 PCI Express 控制器硬核，可提供高带宽、可配置和高可靠性的 PCI Express 互联。本文将介绍 PH1A100 FPGA 的 PCI Express 控制器的配置方法和使用方法。下文中的 PCIe，为 PCI Express 的简称。

PH1A100 系列 FPGA 的 PH1A100（SFG676 封装）的 PCI Express 控制器和 PHY 资源如下图所示。

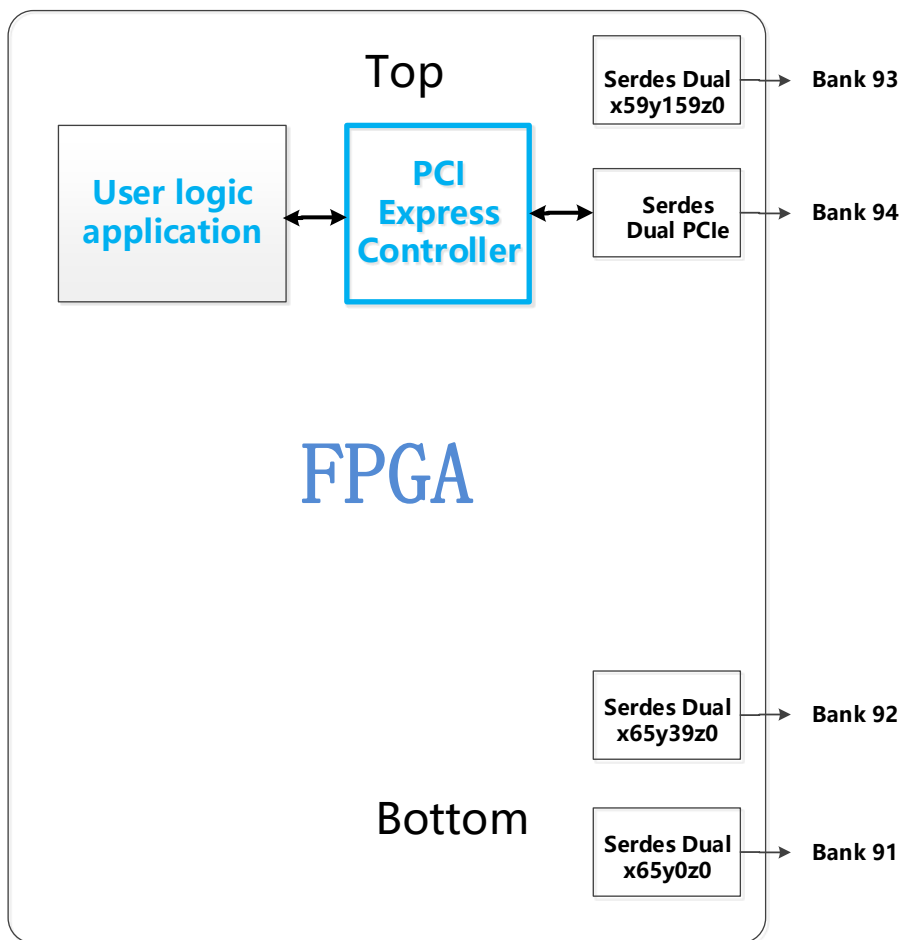


图 1-1 PH1A100（SFG676 封装）中的 PCI Express 控制器和 PHY 资源示意图

PH1A100 共有 1 个 PCI Express 控制器硬核，如图中蓝色方框所示。它通过 PIPE 接口连接 SERDES PHY（Bank 94）。PH1A100 器件中共有 4 个 SERDES DUAL，各 SERDES DUAL 的位置编号分别是 Bank 91，Bank 92，Bank 93 以及 Bank 94。其中 Bank 94 是 PCI Express 硬核专用的 PHY。PCI Express 控制器使用 Bank 94 作为它的 PHY，Bank 94 只能用于 PCI Express 控制器的 PHY，不能做其他用途。

PH1A100 FPGA PCI Express 控制器最高带宽是 8Gb/s，可配置成 1-lane 或 2-lane 的 PCI Express Endpoint (EP)。用户不能把 PCI Express 控制器配置成 2 个 1-lane 的 PCI Express。

下文中的 PCIe，为 PCI Express 的简称。



本文将介绍 PH1A100 FPGA 的 PCI Express 控制器的配置方法和使用方法。

1.1 PCIe 控制器支持的 PCIe 协议版本

本 PCIe 控制器符合 PCI Express Base Specification Revision 5.0, Version 1.0 (线速率最高支持到 Gen2), 细节请见下一节。

本 PCIe 控制器还符合 PCI Local Bus Specification, Revision 3.0、PCI Bus Power Management Specification, Revision 1.2。

1.2 PCIe 控制器支持的功能

PH1A100 系列 FPGA 的 PCIe 控制器支持以下功能：

- PCI Express Base Specification, Revision 5.0, Version 1.0 中所有的必选功能 (线速率最高支持到 Gen2)。
- 还支持以下 PCI Express Base Specification, Revision 5.0, Version 1.0 中的部分可选功能：
 - Single Root I/O Virtualization (SR-IOV)
 - Alternative Routing-ID Interpretation (ARI)
 - Address Translation Services (ATS)
 - Page Request Services (PRS)
 - Function Level Reset (FLR)
 - ID-Based Ordering (IDO)
 - TLP Processing Hints (TPH)
 - Atomic Operations
 - Dynamic Power Allocation (DPA)
 - L1 Substates (L1SS)
 - Extended Tag Support
 - Separate Refclk with Independent Spread Spectrum Clocking (SRIS)
 - Readiness Notifications (RN)
 - PCI Express Active State Power Management (ASPM)
 - PCI Express Advanced Error Reporting (AER) with Multiple Header Logging
 - Access Control Services (ACS)
- Link Width 支持 x1 或 x2, Link Speed 支持 Gen1, Gen2



- 支持先进的时钟和功耗管理 (L1 的子状态)
- 内置了地址翻译单元 (iATU, internal address translation unit)
- 支持自动 Lane 反转 (Lane Reversal)
- 支持 Upconfigure
- 支持 2 个带 ARI 或 SR-IOV 的 PF (Physical Function), 每个 PF 支持 2 个 Virtual Function
- 支持 RAS DES (Debug, Error injection, and Statistics)
- 支持 ECRC 产生和检查
- 只支持 VCO
- Max_Payload_Size 最高支持 2 KB
- 支持对 Posted 报文, Non-posted 报文和 Completion 报文实施可配置的过滤规则
- 支持基于 BAR 的地址过滤、I/O 过滤、Configuration 事务过滤和 Completion 查找表和 Timeout
- 支持 2 组用户侧发送端口 (client0 接口和 client1 接口)
- 支持带 Per-Vector Masking (PVM) 的 MSI, 支持 MSI 扩展消息数据
- 支持带 Per-Vector Masking (PVM) 的 MSI-X
- 支持 Type 0 配置空间
- RC 模式支持 Type 1 配置空间
- 如果 PCIe 控制器检测不到 Lane 0, 支持用户手动进行 Lane Reversal
- 支持通过 PHY CRI 接口访问 PHY 寄存器

1.3 PCIe 控制器设备类型

下图是一个 PCI Express 网络拓扑的例子。PCI Express 网络中有 Root Complex、Switch、PCI Express 到 PCI/PCI-X 桥、PCI Express Endpoint、Legacy (PCI Express) Endpoint。

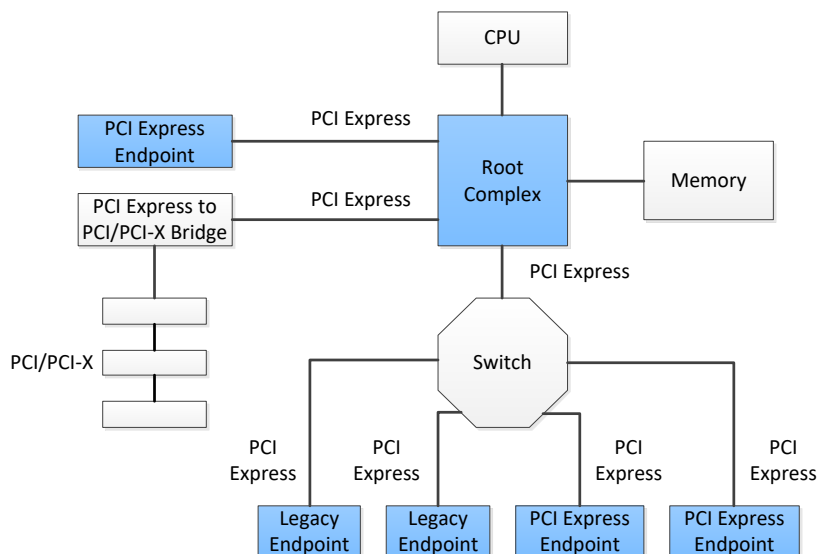


图 1-2 PCI Express 系统结构拓扑图示例

用户可以通过 IP 配置界面选择设备类型，来设置 device_type[3:0] 寄存器，将 PCIe 控制器配置成 PCI Express Root Complex、PCI Express Endpoint 或 Legacy PCI Express Endpoint。本 PCIe 控制器可以应用于上图中蓝色的设备。

表 1-1 PCI Express 控制器模式

Device_type[3:0]	PCIe Controller mode
4' b0000	PCI Express Endpoint
4' b0001	Legacy PCI Express Endpoint
4' b0100	Root Port of PCI Express Root Complex

用户根据实际应用需求,选择相应的模式。

1.4 PCIe 控制器应用领域

集成有此 PCIe 控制器 FPGA，既可应用在 PCIe 插卡，也可应用于 CPU 侧的主板，应用非常广泛。应用领域主要有：

- 数据通信网络设备
- 电信网络设备
- 存储设备
- 宽带有线和无线设备
- 交叉互联设备和交换机
- 板上芯片间互联和背板互联



- 无线基站设备等

1.5 PCIe 控制器用户侧数据位宽和时钟频率

对于各种线速和通道的配置，用户侧的数据线宽和时钟频率如下图所示。

表 1-2 PCIe 控制器带宽、用户侧数据位宽和时钟频率对照表

PCI Express Gen	Line Rate (b/s)	Lane Number	Application Data Width (bit)	Application Clock (MHz)
Gen1	2.5G	1-lane	64	62.5
		2-lane		
Gen2	5.0G	1-lane	64	125
		2-lane		

1.6 PCIe 控制器内部框图

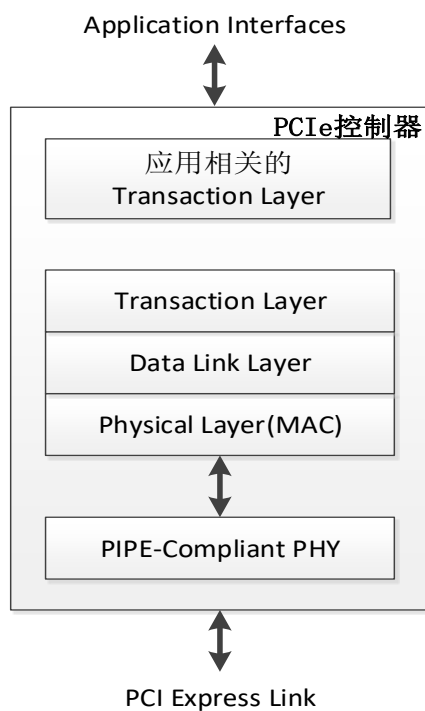


图 1-3 PCI Express 控制器内部框图

PCIe 控制器内部框图如图所示，可以分为以下几个模块：

■ 事务层 Transaction Layer

包括基本的 Transaction Layer 和应用相关的 Transaction Layer 两部分。应用相关的 Transaction Layer 主要包括 RADM, XADM, CDM 等模块。这些模块将在下文中详细介绍。事务层模块在发送方向的功能有：



- TLP 仲裁
- TLP 组包
- 流控信用检查

事务层模块在接收方向的功能有：

- 把收到的 TLP 进行分类和过滤。其中过滤规则和路由是可配置的。
- 缓存收到的 TLP
- 把收到的 TLP 转发到用户侧接口

本模块还实现 Configuration 事务访问、电源管理、内部寄存器和外部应用寄存器的访问、Message 生成等。

■ 数据链路层 Data Link Layer

本模块负责把来自事务层的 TLP 可靠、完整地跟对端的 Data Link Layer 交换信息。

本模块主要功能有初始化、电源管理、数据保护、错误检测和重发等。

■ 除 PIPE PHY 以外的物理层 Physical Layer (MAC 部分)

本模块和 PIPE-Compliant PHY 一起实现了 PCI Express 的物理层。

本模块主要功能是 LTSSM (Link Training and Status State Machine) 状态机和 PLP (Physical Layer Packet) 组包拆包。Data Link Layer 使用通过本模块进行链路训练、管理链路状态、实现链路恢复和 Power Management。

■ 符合 PIPE 规范的物理子层 PIPE-Compliant PHY

本模块符合 PIPE Specification for PCI Express, Version 4.4.1。本模块执行对端检测、复位、链路初始化训练、电源管理，实现数据和控制字符的编解码，支持 Compliance pattern 测试。

本模块跟 PCI Express Link 的接口为 PCIe 高速串行差分对，跟 Physical Layer 模块的接口为 PIPE。SERDES DUAL Bank 94 是本模块的一部分。

1.7 PCIe 控制器 BAR 空间

本控制器有 2 个 PF (Physical Function)，每个 PF 支持 2 个 VF (Virtual Function)。每个 Function 可以有最多 3 个 BAR 空间，分别为 BAR0，BAR2 和 BAR4。这三个 BAR 空间可以配置成 32 位 BAR 空间，也可以配置成 64 位 BAR 空间。除了 3 个 BAR 空间之外，每个 Function 还支持一个 Expansion ROM 空间。

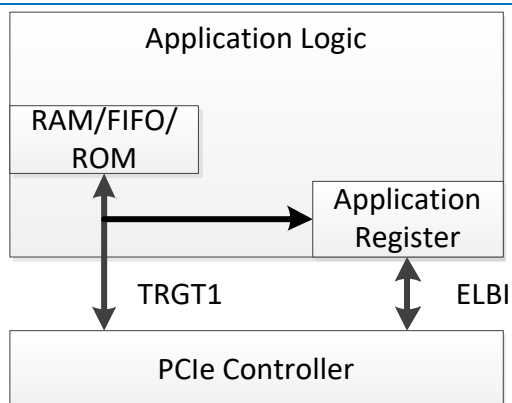


图 1-4 Memory/I/O 事务访问路径

用户逻辑里的 Application Register，可以分配到 BAR0，BAR2，BAR4 等空间。其中，BAR0 是通过 ELBI 总线访问的；BAR2 和 BAR4 是通过 TRGT1 总线访问的。

用户逻辑里的 RAM/FIFO/ROM 等，也可以分配到 BAR0，BAR2，BAR4 等空间。其中 BAR0 是通过 ELBI 总线访问的；BAR2 和 BAR4 是通过 TRGT1 总线访问的。

BAR0，BAR2 和 BAR4 的特性如下表：

表 1-3 BAR0，2，4 的特性列表

特性	BAR0	BAR2	BAR4
默认打开/关闭	打开		
用户存储类型	Application Register	Application Register 或 RAM/FIFO/ROM	
访问接口	ELBI	TRGT1	
默认 Memory/I/O	Memory		Memory (EP) IO (Legacy EP)
Prefetchable /Non- prefetchable	默认都是 Non-Prefetchable，可以配置成 prefetchable		
是否允许用户配置 大小	是		

1.8 寄存器配置空间 (Configuration Space)

1.8.1 配置空间

PCI Express Base Specification 里，配置空间地址范围为 0x000~0xFFF。其中，0x000-0xFF 为 PCI 配置空间，0x100~0xFFF 为 PCI Express 扩展能力寄存器地址空间。

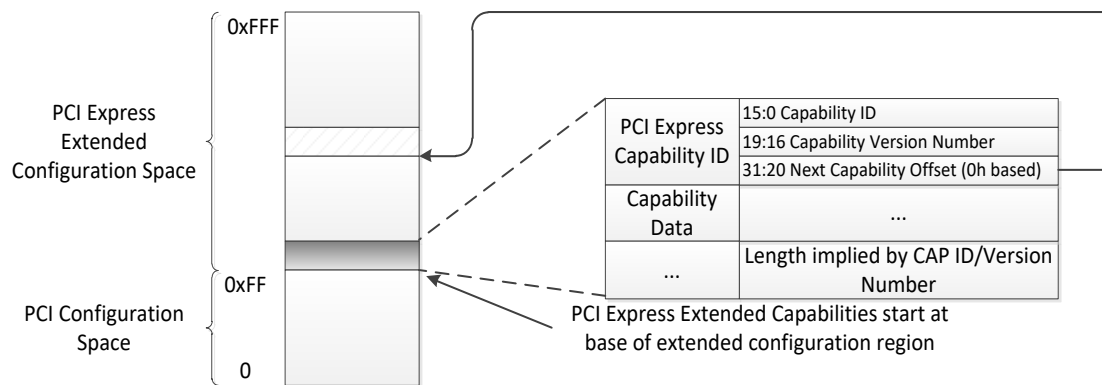


图 1-5 PCI Express 扩展配置空间地址分配

在 PH1A100 系列 FPGA 的 PCIe 控制器中，配置空间分成 3 个部分，各部分的地址范围和类型如下表：

表 1-4 PCIe 控制器能力寄存器配置空间

地址范围	寄存器类型
0x0~0x6FF	PCIe 配置空间
0x700~0xCFF	PCIe 控制器自定义寄存器空间
0xD00~ 0xFFFF	保留

PCIe 控制器分配给每个 Function 的能力寄存器配置空间地址范围是 0x0 ~ 0x6FF。

1.8.2 PCIe 能力寄存器列表

表 1-5 PCIe 控制器支持的 PCIe 能力寄存器列表

Capability ID	Capability Description	PF	VF
01h	PCI power management interface	支持	
05h	MSI	支持	
10h	PCI Express	支持	支持
11h	MSI-X	支持	支持

1.8.3 PCIe 扩展能力寄存器

PCIe 控制器支持的 PCIe 扩展能力寄存器列表

表 1-6 Extended Capability 寄存器

Extended Capability ID	Capability Description	PF	VF
16'h0001	Advanced Error Reporting (AER)	支持	



Extended Capability ID	Capability Description	PF	VF
16'h0003	Device Serial Number	支持	
16'h000b	Vendor-specific (VSEC) (RAS_DES)	支持	
16'h000c	Configuration Access Correlation		
16'h000d	ACS	支持	支持
16'h000e	ARI	支持	支持
16'h000f	ATS	支持	
16'h0010	SR-IOV	支持	
16'h0013	Page Request (PRS)	支持	
16'h0016	Dynamic Power Allocation (DPA)	支持	
16'h0017	TPH Requester	支持	
16'h0018	LTR	支持	
16'h0019	Secondary PCIe	支持	
16'h001e	L1 PM Substates	支持	
16'h0022	Readiness Time Reporting (RTR)	支持	支持
16'h0025	Data Link Feature	支持	
16'h0026	Physical Layer 16.0 GT/s	支持	
16'h0027	Lane Margining at the Receiver	支持	

1.8.4 PCIe 寄存器的本地地址映射

PCIe 控制器给每个支持的能力寄存器分配了默认的 DBI 总线地址, 以及用于能力寄存器连接的 next pointer 地址。(DBI 总线的介绍, 请见《DBI 接口》章节。)

表 1-7 PF 本地地址映射表

属性	DBI Address[11:0] for PF0	DBI Address[11:0] for PF1
Type 0 Header	0x0	0x0
PM	0x40	0x40
MSI	0x50	0x50
PCIe	0x70	0x70
MSIX	0xB0	0xB0
AER	0x100	0x100
SN	0x148	0x148
ARI	0x158	0x158
SPCIE	0x168	—
PL16G	0x178	—



MARGIN	0x19C	—
SRIOV	0x1AC	0x1AC
TPH	0x1EC	0x1EC
ATS	0x278	0x278
ACS	0x288	0x288
PRS	0x294	0x294
LTR	0x2A4	—
L1SUB	0x2AC	—
DPA	0x2BC	0x2BC
RTR	0x2FC	0x2FC
RAS_DES	0x308	0x308
DLINK	0x408	—
PORT_LOGIC	0x700	—

表 1-8 VF 本地地址映射表

属性	DBI Address[11:0]
Type 0 Header	0x0
PM	—
MSI	—
PCIE	0x70
MSIX	0xB0
AER	—
SN	—
ARI	0x158
SPCIE	—
PL16G	—
MARGIN	—
SRIOV	—
TPH	—
ATS	—
ACS	0x288
PRS	—
LTR	—
L1SUB	—
DPA	—
RTR	0x2FC
RAS_DES	—
DLINK	—



PORT_LOGIC	—
------------	---

所有 PF 和 VF,都是用 BDI 总线地址的低 12 位进行访问。PF0 和 PF1,用 DBI 端口 drp_dbi_func_num 设置。PF0 的 2 个 VF 和 PF1 的两个 VF,用 DBI 的 drp_dbi_func_num 和 drp_dbi_vfunc_num 联合设置。

2 PH1A100 系列 FPGA PCI Express 控制器信号接口

PH1A100 系列 FPGA PCIe 控制器主要功能模块和接口如下图所示。

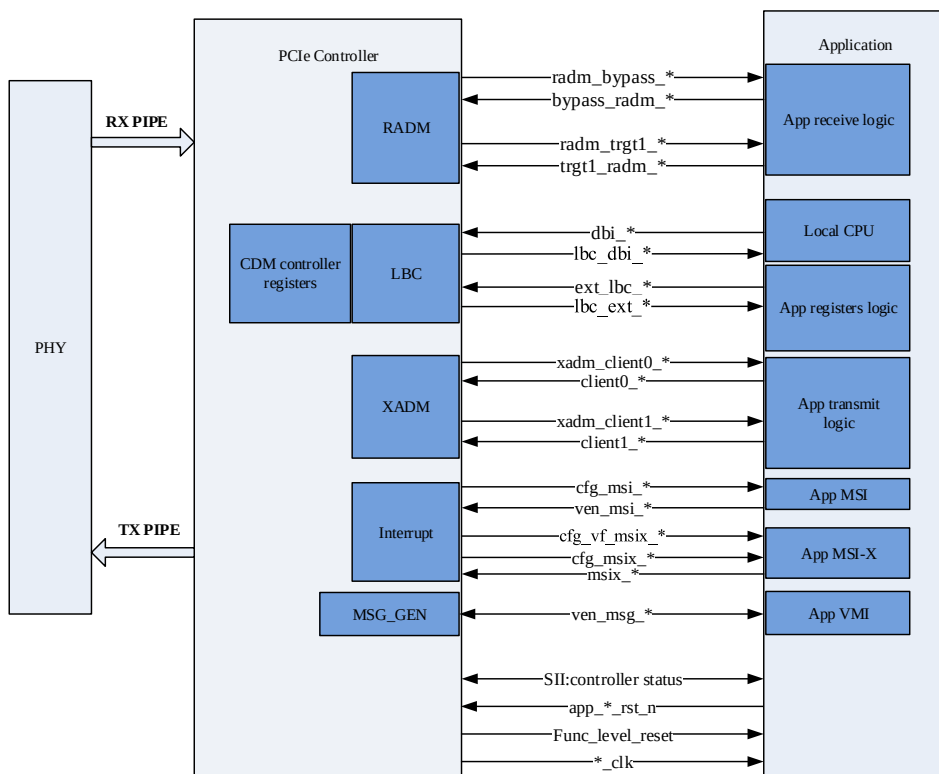


图 2-1 PCIe 控制器的主要模块和接口

PH1A100 系列 FPGA PCIe 控制器主要实现了 PCI Express 物理层 (Physical Layer)，链路层 (Link Layer) 和事务层 (Transaction Layer) 的基本功能。它实现了事务层逻辑的大部分功能，全部的数据链路层逻辑，以及物理层的 MAC 部分 (包括 Link Training and Status State Machine，即 LTSSM)。它通过 PIPE 接口连接到 PHY。

PCIe 控制器主要包括的功能模块有：

1) Transmit Application-Dependent Module (XADM)

它实现了和应用相关的 PCIe 事务层的功能用于传输数据报文。其功能包含

- TLP 仲裁 (Arbitration)
- TLP 组包 (Formation)
- 流控信用 (Flow Control Credit) 检查

发送路径使用 cut-through 结构，它没有实现发送缓冲队列 (有 retry buffer)。当此接口发送 CPL 报文时，控制器提供一个可选的功能，即通过维护一个内部的“目标完成包查找表”(Target Completion



Lookup Table) 去存储收到的 Request TLP 的一部分包头信息。用户逻辑可以用这些信息来发送完成包。详情参见《目标完成包查找表》这一章节。

2) Receive Application-Dependent Module (RADM)

它实现了和应用相关的 PCIe 事务层的功能，用于接收数据报文。其功能包含：

- 对接收到的 TLP 包进行排序和过滤。过滤的规则和路由是可以配置的。
- 缓存接收到的 TLP 报文。
- 路由接收到的 TLP 到相应接口。RADM 维护一个接收完成包查找表，用于追踪完成包，对于发出的 non-posted 请求监视完成包超时 (completion timeout)。如果期望的 RX 完成包未能在预定时间之前到达，它会指示超时。

3) Configuration-Dependent Module (CDM)

CDM 实现了下述功能：

- 标准的 PCI Express 配置空间 (Configuration Space)
- 控制器特有的寄存器空间

4) Power Management Controller (PMC)

PMC 实现了 Power Management 功能。

5) Local Bus Controller (LBC) 和 Data Bus Interface (DBI)

LBC 模块提供了对端器件或本地 CPU 通过 DBI 接口去访问下述寄存器的功能：

- CDM 内部寄存器
- 连接到外部 ELBI 接口的用户逻辑寄存器

6) Message Generation Module (MSG_GEN)

MSG_GEN 模块为控制器发送 message 报文。

对应上述功能模块，PH1A100 系列 FPGA PCIe 控制器的主要接口有：

- Transmit Client 0 Interface (XAL10)
信号名为 client0_*, xadm_client0_*。
- Transmit Client 1 Interface (XAL11)
信号名为 client1_*, xadm_client1_*。
- Receive Target 1 (TRGT1)



信号名为 `radm_trgt1_*`, `trgt1_radm_*`。

- Receive Bypass Interface (RBYP)

信号名为 `radm_bypass_*`。

- Data Bus Interface (DBI)

信号名为 `drp_dbi_*`, `drp_lbc_dbi_*`。

- External Local Bus Interface (ELBI)

信号名为 `lbc_ext_*`, `ext_lbc_*`。

- Message Signaled Interrupt (MSI) Interface

信号名为 `cfg_msi_*`, `ven_msi_*`

- MSI-X Interface

信号名为 `cfg_msix_*`, `cfg_vf_msix_*`, `msix_*`

- Vendor Message Interface (VMI)

信号名为 `ven_msg_*`

- System Information Interface (SII)

该接口为可选接口，可以不使用。不使用时输入端口接地。

- PIPE

PHY 层接口

另外，PCIe 控制器还在 SII 接口中提供传输状态接口，比如：SII: Receive Control, SII:CPL Timeout Signals, SII: Transmit Control Signals 等。

用户常用收发报文接口如下图所示。

XALIO 和 XALI1 用于 TLP 报文发送，TRGT1 和 RBYP 用于 TLP 报文接收。

XALIO 和 XALI1 是功能相同的两套发送接口，既可以发请求包 (Request TLP)，也可以发完成包 (CPL TLP)，这两组端口的功能是完全相同的。下图的示例中，XALIO 被用于发送 CPL TLP，而 XALI1 被用来发 Request TLP，实际使用时可以不局限于这一种用法。PCIe controller 的 XALIO 和 XALI1 接口不对用户的输入数据进行缓存，仅通过 “*halt” 信号进行反压控制。

TRGT1 用于接收请求包 (Request TLP)，包括 Posted TLP 和 Non Posted TLP。控制器内部分配了接收缓存 (Receive Buffer)。RBYP 用于接收 CPL TLP，控制器内部未分配 Receive Buffer，该接口为直通 (Bypass) 模式。

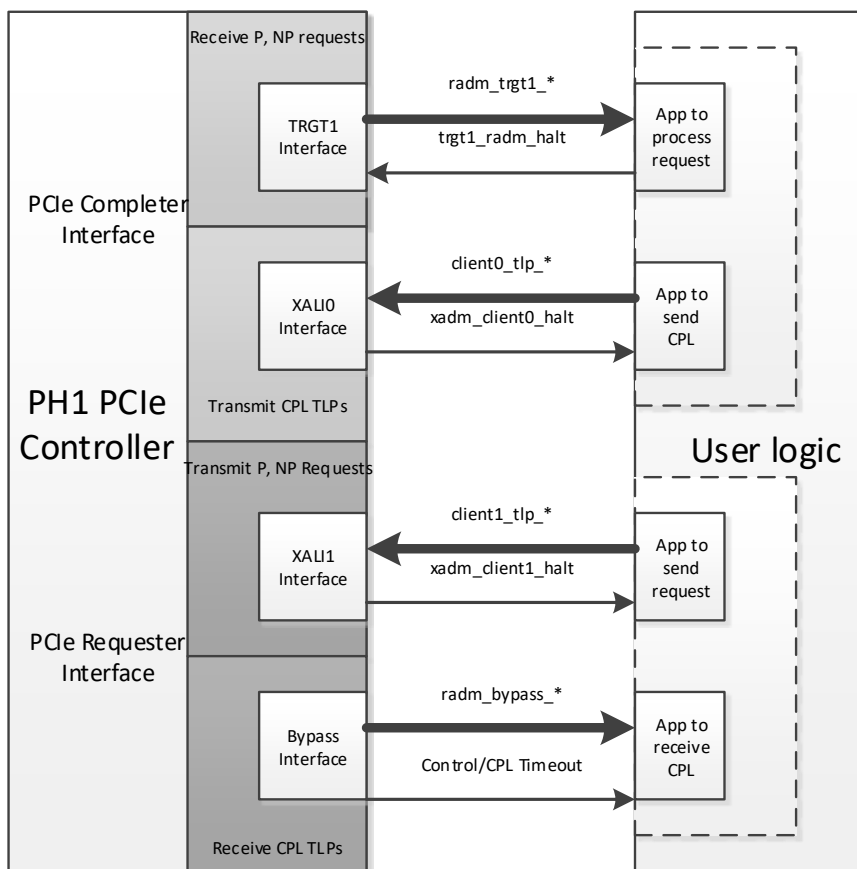


图 2-2 PCIe 控制器主要接口用法示例

下文分别介绍各种 TLP 包的路由方式。

CFG Request 路由方式:

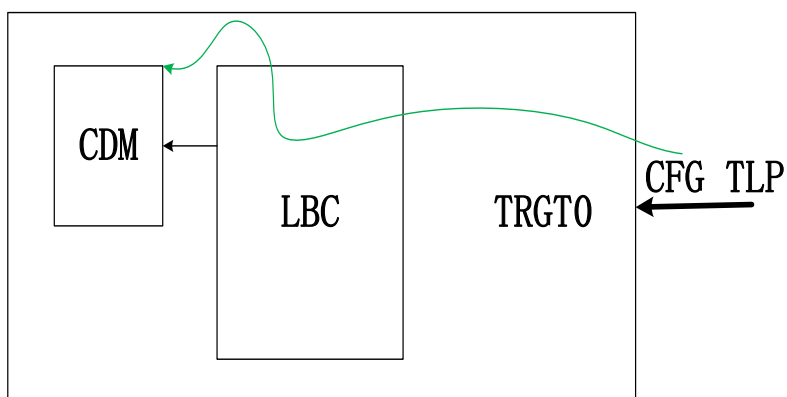


图 2-3 CFG Request TLP 路由

控制器接收到 CFG TLP 报文后会映射到 TRGT0 接口，该接口为内部接口，经 LBC 到达 CDM。

MEM/IO Request TLP 路由方式:

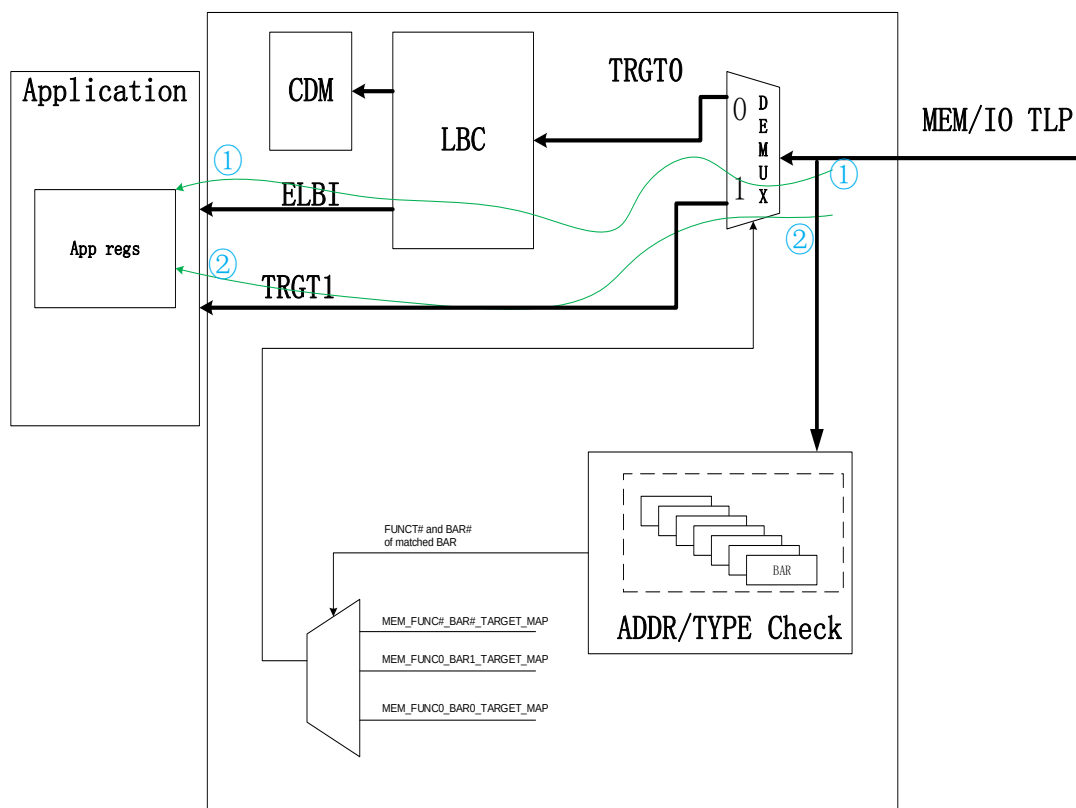


图 2-4 MEM 和 IO 操作路由图

MEM/IO Request TLP 有 2 种路由方式，如上图所示。第一种方式是通过 BAR 空间映射到 TRGT0 空间，经由 LBC 映射到 ELBI 接口，访问到 PCIe 控制器外部的用户逻辑寄存器。第二种方式是通过 BAR 空间映射到 TRGT1 接口访问到 PCIe 控制器外部的用户逻辑寄存器。

2.1 时钟和复位接口

2.1.1 功能简介

PCIe 控制器支持三种复位方式：Cold reset, Warm reset 和 Hot reset。

2.1.2 端口

时钟和复位相关信号列表如下。



表 2-1 时钟和复位信号列表

Port	Width	I/O	Description
app_perst_n ^{1, 2}	1	I	上位机 PERST#复位, 即 Warm Reset, 如果辅助电源 Vaux 在位, 通过板级的 PERST#复位; 否则从板级 Power On Reset 信号来复位。低有效。
app_power_up_rst_n ¹	1	I	Cold Reset, 用户板级 Power On Reset 复位, 即上电复位。低有效。若外部参考时钟异常, 需要在参考时钟恢复后, 通过该信号复位 PCIe。
app_button_rst_n ¹	1	I	用户板级按键复位, 可以接按键, 低有效。
app_app_ltssm_enable	1	I	用户启动 LTSSM 使能信号。用户逻辑在 cold reset, warm reset 或 hot reset 之后, 拉低该信号可以将 LTSSM 保持在 Detect 状态, 直到用户逻辑准备好开始 Link Training 之后才拉高该信号, 开始 Link Training 过程。当用户逻辑完成通过 DBI 接口对控制器的配置寄存器进行编程后, 拉高此信号允许 LTSSM 进行建链。该信号也可以被用来延时 hot reset, 直到用户已经读出所有寄存器状态。高有效
core_rst_n	1	O	PCIe 控制器输出给用户逻辑的复位信号, 用户使用此信号来复位 PCIe 相关接口逻辑。上电时该信号应拉低, 用于复位辅助电源域之外的逻辑。当 link_req_rst_not 发生从 high 到 low 的跳变时, 复位逻辑应拉低 core_rst_n。推荐用户逻辑的复位和控制器的复位同步进行。异步复位同步解复位。低有效。
core_clk	1	O	PCIe core 时钟, 用于驱动用户 core_clk 时钟域接口信号逻辑。此为控制器主时钟。该时钟来自于 pipe_clk。pipe_clk 是参考时钟通过 PLL 产生的。
muxd_aux_clk	1	O	PCIe 控制器输出时钟, 在低功耗状态下用此时钟来驱动逻辑。在低功耗模式下, 当 PHY 不提供时钟时, muxd_aux_clk 由用户从 aux_clk 提供; 当 PHY 正常工作时, muxd_aux_clk 由 PCIe 控制器输出的 core_clk 来产生, 此时用户一般使用 core_clk 来驱动 PCIe 相关接口逻辑。
muxd_aux_clk_g	1	O	muxd_aux_clk_g 是和 muxd_aux_clk 相同的时钟, 区别在于 muxd_aux_clk_g 受到 power gate 的影响, 在 L1.2 状态时该时钟无输出。
app_auxclk	1	I	用户低功耗模式下提供给 PCIe 控制器的工作时钟, 同时也用于 PCIe 的上电加载配置的时钟。该时钟为管脚输入的时钟, 且在加载前就必须保持稳定, app_auxclk 的频率范围参见 DS700_PH1A100_Datasheet。
smlh_link_up	1	O	物理层建链成功信号。 时钟域: core_clk 高有效
smlh_req_rst_not	1	O	link_req_rst_not 的 early version
rdlh_link_up	1	O	Data link 层链接指示信号。高有效。



Port	Width	I/O	Description
link_req_rst_not	1	0	指示复位请求。低电平指示控制器由于某种原因请求外部用户逻辑复位控制器。一般是物理层断链或控制器接收到一个 Hot Reset 请求。时钟域: core_clk 低有效。
app_clk_req_n	1	1	表示用户逻辑已经准备好，系统可以移除参考时钟。在支持参考时钟移除（reference clock removal）的设计中，例如 Power State 在 L1 PM Sub-state 或者 L1 CPM 状态时，当用户逻辑已经做好准备参考时钟可以被移除时将此信号置为 1'b1。若用户逻辑不想移除参考时钟将此信号置为 1'b0。 时钟域: aux_clk 低有效
radm_idle	1	0	RADM 活跃状态信号。调试专用。 时钟域: core_clk 高有效
app_init_rst	1	1	RC 端用户逻辑发送 Hot Reset Request 到 upstream port。Hot Reset Request 在这个信号拉高单个时钟周期后发出。在 EP 模式下该信号接 1'b0。 时钟域: core_clk 高有效
training_rst_n ²	1	0	表示接收到来自 upstream component 的 Hot Reset Request。当控制器的 LTSSM 接收到连续 2 个 hot_reset 位为 1'b1 的 TS1 Ordered Sets 以后，这个 training_rst_n 端口会被拉高 1 个时钟周期。这个信号是为兼容 Legacy 用法而保留的，用户逻辑在收到 Hot Reset Request 以后应使用 link_req_rst_not 去复位控制器。 时钟域: core_clk 低有效

注：1. 该信号有效时会复位 PCIe 的寄存器，需要用户对被复位后的寄存器重新配置为所需的值。

2. 该端口只在 EP 模式下使用，RC 模式无效。

2.1.3 Link Training 和 LTSSM 相关用法

刚一上电，PCIe 控制器可以进入 EP 模式，Legacy EP 模式或者 RC 模式三种模式其中之一，取决于 device_type[3:0] 的输入值。初始化过程如下图所示。用户逻辑通过控制 app_ltssm_enable 来开始 Link Training 过程。在 cold reset, warm reset 和 hot reset 之后，用户逻辑将 app_ltssm_enable 驱动为低保持 LTSSM 在 Detect 状态，直到用户逻辑认为可以开始 Link Training。当用户逻辑完成了对控制器的配置寄存器的 DBI 编程，拉高 app_ltssm_enable 信号以允许 LTSSM 继续进行建链的尝试。该机制还可以被用来延迟 hot reset，直到用户逻辑已经读出所有状态寄存器。

使用 app_ltssm_enable 来延迟 Link Training 的方法有以下几种情况：



Cold Reset 情况

使用 `app_ltssm_enable` 可以保持 LTSSM 状态并延迟 Link Training，以便用户逻辑可以通过 DBI 接口编程一些寄存器。

• 用户逻辑在 Power-On Reset (`app_power_up_rst_n`) 解复位之前设置 `app_ltssm_enable` 为 0。最好的方式是上电后或 `core_rst_n` 有效时直接设置 `app_ltssm_enable = 0`。

- 等待，直到 `core_rst_n` 解复位。
- 根据需要通过 DBI 接口写寄存器。
- 设置 `app_ltssm_enable = 1`。
- Link Training 开始启动。

Hot Reset (Link Down Reset) 情况

使用 `app_ltssm_enable` 可以延迟控制器复位，以便于用户逻辑可以通过 DBI 读取所需的寄存器。

- 在 `smlh_req_rst_not` 下降沿后立即设置 `app_ltssm_enable = 0`。
- 保持 `app_ltssm_enable = 0`。
- 通过 DBI 读取寄存器。
- 设置 `app_ltssm_enable = 1`。
- 控制器开始复位。

使用 `app_ltssm_enable` 可以保持 LTSSM 并延迟 Link Training 过程，以便于用户逻辑可以通过 DBI 编程一些寄存器。

- 在 `core_rst_n` 下降沿立即设置 `app_ltssm_enable = 0`。
- 通过 DBI 写寄存器。
- 设置 `app_ltssm_enable = 1`。
- Link Training 开始。

对于 Hot Reset，用户可以做上述两种延迟(delay reset and/or delay link training)或其中一种。如果两种都做，必须按照上文中介绍的顺序。`app_ltssm_enable` 设置为 0 必须在 LTSSM=Detect 时候做，在 LTSSM 不在 Detect 状态时若将 `app_ltssm_enable` 设置为 0 则会导致控制器被复位，且 LTSSM 立即回到 Detect 状态，这可能会导致和 PIPE 协议有冲突。

下图表示上电过程中，PCIe 初始化过程。用户逻辑可以通过控制 `app_ltssm_enable` 信号，来延迟 LTSSM 和 Link Training 过程。

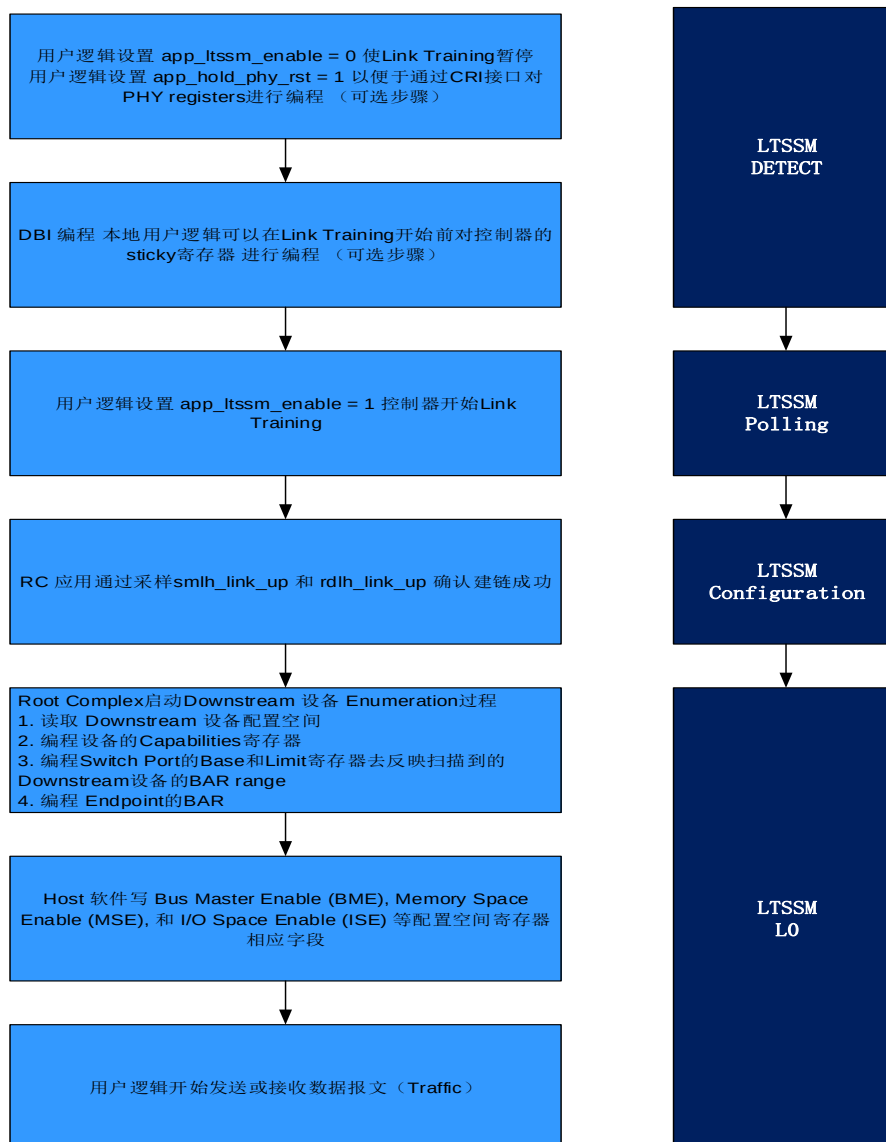
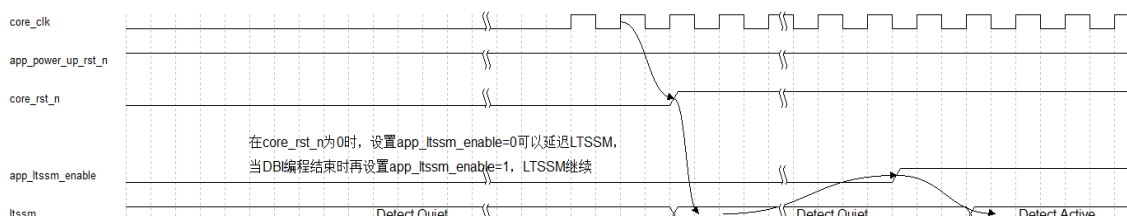


图 2-5 PCIe 初始化过程

2.1.4 时序图

用 `app_ltssm_enable` 延时 Link Training 过程的时序图如下。

图 2-6 用 `app_ltssm_enable` 延时 LTSSM

2.2 系统信息接口 (SII)



2.2.1 功能简介

SII (System Information Interface) 是用于控制器和用户逻辑之间交换大量的系统信息的接口。其中绝大部分接口是为了提高灵活性而提供的可选接口。用户逻辑不一定会用到所有的接口，只需要用到保证功能正确的一部分。当 SII 接口不用时，输入的端口必须接 0。

2.2.2 端口

本文仅提供了部分 SII 接口的端口介绍。另在下文中还介绍了 Legacy Interrupt 的相关端口，该端口也属于 SII 接口的一部分，详情参见《SII: Legacy Interrupt》章节。

表 2-2 SII 的部分端口列表

Port	Width	I/O	Description
cfg_bus_master_en	2	0	PCI-compatible Command 寄存器中的 Bus Master Enable 位状态。每个 PF 对应一位。时钟域: aux_clk_g 高有效
cfg_mem_space_en	2	0	PCI-compatible Command 寄存器中的 Memory Space Enable 位状态。每个 PF 对应一位。时钟域: aux_clk_g 高有效
cfg_pbus_num	8	0	对应 Function 的 Primary Bus Number。时钟域: aux_clk_g 高有效
cfg_pbus_dev_num	5	0	对应 Function 的 Device Number。时钟域: aux_clk_g 高有效
prs_res_failure ¹	2	1	用户逻辑通过脉冲指示对应的 PF (或其关联的 VF) 接收到一个 PRG Response Failure。作用是将 Page Request Status 寄存器 (PRS_CONTROL_STATUS_REG) 的 PRS_RESP_FAILURE 域置位。时钟域: aux_clk_g 高有效
prs_uprgi ¹	2	1	用户逻辑通过脉冲指示对应的 PF (或其关联的 VF) 接收到一个 Unexpected Page Request Group Index 的响应。作用是将 Page Request Status 寄存器 (PRS_CONTROL_STATUS_REG) 的 PRS_UPRGI 域置位。时钟域: aux_clk_g 高有效
prs_stopped ¹	2	1	用户逻辑通过脉冲信号指示对应的 PF (或其关联的 VF) 已经完成了所有前面发起的 Page Requests 并停止了更多后续的 page requests。只有在 PRS Enable 位清空时有效。作用是将 Page Request Status 寄存器 (PRS_CONTROL_STATUS_REG) 的 PRS_STOPPED 域置位。时钟域: aux_clk_g 高有效



Port	Width	I/O	Description
diag_ctrl_bus	3	I	诊断控制总线。按位编码映射关系如下： ■ x01: 通过反转 LCRC 的 LSB 插入 LCRC 错误 ■ x10: 通过反转 ECRC 的 LSB 插入 ECRC 错误 □ 这 2bit 信号([1:0]) 的上升沿使得控制器对正在传输的包使能了 LCRC 或 ECRC □ 当 LCRC 或 ECRC 出错的包被传输完以后，控制器拉高 diag_status_bus 中的[lrc_err_asserted]位或[ecrc_err_asserted] 位，报告该报文已经完成。这个 control 和 status 之间的握手机制使得用户逻辑能够控制对特定的包插入 LCRC 或 ECRC 错误。 □ LCRC 和 ECRC 错误的产生是对 LCRC 和 ECRC 值的 LSB 反转来实现的。 ■ 1xx: 选择快速建链模式 □ 设置 LTSSM 内部的 ms 级计时器到快速模式，用于加速仿真过程。强制 LTSSM 训练或建链过程使用更短的 timeouts 来更快的建链。 □ 快速建链模式也可以通过对 PORT_LINK_CTRL_OFF 寄存器的 FAST_LINK_MODE 域置位来激活。 时钟域: core_clk 高有效
diag_status_bus_sel	6	I	保留
diag_status_bus	16	O	保留
rx_lane_flip_en	1	I	对接收 Lanes 进行手动 Lane Reversal。Lane0 如果未能检测到，自动 Lane Reversal 不会发生，此时可以用此信号触发手动 Lane Reversal。时钟域: core_clk 高有效
tx_lane_flip_en	1	I	对发送 Lanes 进行手动 Lane Reversal。Lane0 如果未能检测到，自动 Lane Reversal 不会发生，此时可以用此信号触发手动 Lane Reversal。时钟域: core_clk 高有效
cfg_vf_bme	4	O	每个 VF 的 Control Register 中的 Bus Master Enable 位。每一个 bit 对应一个 VF。时钟域: aux_clk_g
app_sris_mode	1	I	SRIS (Separate Refclk Independent SSC) 模式选择，按位编码映射如下： ■ 0b: non-SRIS 模式 ■ 1b: SRIS 模式
app_req_retry_en	1	I	给控制器提供了延迟处理收到的配置请求 (Configuration Requests)，直到初始化已经完成。当 app_req_retry_en 拉高时，控制器对接收到的 Configuration Requests 返回 retry 状态。该功能不影响其他类型的请求。时钟域: core_clk 高有效



Port	Width	I/O	Description
app_pf_req_retry_en ¹	2	I	给控制器的某个或某些 PF 提供了延迟处理收到的配置请求（Configuration Requests），直到初始化已经完成。当 app_pf_req_retry_en 拉高时，控制器对接收到的 Configuration Requests 返回 retry 状态。该功能不影响其他类型的请求。app_pf_req_retry_en 的优先级低于 app_req_retry_en，当 app_req_retry_en 拉高时，所有的 PF 和 VF 都会对输入的配置请求返回 retry 状态，无论 app_pf_req_retry_en 或者 app_vf_req_retry_en 是否拉高。时钟域：core_clk 高有效
app_vf_req_retry_en ¹	4	I	给控制器的某个或某些 VF 提供了延迟处理收到的配置请求（Configuration Requests），直到初始化已经完成。当 app_vf_req_retry_en 拉高时，控制器对接收到的 Configuration Requests 返回 retry 状态。该功能不影响其他类型的请求。app_vf_req_retry_en 的优先级低于 app_req_retry_en，当 app_req_retry_en 拉高时，所有的 PF 和 VF 都会对输入的配置请求返回 retry 状态，无论 app_pf_req_retry_en 或者 app_vf_req_retry_en 是否拉高。时钟域：core_clk 高有效
app_dbi_ro_wr_disable	1	I	控制 DBI_RO_WR_EN 是否可读可写。按位编码映射如下： ■ 0: MISC_CONTROL_1_OFF. DBI_RO_WR_EN 可写可读 ■ 1: MISC_CONTROL_1_OFF. DBI_RO_WR_EN 不可写 时钟域：aux_clk_g 高有效

注：1. 该端口只在 EP 模式下使用，RC 模式无效。

2.3 发送接口 XALIO and XALI1

2.3.1 功能简介

XALIO（Transmit Client 0 Interface）是用户逻辑发送 outbound requests 和 CPL TLP 的接口。XALI1（Transmit Client 1 Interface）是额外提供的接口，和 XALIO 相同。XALIO 和 XALI1 可以同时使用，控制器的 XADM 在内部提供仲裁。

Client0/1 接口为 cut-through 模式，除 retry buffer 之外不对数据包进行缓存。不支持用户逻辑在发包过程中暂停，用户逻辑必须连续不间断的发送一个完整的 TLP 报文，如果用户发送不连续的数据包，会导致 Client0/1 接口发送错误的报文。

2.3.2 端口



XALIO 和 XALI1 接口信号列表如下。

表 2-3 XALIO 和 XALI1 接口信号列表

Port Name	Width	I/O	Description
client0_addr_align_en	1	I	使能“发送地址对齐 (Transmit Address Alignment)”模式。具体信息见《发送地址对齐》(Transmit Address Alignment) 章节。对于用户逻辑产生 ECRC 的 TLP，不能使用该模式。只有在用户 TLP 中没有 ECRC 时，才可以使能。所有 CPL TLP 都不能使能该模式。 时钟域: core_clk 高有效 当 client0_tlp_hv 拉高时有效
client0_tlp_byte_en	8	I	TLP 中第一个 Dword 和最后一个 Dword 的 Byte Enable 信号。Bit[3:0] 对应第一个 Dword; Bit[7:4] 对应最后一个 Dword。在“发送地址对齐”模式下，应设置为 0x0。当发送 Message 报文的时候，用户逻辑把 Message Code 赋值到该端口。 时钟域: core_clk 高有效 当 client0_tlp_hv 拉高时有效，当 client0_addr_align_en 拉高时无效
client0_remote_req_id	16	I	Completion 报文 Header 中的 Requester ID 字段。必须和接收到的 Request 中的 Requester ID 相同。按位编码映射如下： ● Bit[15:8]: Bus number ● Bit [7:3]: Device number ● Bit [2:0]: Function number 时钟域: core_clk 高有效 当 client0_tlp_hv 拉高时有效
client0_cpl_byte_cnt	12	I	CPL TLP Header 中的 Byte Count 字段。控制器不读取 client0_cpl_byte_cnt 端口，而是使用 target completion LUT entry 中的 Byte Count。 时钟域: core_clk 当 client0_tlp_hv 拉高时有效
client0_tlp_tc	3	I	TLP Header 中的 Traffic Class (TC) 字段。 时钟域: core_clk 当 client0_tlp_hv 拉高时有效
client0_tlp_attr	3	I	TLP Header 中的 Attribute 字段。当“目标完成包查找表模式”默认为使能状态，控制器不读取 client0_tlp_attr 端口，而是使用 target completion LUT entry 中的 Attribute 值。 时钟域: core_clk 当 client0_tlp_hv 拉高时有效
client0_cpl_status	3	I	CPL TLP Header 中的 Status 字段。 时钟域: core_clk 当 client0_tlp_hv 拉高时有效
client0_cpl_bcm	1	I	CPL TLP Header 中的 Byte Count Modified (BCM) 字段。应接 0。 时钟域: core_clk 当 client0_tlp_hv 拉高时有效
client0_tlp_dv	1	I	指示 client0_tlp_data 有效。 时钟域: core_clk



Port Name	Width	I/O	Description
client0_tlp_eot	1	I	TLP 包尾指示, 如果有 Payload 数据, 指示 payload 结束周期; 如果没有 payload, 和 client0_tlp_hv 同时拉高拉低。 时钟域: core_clk 高有效
client0_tlp_bad_eot	1	I	指示当前 TLP 被终止 (nullified)。和 client0_tlp_eot 同时拉高, 使得当前 TLP 被终止。 时钟域: core_clk 高有效
client0_tlp_hv	1	I	指示 TLP Header 有效。 时钟域: core_clk 高有效
client0_tlp_fmt	2	I	TLP Header 中的 Format 字段。 时钟域: core_clk 当 client0_tlp_hv 拉高时有效
client0_tlp_type	5	I	TLP Header 中的 Type 字段。 时钟域: core_clk 当 client0_tlp_hv 拉高时有效
client0_tlp_td	1	I	TLP Header 中的 TLP Digest (TD) 字段。 时钟域: core_clk 当 client0_tlp_hv 拉高时有效
client0_tlp_ep	1	I	TLP Header 中的 EP 字段。 时钟域: core_clk 当 client0_tlp_hv 拉高时有效
client0_tlp_byte_len	13	I	TLP Payload 字节长度, 不包括 ECRC, 有效值范围 0x0000 (0) 至 0x1000 (4096)。在“发送地址对齐”模式下, 设置为完整字节数; 在非“发送地址对齐”模式下, Bit[1:0] 固定为 0, client0_tlp_byte_len[12:0] 向上取整到 4 字节边界, 例如, 长度为 1~3 字节时 client0_tlp_byte_len[12:0] 要设置为 4, 字节长度为 5~7 时, client0_tlp_byte_len[12:0] 要设置为 8。 时钟域: core_clk 当 client0_tlp_hv 拉高时有效
client0_tlp_addr	64	I	TLP Header 中的 Address 字段。发送 CPL TLP 时, Bit[6:0] 作为 Lower Address, 3DW Header 中最后一个 Dword (8~11byte) 对应 Bit[31:0], 4DW Header 中第三个 Dword (8~11byte) 对应 Bit[63:32], 第四个 DW (12~15byte) 对应 Bit[31:0]。当发送 CFG 请求时, 按位编码映射关系如下: ● Bit[31:24]: Bus Number ● Bit [23:19]: Device Number ● Bit [18:16]: Function Number ● Bit [11:8]: Extended Register Number ● Bit [7:2]: Register Number 在“发送地址对齐”模式下, 指示完整字节地址 (字节对齐); 在非发送地址对齐模式下, 最后 2bit 为 0 (Dword 对齐)。 时钟域: core_clk 当 client0_tlp_hv 拉高时有效



Port Name	Width	I/O	Description
client0_tlp_tid	10	I	TLP Header 中的 TAG 字段。用户逻辑必须为所有的 Posted 和 Non Posted 请求提供有效的 TAG，最大支持数量为 64。用户逻辑必须管理每一个 Request 的 TAG。用于 CPL TLP 的 TAG 必须和对应的 Request 的 TAG 值一致。控制器不读取 client0_tlp_tid，而是使用 target completion LUT entry 中的 TAG。 时钟域：core_clk 当 client0_tlp_hv 拉高时有效
client0_tlp_data	64	I	TLP 的 Payload 数据，格式为 little endian，Bit[7:0]为第一个传输的字节。 当“发送地址对齐”模式使能时，第一个使能的字节必须是 Bit[7:0]。 PH1A100 固定为 64bit。 时钟域：core_clk client0_tlp_hv 拉高时有效
client0_tlp_func_num	1	I	发起当前 request 的 PF 的 Function Number，控制器用此信号值来组成 Request TLP Header 的 Requester ID 和 CPL 的 Completer ID。起始值为 0。只有一个 PF 的时候，设置为 0。 时钟域：core_clk client0_tlp_hv 拉高时有效
client0_tlp_vfunc_num ¹	2	I	发起当前 Request 的 VF 的 Function Number，控制器用此信号值来组成当前 Request TLP Header 的 Requester ID 和 CPL 的 Completer ID。起始值为 0，表示 client0_tlp_func_num 所选中的 PF 的第一个 VF。 时钟域：core_clk client0_tlp_hv 和 client0_tlp_vfunc_active 同时拉高时有效
client0_tlp_vfunc_active ¹	1	I	表示 Request 来自于一个 VF，0 表示没有活跃的 VF，且 client0_tlp_vfunc_num 无效，此时 client0_tlp_func_num 选中的 PF 为有效；1 表示 client0_tlp_vfunc_num 选中的 VF 有效。 时钟域：core_clk client0_tlp_hv 拉高时有效
client0_tlp_ats	2	I	TLP Header 中的 AT 字段。 时钟域：core_clk client0_tlp_hv 拉高时有效
client0_tlp_nw	1	I	TLP Header 中的 NW 字段。No Write Flag。 时钟域：core_clk client0_tlp_hv 拉高时有效
client0_tlp_th	1	I	TLP Header 中的 TH 字段。 时钟域：core_clk client0_tlp_hv 拉高时有效
client0_tlp_ph	2	I	TLP Header 中的 PH 字段。 时钟域：core_clk client0_tlp_hv 拉高时有效
client0_tlp_st	8	I	TLP Header 中的 ST 字段。 时钟域：core_clk client0_tlp_hv 拉高时有效



Port Name	Width	I/O	Description
xadm_client0_halt	1	0	控制器通过拉高 xadm_client0_halt 信号来告知用户逻辑，控制器还没有准备好发送数据包。在 xadm_client0_halt 置位期间，用户逻辑必须保持数据和 Header 接口上的值不变。控制器需要置位 xadm_client0_halt 的情况如下： XADM 在执行另外的发送 client 端口事务 没有足够的 Flow Control 信用来进行传输请求报文（P，NP，CPL） LTSSM 还未完成 正在执行重传 正在插入 ECRC 或 LCRC 时钟域：core_clk 高有效
client0_tlp_atu_bypass	1	I	旁路（Bypass）内部的 ATU。置位该信号表示本次传输不应该被内部地址翻译单元执行。 时钟域：core_clk 高有效 当 client0_tlp_hv 拉高时有效
client1_addr_align_en	1	I	同 client0（Description 列中所有 client0 应改为 client1，下同）
client1_tlp_byte_en	8	I	同 client0
client1_remote_req_id	16	I	同 client0
client1_cpl_status	3	I	同 client0
client1_cpl_bcm	1	I	同 client0
client1_cpl_byte_cnt	12	I	同 client0
client1_tlp_dv	1	I	同 client0
client1_tlp_eot	1	I	同 client0
client1_tlp_bad_eot	1	I	同 client0
client1_tlp_hv	1	I	同 client0
client1_tlp_fmt	2	I	同 client0
client1_tlp_type	5	I	同 client0
client1_tlp_tc	3	I	同 client0
client1_tlp_td	1	I	同 client0
client1_tlp_ep	1	I	同 client0
client1_tlp_attr	3	I	同 client0
client1_tlp_byte_len	13	I	同 client0
client1_tlp_tid	10	I	同 client0
client1_tlp_addr	64	I	同 client0
client1_tlp_data	64/128	I	同 client0
client1_tlp_func_num ¹	1	I	同 client0
client1_tlp_vfunc_num ¹	2	I	同 client0
client1_tlp_vfunc_active	1	I	同 client0
client1_tlp_ats	2	I	同 client0
client1_tlp_nw	1	I	同 client0
client1_tlp_th	1	I	同 client0
client1_tlp_ph	2	I	同 client0
client1_tlp_st	8	I	同 client0



Port Name	Width	I/O	Description
xadm_client1_halt	1	0	同 client0
client1_tlp_atu_bypass	1	1	同 client0

注：1. 该端口只在 EP 模式下使用，RC 模式无效。

2.3.3 PCIe 控制器发送 Request 用法

如下图所示，XALIO 和 XALI1 接口由以下几组信号组成：

- client0/1_tlp_hv/_dv/_eot, xadm_client0/1_halt。该组信号指示当前传输 TLP Header 和 Data 有效，标志 TLP 之间的间隔，以及使控制器与用户逻辑之间进行握手。
- client0/1_tlp_*head*/_data。该组信号指示当前 TLP 传输的 Header 与 Data。
- client0 (XALIO) 和 client1 (XALI1) 是完全相同的接口，下面图例中可能以其中一个接口作为示例，同样适用于另一个端口。

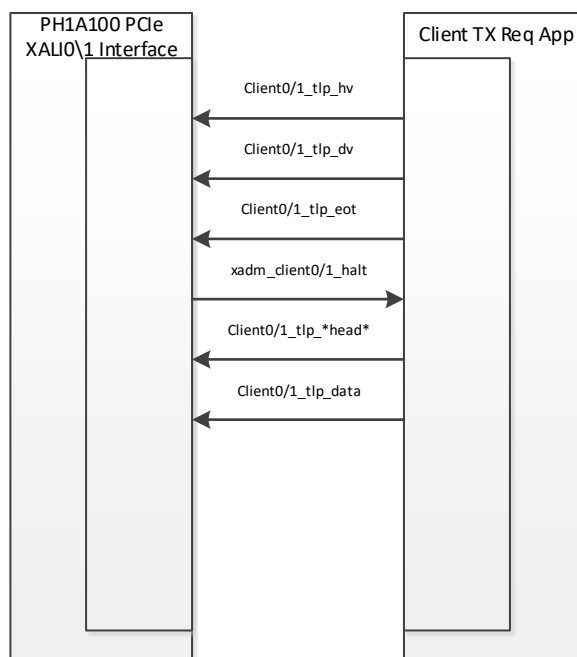


图 2-7 Client0/1 接口 (XALIO/1) 发送 Request TLP 相关信号

下表为 PCI-SIG 规定的 TLP 请求 (request) Header 格式和 Analogix PCIe 控制器的 client0 或者 client1 接口信号的对应关系。该表格中斜体字为 Client 接口信号名称。星号*代表 0 或者 1，分别表示 client0 接口和 client1 接口。

表 2-4 TLP Request Header 格式和 client 接口信号对应表

+0								+1								+2								+3							
7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0



Byte 0 >	Length IP 自动产生	AT client*_tlp_ats Attr[1:0]	EP client*_tlp_en ID client*_tlp_id TH client*_tlp_th LN (unsupported) Attr[2] client*_tlp_attr[2] TG client*_tlp_tid[8] TC client*_tlp_tc TG client*_tlp_tid[9] Type client*_tlp_type Fmt[1:0] Fmt[2]
Byte 4 >	Requester ID IP 自动产生	TAG client*_tlp_tid[7:0]	Last BE client1_tlp_byte_en[7:4] 1 st BE client1_tlp_byte_en[3:0] (地址对齐关闭模式)
Byte 8 >	Address[63:32] client*_tlp_addr[63:32]		
Byte 12 >	Address[31:2] client*_tlp_addr[31:2]		
			PH client*_tlp_ph

下表为 Translation Request Header 格式和 Anlogic PCIe 控制器的 client0 或者 client1 接口信号的对应关系。该表格中斜体字为 Client 接口信号名称。星号*代表 0 或者 1，分别表示 client0 接口和 client1 接口。

表 2-5 Traslation Request Header 格式和 client 接口信号对应表

	+0								+1								+2								+3								
	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	
Byte 0 >	Fmt[2]	Fmt[1:0] client*_tlp_fmt		client*_tlp_type				Type	TG_client*_tlp_tid[9]	client*_tlp_tc		TC	TG_client*_tlp_tid[8]	Attr[2] client*_tlp_attr[2]	LN (unsupported)	TH_client*_tlp_th	ID_client*_tlp_tid	EP_client*_tlp_en	Attr[1:0]	AT (01) client*_tlp_ats	Length IP 自动产生												
Byte 4 >	Requester ID IP 自动产生																TAG client*_tlp_tid[7:0]								Last BE IP 自动产生				1 st BE IP 自动产生				
Byte 8 >	Untranslated Address[63:32] client*_tlp_addr[63:32]																																
Byte 12 >	Untranslated Address[31:12] client*_tlp_addr[31:12]																Reserved																NW client*_tlp_nw

2.3.3.1 发送地址对齐 (Transmit Address Alignment)

对于发送 Memory 和 I/O 请求 (完成报文 CPL TLP 除外)，用户逻辑可以使用 PCIe 控制器提供的 client*_addr_align_en 信号来选择发送每个 TLP 报文的对齐方式。为了使用这一功能，需将



GLOBAL_ADDR_ALIGN_EN 寄存器设置为 1（默认值即为 1）。当地址对齐关闭（client0_addr_align_en=0），用户逻辑负责设置合适的 first and last byte enable（FBE, LBE）来表示所需要传输的 Payload 在数据总线上的位置。当地址对齐开启（client*_addr_align_en=1）时，控制器采用地址（client*_tlp_addr）的最低两 bit 来表示所需要传输的负载在数据总线上的位置。

地址对齐关闭模式（Dword 对齐模式）

关闭地址对齐后，client 接口实际上是以 DWORD 为单位来进行传输的。在这种情况下，用户逻辑必须给 client 接口提供与 PCI-SIG 协议规范完全一致的 TLP Header 格式。下面以 client1 为例介绍 client 接口使用规则：

- client1_tlp_addr: Bit[1:0] 必须为 00，即地址必须为 DWORD 对齐。
- client1_tlp_byte_en: 字节使能直接放置在传输的 TLP 中。Bit[3:0] 位构成 FBE；Bit[7:4] 位构成 LBE。不连续的字节使能控制可使用这种地址对齐关闭的传输模式发送。
- client1_tlp_byte_len: Bit[1:0] 必须设置为 00。字节长度值向上取整到下一个 DWORD 边界。例如，当请求为 5 个字节长时，Bit[12:0] 的值必须为 8。
- client1_tlp_data: 控制器将要传输的数据和 client1_tlp_data 端口上显示的一致。用户逻辑负责根据字节地址上移数据。

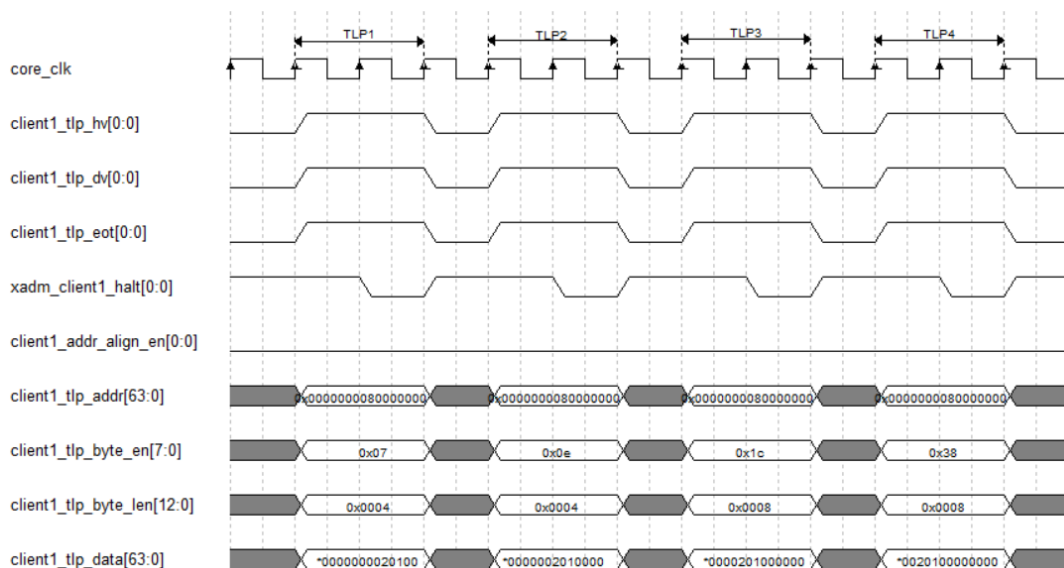


图 2-8 地址对齐关闭典型时序图

对于读请求，client1_tlp_data 总线没有特别要求。

上图给出了传输四个 3 字节存储器写请求的示例波形，这些请求的字节地址从 0x80000000 开始递增，以 0x80000003 结尾。请注意，对于每个请求，随着地址的增加，第一个有效的字节将会左移一个字节。同样，当地址加长度超过 DWORD 边界时（图中的第三个 TLP），字节长度直接从 4 变为 8。



地址对齐打开模式（Byte 对齐模式）

当数据包的地址对齐处于打开状态时，客户端接口将变为字节接口。用户逻辑提供的格式信息如下：

- **client1_tlp_addr**: 该地址是请求包的第一个有效字节的完整字节地址。也就是说，请求必须是字节对齐的。
- **client1_tlp_byte_en**: 应为 0x0，即不适用于使能了发送地址对齐模式的数据包。
- **client1_tlp_byte_len**: 必须为请求的完整字节长度。
- **client1_tlp_data**: 请求的第一个有效字节必须在此数据总线的 Bit[7:0] 上。控制器根据字节地址的低两位在传输的 TLP 中上移数据。

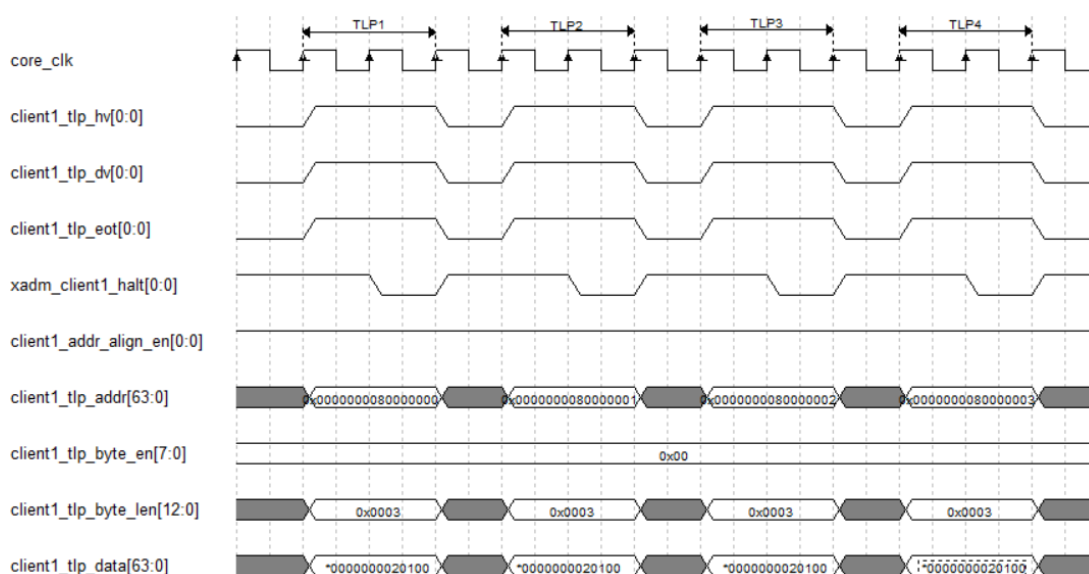


图 2-9 地址对齐开启典型时序图

根据地址和字节长度，控制器自动控制 TLP 的 First BE 和 Last BE 字段。对于读请求，client0_tlp_data 信号没有要求。对于写请求，控制器决定在 TLP 中传输了多少个 DWORD 数据有效 Payload。图 2-9 显示了与图 2-8 相同的存储器写请求序列，但地址对齐功能处于打开状态。数据不会根据字节地址而改变。控制器不会生成具有非连续 Last BE 或 First BE 的 TLP。也就是说，在这个模式下，控制器不支持非连续的 Last BE 和 First BE TLP 报文。

地址对齐和数据总线周期

对于写请求，关闭发送地址对齐后，用户逻辑数据总线上的时钟周期数可能会比开启地址对齐时更多。当写请求的数据有效载荷越过数据向量大小的边界时，就会发生这种情况。下图显示了在 32 位发送数据总线上的两个相同的存储器写请求。字节地址为 0x80000002，字节长度为 3。第一个请求包开启了地址对齐功能，在传输客户端接口上仅需要一个周期。第二个请求包关闭了地址对齐功能，在传输客户端接口上需要两个时钟周期。这对总带宽没有影响。唯一的区别是控制器生成 TLP 报文格式（第一个请求）还是用户逻辑生成 TLP 报文格式（第二个请求）。

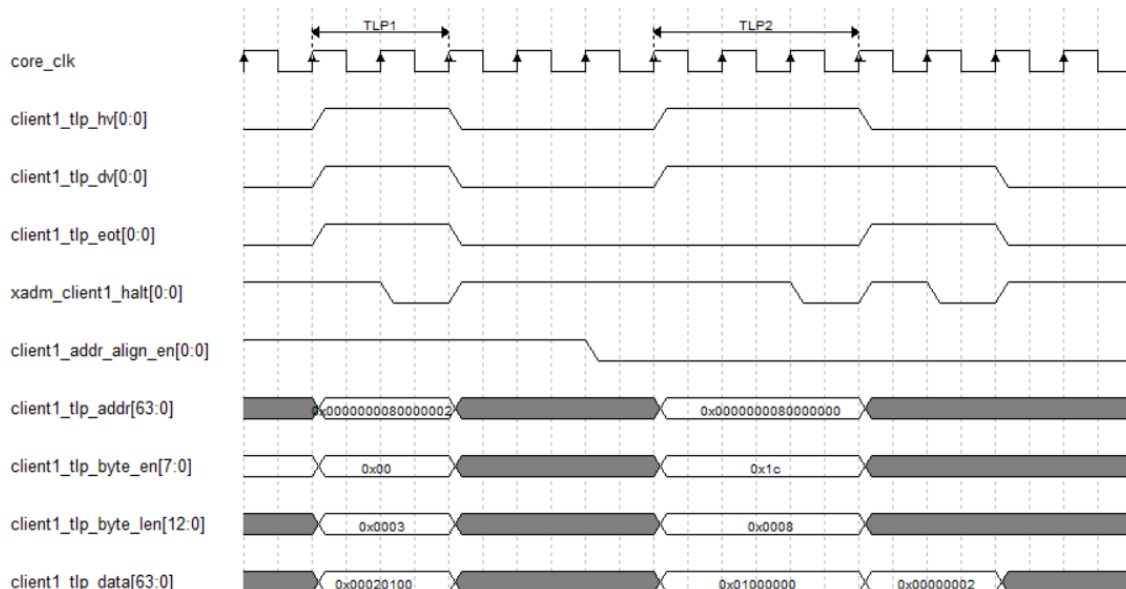


图 2-10 地址对齐开启和关闭对数据总线周期数的影响

地址对齐和 ECRC

当用户逻辑通过 `client` 接口发送的请求包含 ECRC 的情况下，对于包含 ECRC 的 TLP (TD=1)，`client1_addr_align_en` 信号必须为 0。

2.3.3.2 XAL11 接口协议规则

发送客户端接口提供两个通路：报文头（TLP Header）通路和报文数据（TLP Data）通路。控制器内部并没有发送 TLP 报文的队列。一旦用户逻辑开始一个 TLP 报文的传输，必须按照流（streaming）模式传输数据，除非控制器自己暂停数据传输。具体规则如下：

- 1) 对于没有有效数据载荷的事务（例如 MRd），必须保持数据有效（`client1_tlp_dv`）信号为非有效（低电平）。有关更多信息，请参见时序图。
- 2) 对于具有有效载荷（例如 MWrr）的事务，必须在报文头阶段的同一周期中声明报文头有效（`client1_tlp_hv`）和数据有效（`client1_tlp_dv`）信号。在有效载荷传输阶段，可以仅使能（或拉高）`client1_tlp_dv`，而不必使能（或拉高）`client1_tlp_hv`。有关更多信息，请参见时序图。
- 3) 在没有有效载荷的情况下，报头有效指示符（`client1_tlp_hv`）的最后一个周期必须拉高事务结束指示信号（`client1_tlp_eot`）。在具有有效载荷的情况下，数据有效指示符（`client1_tlp_dv`）的最后一个周期用户逻辑必须拉高事务结束指示信号（`client1_tlp_eot`）。用户逻辑不能在其他任何时刻拉高 `client1_tlp_eot`。
- 4) 控制器拉高 `xadm_client1_halt` 信号后，用户逻辑必须保持报文头（Header）总线和数据总线上的信息不变。
- 5) 空闲时，传输客户端接口在默认情况下保持 `xadm_client1_halt` = 1。在数据包末尾，当正在进行下一个数据包的客户端仲裁时，`xadm_client1_halt` 可能会跳变为高。`xadm_client1_halt` 信号的行为取决于控制器的数据路径宽度。
- 6) `client1_tlp_bad_eot` 信号用于终止（abort）正在进行的传输报文。必须在与 `client1_tlp_eot` 相同的周期内拉高。



- 7) 不是所有的报文头信息都是动态的，用户逻辑可以根据需求接成常 0 或常 1，即进行 Tie low 或者 Tie high。
- 8) 要通过此端口发送 Message，用户逻辑需要把 Message Code 放在 client1_tlp_byte_en 接口上，Header 的最后两个 Dword 放在 client1_tlp_addr[63:0] 接口上。其中 client1_tlp_addr[63:32] 为第三个 Dword (bytes 8-11)。当 Message 中有 Data 时，用户逻辑把 Data 放在 client1_tlp_data，和其他类型的 Request TLP 类似。

2.3.3.3 发送请求 (Request) 时序

下图描述了三个内存读取 (MRd) 请求 TLP。对于两个背靠背 (back to back) MRd TLP，client1_tlp_eot 和 client1_tlp_hv 信号保持为高电平。TLP Header 输入信号保持有效，直到控制器拉低 xadm_client1_halt。当控制器尚未准备好处理任何事务时，控制器会拉高 xadm_client1_halt 信号。

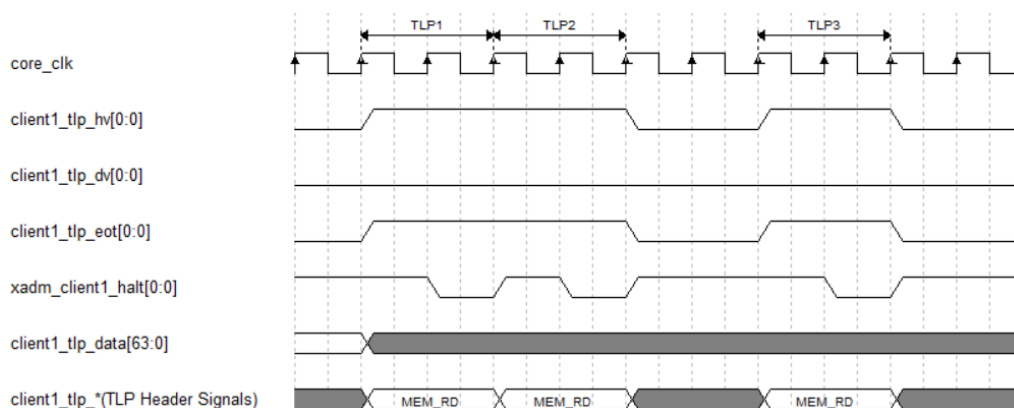


图 2-11 Client1 接口上的 MRd Request TLP

下图展示一个存储器写 (MWrr) 请求 TLP。当拉高 xadm_client1_halt 时，TLP 报文头信号 (Header) 和有效数据载荷输入保持不变。

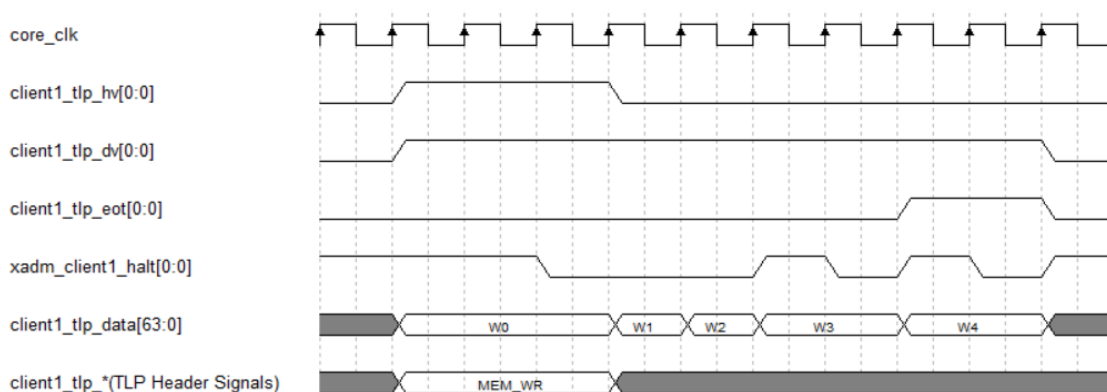


图 2-12 Client1 接口上的 MWrr Request TLP

下图显示了背靠背的 MRd 和 MWrr 事务。client1_tlp_eot 和 client1_tlp_hv 信号在多个 TLP 之间保持有效。

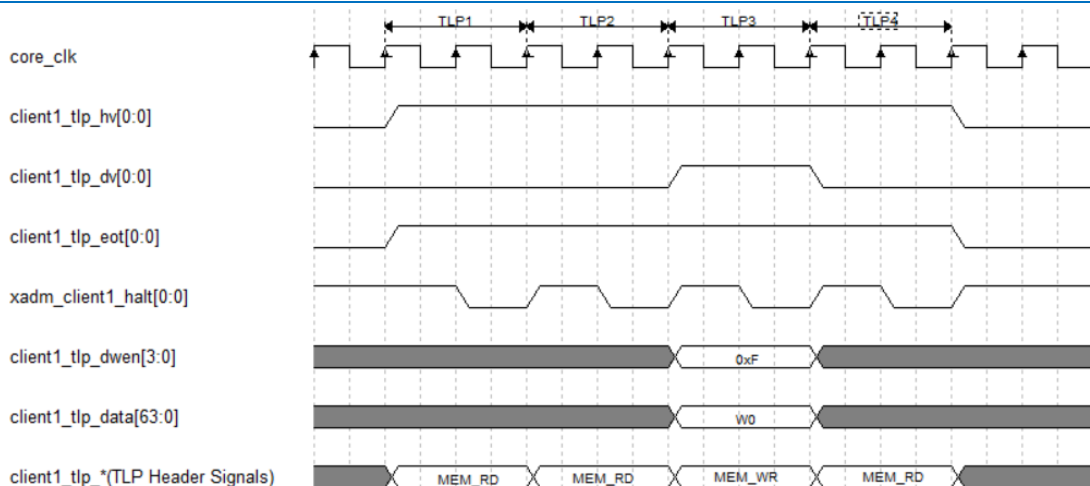


图 2-13 Client1 接口上的背靠背 Back to Back MRd 和 MWr TLP 示例

图 2-14 至图 2-15 的时序波形中带有具体 TLP Header 信息，它们分别展示 MWr, MRd, Atomic, Message, I/O 报文的发送示例波形。

下图时序展示带有具体 TLP Header 信号的 MWr TLP 发送示例波形。client1_tlp_hv/client1_tlp_dv 分别指示 TLP Header/TLP Data 有效；TLP 报文格式和类型通过 client1_tlp_fmt 和 client1_tlp_type 信号端口来表示，当 client1_tlp_fmt=0x3、client1_tlp_type=0x00 时表示 TLP 大小为 4 个 Dword 带数据负载；发送地址为 0xcda7_add6_9ab2_4000，地址对齐方式为双字 (Dword) 对齐，FBE 和 LBE 为 0xff；TLP 数据负载 client1_tlp_byte_len=0x800。

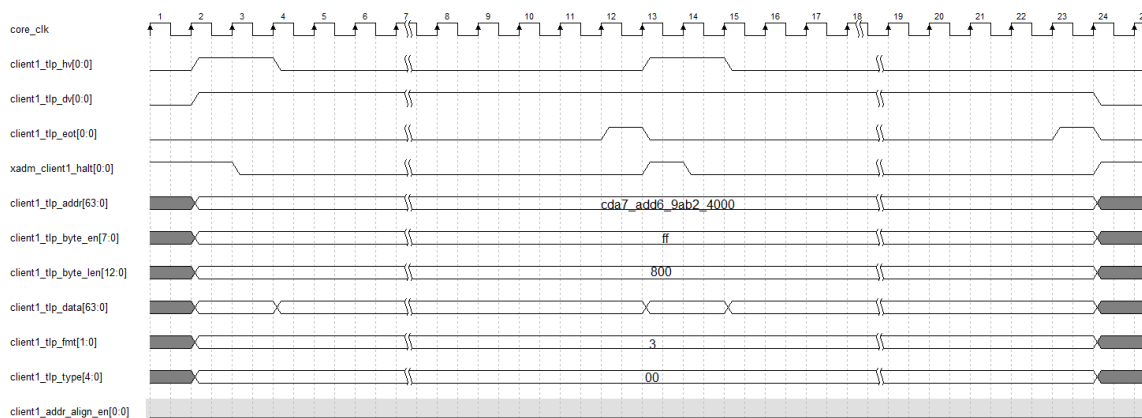


图 2-14 带有具体包头信号的 Memory Write Transaction

下图展示带有具体 TLP Header 信号的 MRd TLP 发送示例波形。client1_tlp_hv/client1_tlp_dv 分别指示 TLP Header 和 TLP Data 有效；client1_tlp_fmt=0x0, client1_tlp_type=0x00 表示当前 TLP 报文为 MRd, client1_tlp_byte_len=0x4 表示 MRd 请求 4bytes 数据, client1_tlp_tid 为 0~3 表示发送的 4 个 TLP 报文的 TAG, 回复报文的 TAG 要与 MRd 报文的 TAG 一致。

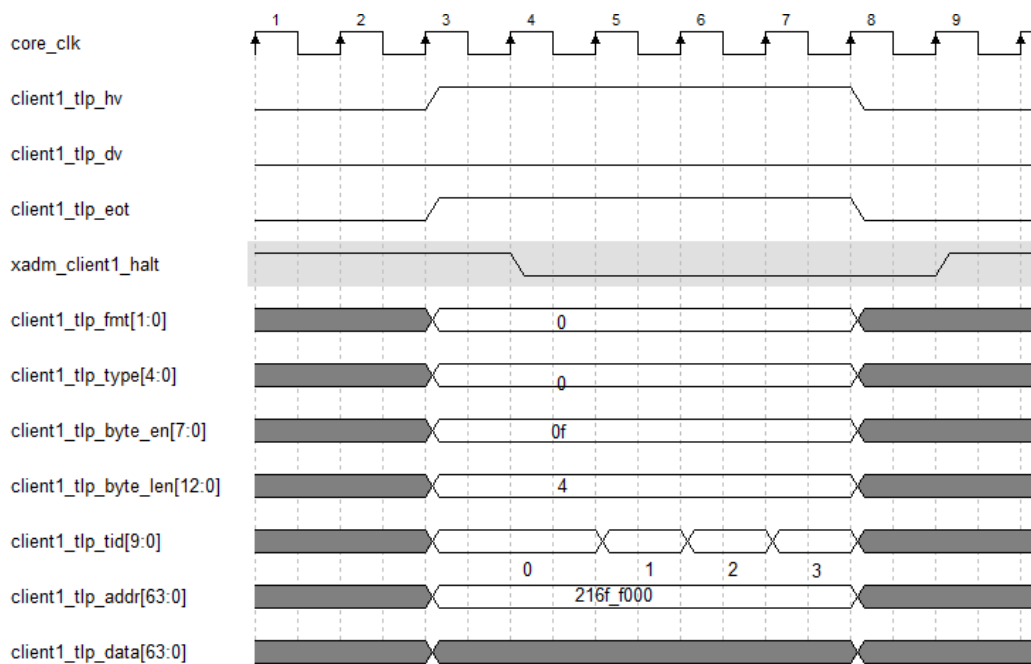


图 2-15 带有具体包头信号的 Memory Read Transaction

下图时序展示带有具体 TLP Header 信号的 Atomic TLP 发送示例波形。client1_tlp_hv/client1_tlp_dv 分别指示 TLP Header 和 TLP Data 有效；client1_tlp_fmt=0x2, client1_tlp_type=0xc 表示 FetchAdd 操作；client1_tlp_fmt=0x2, client1_tlp_type=0xd 表示 Swap 总线事务；client1_tlp_fmt=0x2, client1_tlp_type=0xe 表示 CAS 总线事务；client1_tlp_byte_len 表示读取原始数据长度；client1_tlp_tid 为 TLP 中的 TAG 字段。

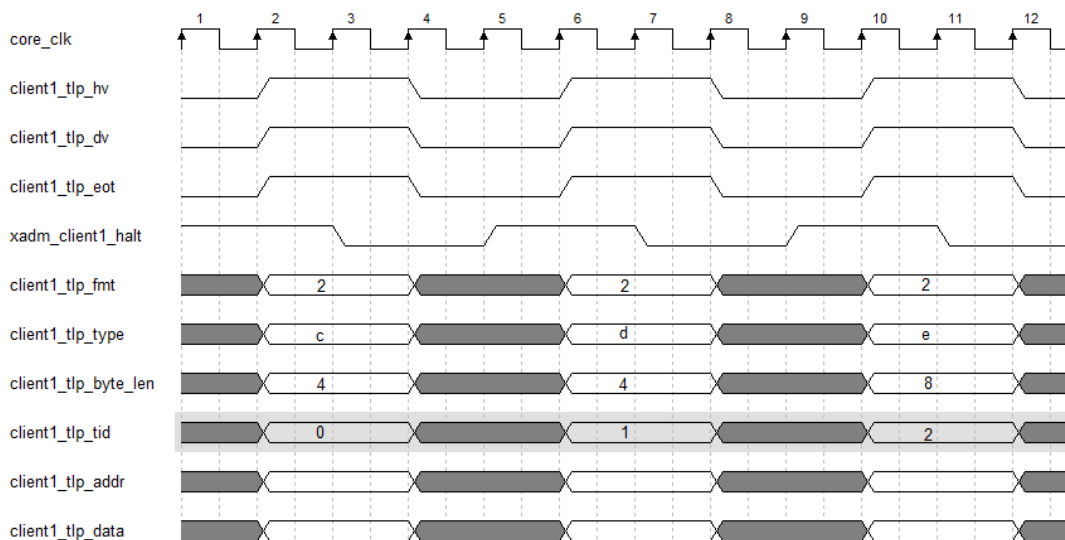


图 2-16 带有具体包头信号的 Atomic Transaction

下图时序展示带有具体 TLP Header 信号的 Message TLP 发送示例波形。client1_tlp_hv/client1_tlp_dv 分别指示 TLP head/TLP data 有效；client1_tlp_fmt=0x1 和 client1_tlp_type=0x14 表示当前 TLP 为 Msg；其他信号与 MWr 类似。

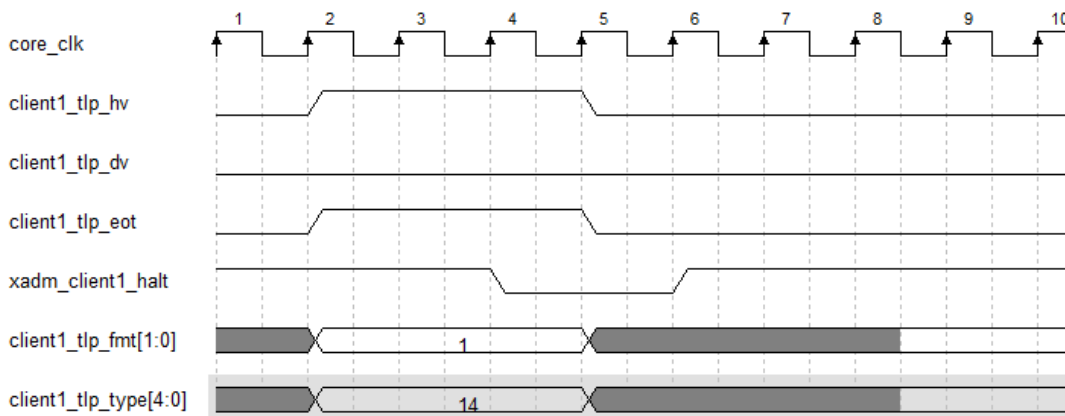


图 2-17 带有具体包头信号的 Message Transaction

下图时序展示带有具体 TLP Header 信号的 I/O TLP 发送示例波形。Client1_tlp_hv/client1_tlp_dv 分别指示 TLP Header 和 TLP Data 有效；client0_tlp_fmt=0x2 和 client0_tlp_type=0x2 表示当前 TLP 为 IOWr；client1_tlp_fmt=0x0 和 client1_tlp_type=0x2 表示当前 TLP 为 IORd；I/O 读写请求的下列字段必须为以下值：

- client1_tlp_tc: 0x0
- client1_tlp_th: 0x0
- client1_tlp_attr: 0x0
- client1_tlp_at: 0x0
- client1_tlp_byte_len: 0x4
- client1_tlp_byte_en: 0x0f

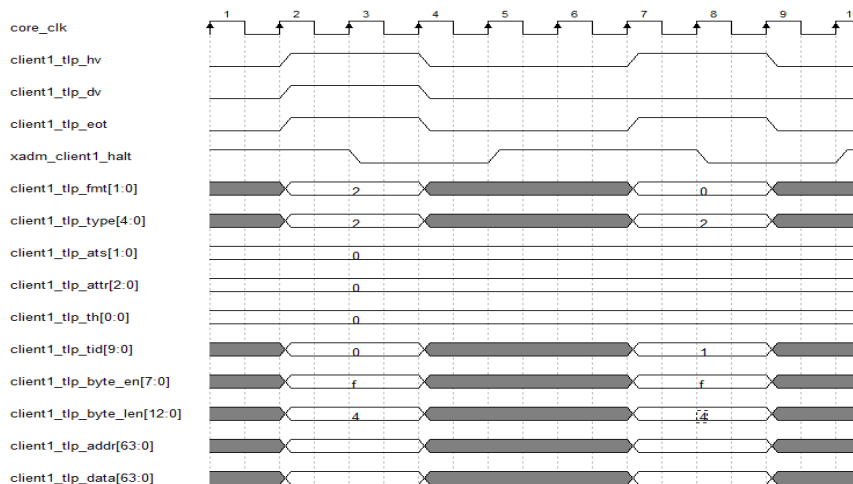


图 2-18 带有具体包头信号的 I/O Transaction

2.3.4 发送 CPL TLP 用法

如下图，Client0/1 Interface (XALIO/1) 由以下几组信号组成：

- client0/1_tlp_hv/_dv/_eot, xadm_client0/1_halt: 该组信号指示当前传输 TLP 头(Header)



和数据载荷（Data Payload）有效，标志 TLP 之间的间隔，以及使控制器与用户逻辑之间进行握手。

- `client0/1_tlp_*header*/_data`: 该组信号指示当前 TLP 传输的头（Header）与数据（Data）内容。
- `client0/1_cpl_lookup_id`: 该组信号指示当前 CPL 报文对应的 Target Completion LUT 的 ID，从该 ID 指示的 LUT 获取头信息。

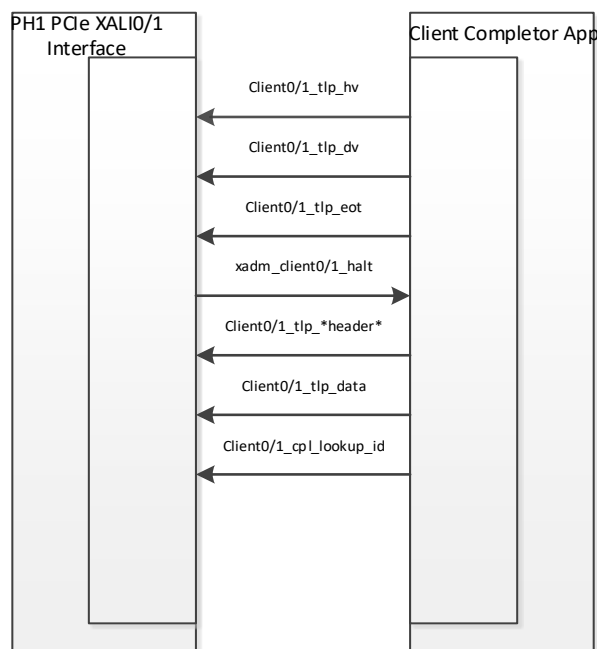


图 2-19 Client0/1 接口（XALI0/1）发送 CPL 报文相关信号

下表为 PCI-SIG 规定的完成包（Completion）Header 格式和 PCIe 控制器的 `client0` 或者 `client1` 接口信号的对应关系。该表格中斜体字为 Client 接口信号名称。星号*代表 0 或者 1，分别表示 `client0` 接口和 `client1` 接口。

使用 Client 接口发送完成包时，不能使用地址对齐模式，必须设置 `client*_addr_align_en` 为 0，用户逻辑必须给 `client*_tlp_addr` 提供正确的地址，这样才能保证完成包的 Lower Address[6:0] 字段能够从 Dword 对齐的地址和 First BE 字段正确的推断出来。



表 2-6 Completion TLP Header 格式和 client 接口信号对应表

+0								+1								+2								+3							
7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0
Byte 0 >																															
Fmt[7:0]								Type								Attr[1:0]								Length							
client*_tlp_type								client*_tlp_tc								client*_tlp_ats								IP 自动产生							
Completer ID								Byte Count								client*_cpl_byte_cnt															
client*_remote_req_id[15:0]								TAG								R								Lower Address							
client*_tld[7:0]								client*_tld[7:0]								IP 自动产生															

2.3.4.1 XALIO 接口协议规则

XALIO 接口协议规则和 XALI1 接口协议规则相同。下文中以 XALIO 为发送 CPL TLP 的接口示例，XALI1 是和 XALIO 相同的接口，也可以用于发送 CPL TLP。

发送客户端提供两个通路：报文头（TLP Header）通路和报文数据（TLP Data）通路。控制器内部并没有发送 TLP 报文的队列。一旦用户逻辑开始一个 TLP 报文的传输，必须按照流模式（Streaming）传输数据，除非控制器自己暂停数据传输。具体规则如下：

- 1) 对于没有有效载荷的事务（例如 MRd），必须保持数据有效（client0_tlp_dv）信号为非有效（低电平）。有关更多信息，请参见时序图。
- 2) 对于具有有效载荷（例如 MWr）的事务，必须在报文头阶段的同一周期中拉高报文头有效（client0_tlp_hv）和数据有效（client0_tlp_dv）信号。在有效载荷传输阶段，可以仅使能（或拉高）client0_tlp_dv，而不必使能（或拉高）client0_tlp_hv。有关更多信息，请参见时序图。
- 3) 在没有有效载荷的情况下，报文头有效指示符（client0_tlp_hv）的最后一个周期用户逻辑必须拉高事务结束指示信号（client0_tlp_eot）。在具有有效载荷的情况下，数据有效指示符（client0_tlp_dv）的最后一个周期必须拉高事务结束指示信号（client0_tlp_eot）。不得在其他任何时刻拉高这个信号（client0_tlp_eot）。
- 4) 控制器拉高 xadm_client0_halt 信号后，用户逻辑必须保持报文头总线 and 数据总线上的信息不变。
- 5) 空闲时，传输客户端接口默认情况下保持 xadm_client0_halt = 1。在数据包末尾，当正在进



行下一个数据包的客户端仲裁时, `xadm_client0_halt` 可能会跳变。`xadm_client0_halt` 信号的行为取决于控制器的数据路径宽度。

- 6) `client0_tlp_bad_eot` 信号用于终止 (Abort) 正在进行的传输。必须在与 `client0_tlp_eot` 相同的周期内拉高。
- 7) 不是所有的报文头信息是动态的, 用户逻辑可以根据需求接成常 0 或常 1, 即进行 Tie low 或者 Tie high。
- 8) 要通过此端口发送 Message, 用户逻辑需要把 Message Code 放在 `client0_tlp_byte_en` 接口上, 报文头 Header 的最后两个 Dword 放在 `client0_tlp_addr[63:0]` 上。其中 `client0_tlp_addr[63:32]` 为第三个 Dword (bytes 8-11)。当 Message 中有 Data 时, 用户逻辑把 Data 放在 `client0_tlp_data`, 和其他 TLP 类似。

2.3.4.2 目标完成包查找表 (Target Completion Lookup Table)

Target completion lookup table (LUT) 会在控制器内部将 TRGT1 接口和 XALIO/1 接口部分信号连接起来。这一特性提供以下功能:

- 1) 当使用 XALIO 接口发送 CPL 报文时, 用户逻辑不需要提供某些特定的 TLP Header 信息, 因为控制器会追踪并存储这些信息到查找表 (LUT)。LUT 使用表格来减轻用户逻辑产生 CPL 报文的负担。所有的待处理 Inbound non-Posted 请求都会在查找表中储存并以 Lookup ID 来索引。当用户逻辑将要发送一个 CPL 报文, 只需要提供 Lookup ID 然后 LUT 就会自动填充 CPL 头字段的相关信息。
- 2) 为了传输一个 CPL 报文, 用户逻辑首先存储来自请求包的 Lookup ID (由 `trgt_lookup_id` 信号提供), 然后在发送 CPL 报文时通过 `client0_cpl_lookup_id` 端口提供相同的 Lookup ID。当使用这一功能时, 用户逻辑不再需要提供以下头信息。
 - `client0_remote_req_id`
 - `client0_cpl_byte_cnt`
 - `client0_tlp_attr`
 - `client0_tlp_addr`
 - `client0_tlp_tid`
- 3) 当用户逻辑没有能够在规定时间内生成 CPL TLP, LUT 会提示出现 timeout。CPL 报文的 timeout 信息可以通过 `trgt_timeout_*` 信号进行获取。

2.3.4.3 发送 CPL TLP 时序图

下图显示了通过 `client0` 接口发送 CPL TLP 的典型时序。用户逻辑接收完 MRd 报文后, 需要通过 XALIO 接口发送相对应的 CPL 报文。`client0_tlp_hv/client0_tlp_dv` 分别指示 TLP Header/TLP Data 有效; `client0_tlp_fmt=0x2` 和 `client0_tlp_type=0xa` 表示当前 TLP 为 CPLD (Completion without Data); 控制器会根据 `client0_cpl_lookup_id` 自动的将 CPLD 报文头中的 Byte Count/Lower Address/TAG/Completer ID/Requester ID 字段填充好。

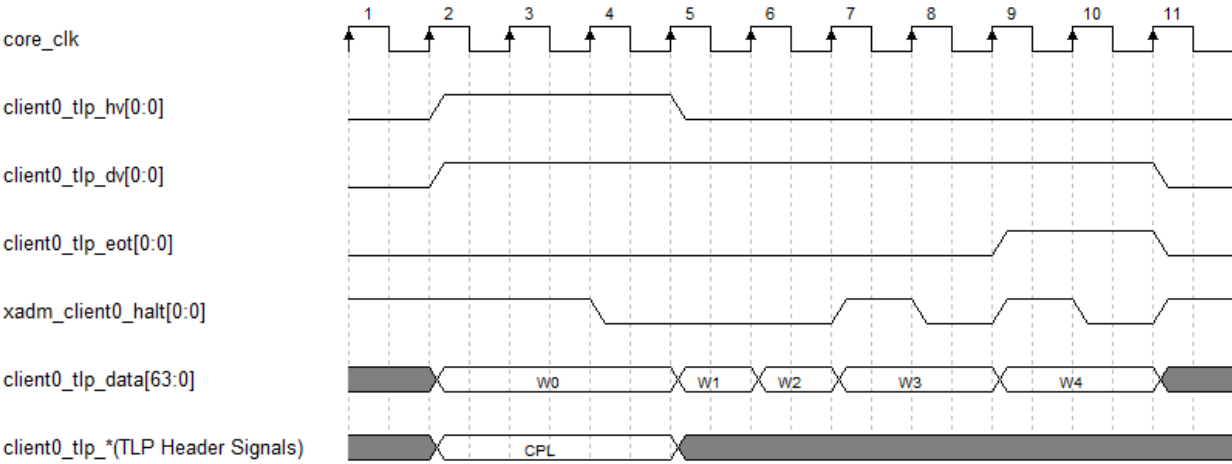


图 2-20 Client0 接口发送 CPL TLP 示例波形

2.4 接收目标端口 TRGT1

2.4.1 功能简介

TRGT1 (Receive Target 1 Interface) 端口用于接收 Posted 和 Non-Posted 请求报文。

2.4.2 端口

TRGT1 接口信号列表如下。

表 2-7 TRGT1 接口信号列表

Port Name	Width	I/O	Description
trgt1_radm_halt	1	I	流控输入信号。当 trgt1_radm_halt 拉高时，控制器停止从 RADM 队列中输出下一个有效数据。用户逻辑需要把用户 halt 信号和 (!trgt_lookup_empty) 取 ‘或’ 后来驱动此输入信号。 时钟域: core_clk 高有效
radm_trgt1_dv	1	O	指示接收到的 TLP 数据，即 radm_trgt1_data，为有效。 时钟域: core_clk 高有效
radm_trgt1_hv	1	O	指示 TLP Header 信息，即 adm_trgt1_*输出端口，为有效。 时钟域: core_clk 高有效:
radm_trgt1_eot	1	O	接收到 TLP 的最后一个有效数据周期。 时钟域: core_clk 高有效 radm_trgt1_hv 或 radm_trgt1_dv 拉高时有效



Port Name	Width	I/O	Description
radm_trgt1_tlp_abort	1	0	指示 TRGT1 接口上当前的 TLP 发生 Malformed TLP, ECRC error 或 completion lookup failures (例如 TAG 或 requester ID 发生不一致)等错误, 用户逻辑应丢弃此包, 不会重传。 时钟域: core_clk 高有效 radm_trgt1_eot 或 radm_trgt1_hv 拉高时有效
radm_trgt1_dllp_abort	1	0	指示当前 TLP 发生数据链路层 (Data Link Layer) 错误。用户逻辑应丢弃此包, 根据协议对端应该重传此包。 时钟域: core_clk 高有效 radm_trgt1_eot 拉高时有效
radm_trgt1_ecrc_err	1	0	指示当前 TLP 发生 ECRC Error, 用户逻辑应丢弃此包, 不会重传。 时钟域: core_clk 高有效 radm_trgt1_eot 拉高时有效
radm_trgt1_data	64	0	接收到的 TLP Payload, 格式为 little endian, 第一个接收到的字节出现在 Bit[7:0]。PH1A100 固定为 64bit。 时钟域: core_clk radm_trgt1_dv 拉高时有效
radm_trgt1_dwen	2	0	用于指示 radm_trgt1_data 最后一个 DW 的位置。 64-bit 位宽 bit[1:0] 编码映射为 ■ 01: 最后一个 Dword 在 radm_trgt1_data[31:0]. ■ 11: 最后一个 Dword 在 radm_trgt1_data[63:32]. 时钟域: core_clk radm_trgt1_eot 拉高且 NW > 1 时有效
radm_trgt1_fmt	2	0	TLP Header 的 Format 字段。 时钟域: core_clk radm_trgt1_hv 拉高时有效
radm_trgt1_type	5	0	TLP Header 的 Type 字段。 时钟域: core_clk radm_trgt1_hv 拉高时有效
radm_trgt1_tc	3	0	TLP Header 的 Traffic Class (TC) 字段。 时钟域: core_clk radm_trgt1_hv 拉高时有效
radm_trgt1_attr	3	0	TLP Header 的 Attribute 字段。 时钟域: core_clk radm_trgt1_hv 拉高时有效
radm_trgt1_reqid	16	0	TLP Header 中的 Requester ID。按位编码映射关系如下 ■ Bit[15:8]: Bus number ■ Bit[7:3]: Device number ■ Bit[2:0]: Function number 时钟域: core_clk radm_trgt1_hv 拉高时有效



Port Name	Width	I/O	Description
radm_trgt1_tag	10	0	TLP Header 中的 Tag 字段。 时钟域: core_clk radm_trgt1_hv 拉高时有效
radm_trgt1_func_num	1	0	当前 TLP 的 Function Number。起始值为 0。 时钟域: core_clk radm_trgt1_hv 拉高时有效
radm_trgt1_vfunc_num ¹	2	0	指示接收到的 Request TLP 的目标 VF 的 Function Number，从 0 起始。例如 radm_bypass_vfunc_num=0 指的是 radm_trgt1_func_num 对应 PF 中的第一个 VF。 时钟域: core_clk 在 radm_trgt1_hv 和 radm_trgt1_vfunc_active 都拉高时有效
radm_trgt1_vfunc_active ¹	1	0	指示接收到的 TLP 目标为 VF。 0: 接收到的 TLP 目标不是 VF 且 radm_bypass_vfunc_num 无效， radm_bypass_func_num 对应的 PF 有效； 1: TLP 目标是 radm_bypass_vfunc_num 对应的 VF。 时钟域: core_clk radm_trgt1_hv 拉高时有效
radm_trgt1_td	1	0	TLP Header 中的 TD 字段。 时钟域: core_clk radm_trgt1_hv 拉高时有效
radm_trgt1_poisoned	1	0	TLP Header 中的 Poisoned (EP) 字段。 时钟域: core_clk radm_trgt1_hv 拉高时有效
radm_trgt1_dw_len	10	0	TLP Header 中的 Length 字段。 时钟域: core_clk 拉高 radm_trgt1_hv 时有效
radm_trgt1_first_be	4	0	TLP Header 中的 First Dword Byte Enable 字段。 时钟域: core_clk radm_trgt1_hv 拉高时有效
radm_trgt1_last_be	4	0	TLP Header 中的 Last DW Byte Enable 字段。 时钟域: core_clk radm_trgt1_hv 拉高时有效
radm_trgt1_addr	64	0	TLP Header 中的 Address 字段，3DW Header 中最后一个 Dword (8-11byte) 对应 [31:0]，4DW Header 中第三个 Dword (8-11byte) 对应 [63:32]，第四个 Dword (12-15byte) 对应 [31:0]。 时钟域: core_clk radm_trgt1_hv 拉高时有效



Port Name	Width	I/O	Description
radm_trgt1_hdr_uppr_bytes	64	0	TRGT1 接口接收到的 TLP Header 中的高字节 (RAW Upper Bytes)。对于 3DW Header 来说, 指的是字节 8-11; 对于 4DW Header 来说, 指的是字节 8-15。当 radm_trgt1_hdr_uppr_bytes_valid 为高时, radm_trgt1_hdr_uppr_bytes 输出有效 Msg 数据。TLP Header 字节映射如下: ■ MEM (32-bit) / IO / CFG 的 Header 为 3DW, radm_trgt1_hdr_uppr_bytes[31:0] = 字节 8-11。这时 bits [7:0] 映射到 TLP Header 的第 8 字节 ■ MEM (64-bit) / MSG 的 Header 为 4DW, radm_trgt1_hdr_uppr_bytes[63:32] = bytes 12-15 并且 radm_trgt1_hdr_uppr_bytes[31:0] = bytes 8-11。这时, bits [7:0] 映射为 TLP Header 的第 8 字节而 bits [39:32] 映射为第 12 字节, 以此类推。 时钟域: core_clk radm_trgt1_hv 拉高时有效
radm_trgt1_hdr_uppr_bytes_valid	1	0	表示 TRGT1 上的 TLP 报文已经成功被 iATU 翻译。radm_trgt1_hdr_uppr_bytes 输出有效的 Msg Data。 时钟域: core_clk radm_trgt1_hv 拉高时有效
radm_trgt1_rom_in_range	1	0	指示接收到的 TLP 目标地址在 Expansion ROM 范围内。 时钟域: core_clk radm_trgt1_hv 拉高时有效
radm_trgt1_in_membar_range	3	0	指示接收到的 TLP request 的目标地址所在的 BAR 空间。 ■ 000b: 目标地址在 BAR 0 范围内 ■ 001b: 目标地址在 BAR 1 范围内 ■ 010b: 目标地址在 BAR 2 范围内 ■ 011b: 目标地址在 BAR 3 范围内 ■ 100b: 目标地址在 BAR 4 范围内 ■ 101b: 目标地址在 BAR 5 范围内 ■ 110b: 保留 ■ 111b: 目标地址不在任何一个 BAR 空间范围内。 时钟域: core_clk radm_trgt1_hv 拉高时有效
radm_trgt1_io_req_in_range	1	0	指示接收到的 TLP 目标地址在 IO 地址范围内。 时钟域: core_clk radm_trgt1_hv 拉高时有效
radm_trgt1_cpl_status	3	0	CPL TLP 中的 Completion Status 字段。 时钟域: core_clk radm_trgt1_hv 拉高时有效
radm_trgt1_ats	2	0	TLP Header 中的 AT 字段。 时钟域: core_clk radm_trgt1_hv 拉高时有效
radm_trgt1_nw	1	0	TLP Header 中的 NW 字段。 时钟域: core_clk radm_trgt1_hv 拉高时有效



Port Name	Width	I/O	Description
radm_trgt1_th	1	0	TLP Header 中的 TH 字段。 时钟域: core_clk radm_trgt1_hv 拉高时有效
radm_trgt1_ph	2	0	TLP Header 中的 PH 字段。 时钟域: core_clk radm_trgt1_hv 拉高时有效
radm_trgt1_st	8	0	TLP Header 中的 ST 字段。 时钟域: core_clk radm_trgt1_hv 拉高时有效
radm_trgt1_bcm	1	0	接收到 TLP 的 BCM (Byte Count Modified) 字段 时钟域: core_clk radm_trgt1_hv 拉高时有效
radm_trgt1_byte_cnt	12	0	接收到 CPL TLP 中的 Byte Count 字段。 时钟域: core_clk radm_trgt1_hv 拉高时有效
radm_trgt1_cpl_last	1	0	若一个 Request TLP 对应多个 CPL, 则指示最后一个 CPL。若一个 Request 只对应一个 CPL, 则每个 CPL 都拉高。和 radm_trgt1_hv 同时拉高。 时钟域: core_clk radm_trgt1_hv 拉高时有效
radm_trgt1_cmpltr_id	16	0	CPL TLP 中的 Completer ID 字段。 ■ Bit[15:8]: Bus number ■ Bit[7:3]: Device number ■ Bit[2:0]: Function number 时钟域: core_clk radm_trgt1_hv 拉高时有效
radm_trgt1_vc	3	0	接收到的 CPL TLP 的 Virtual Channel 号。 时钟域: core_clk radm_trgt1_hv 拉高时有效
radm_grant_tlp_type	3	0	接收队列中准许 (grant) 的 VC 和 TLP 类型。 ■ 0: 准许 VC0 posted TLPs ■ 1: 准许 VC0 non-posted TLPs ■ 2: 准许 VC0 completion TLPs 脉冲信号, 和 trgt1_radm_pkt_halt 一起来控制接收队列输出。 时钟域: core_clk
trgt1_radm_pkt_halt	3	1	挂起 (Halt) 个别队列输出 ■ 0: 挂起 VC0 posted TLPs ■ 1: 挂起 VC0 non-posted TLPs ■ 2: 挂起 VC0 completion TLPs 时钟域: core_clk



Port Name	Width	I/O	Description
radm_trgt1_atu_sloc_match	2	0	ATU Single Location 匹配指示。当 ATU 区域匹配 VDM Single Location 地址翻译时拉高一个 core_clk 周期。 时钟域: core_clk radm_trgt1_hv 拉高时有效
radm_trgt1_atu_cbuf_err	2	0	ATU Error 指示。当接收到的 VDM size 大于 Circular Buffer Increment size 的时候, 拉高一个 core_clk 周期。 时钟域: core_clk radm_trgt1_hv 拉高时有效

注: 1. 该端口只在 EP 模式下使用, RC 模式无效。

2.4.3 用法

TRGT1 接口为 PCIe 控制器的 Memory/I/O Request TLP 包接收端口, TLP 类型为 Posted/Non Posted, 经过控制器内部的 Receive Buffer 缓存, TLP 的包数据传输过程中间不会存在停顿或间隙。

如下图所示, TRGT1 接口由以下几组信号组成:

- `radm_trgt1_hv/_dv/_eot`, `trgt1_radm_halt`: 该组信号指示当前传输 TLP 头 (Header) 和有效数据载荷 (Data Payload), 标志 TLP 之间的间隔, 以及使控制器与用户逻辑之间进行握手控制。
- `radm_trgt1_*header*/_data`: 该组信号指示当前 TLP 传输的头信息与数据。
- `radm_trgt1_tlp_abort/_dlp_abort/_ecrc_err`: 该组信号指示当前传输 TLP 的状态, 是否有 ECRC 错误或者指示用户逻辑是否需要丢弃当前 TLP。
- `trgt_lookup_id/_empty`: 该组信号提示当前 TLP 请求报文头信息存储在 Target Completion LUT 的 ID 和相关信息。

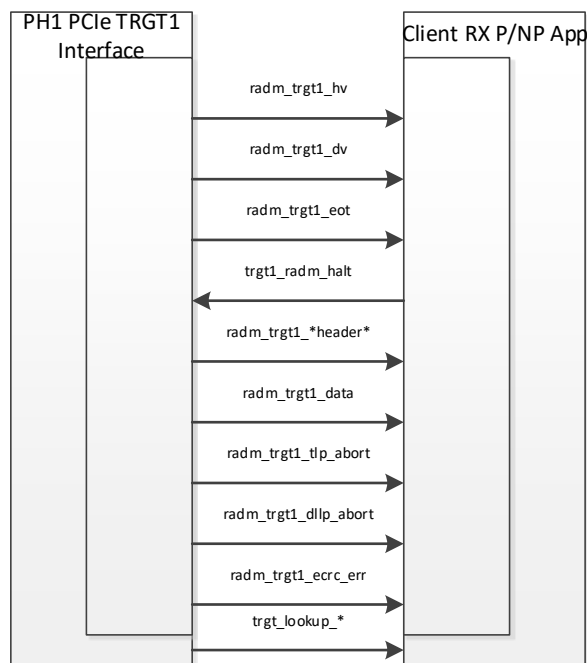


图 2-21 TRGT1 接口信号

下表为 PCI-SIG 规定的请求包 Header 格式和 PCIe 控制器的 TRGT1 接口信号的对应关系。该表格中斜体字为 TRGT1 接口信号名称。



表 2-8 TLP Request Header 格式和 trgt1 接口信号对应表

	+0								+1								+2								+3											
	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0				
Byte 0 >	Fmt[2]		Fmt[1:0]		radm_trgt1_type				T9 radm_trgt1_tae[9]		radm_trgt1_tc		TC		Attr[2] radm_trgt1_attr[2]		IN		TH radm_trgt1_th		TD radm_trgt1_td		EP radm_trgt1_ep		Attr[1:0]		AT radm_trgt1_ats		Length radm_trgt1_dw_len							
Byte 4 >	Requester ID radm_trgt1_reqid																TAG radm_trgt1_tag[7:0]								Last BE radm_trgt1_last_be				1 st BE radm_trgt1_first_be							
Byte 8 >	Address[63:32] radm_trgt1_addr[63:32]																																			
Byte 12 >	Address[31:2] radm_trgt1_addr[31:2]																														HP					

2.4.3.1 TRGT1 接口协议规则

- 1) 在 radm_trgt1_hv 拉高的周期中，所有包头（TLP Header）信息均有效。
- 2) 仅当 TLP 数据传输完成时，控制器才会将 radm_trgt1_dv 置为无效。

下图为接收 TLP 队列示意图。如果用户逻辑使用全局的暂停接收信号（trgt1_radm_halt）来反压接收 TLP 报文，就会出现 non-Posted TLP 阻塞 Posted TLP 报文。为了防止这一情况，用户逻辑可以通过 trgt1_grant_tlp_type 和 trgt1_radm_pkt_halt 来控制暂缓接收某个特定的 TLP 报文。

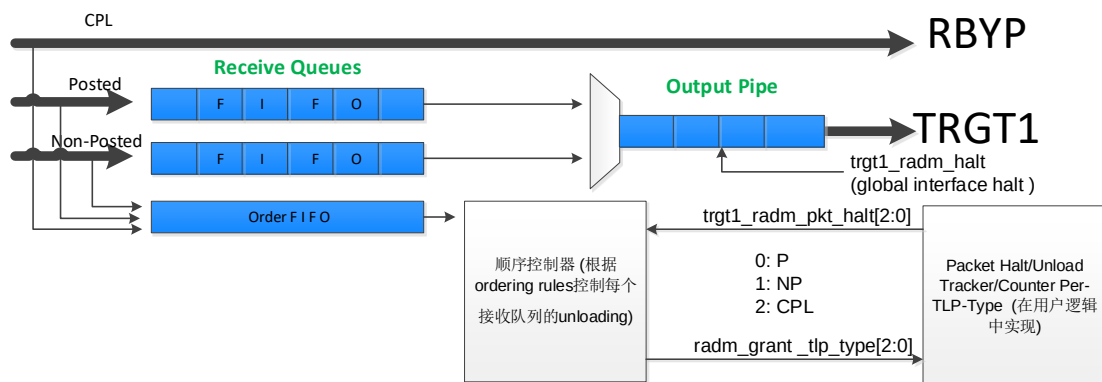


图 2-22 接收队列（Receive Queue）和输出队列（Output Queue）暂停（Halting）操作

radm_grant_tlp_type[2:0]信号是输出的脉冲，表示一个事务（transaction）被准许送出接收队列。trgt1_radm_pkt_halt[2:0]是电平输入信号，就像一个流控开关。它自己不能控制 PCIe 带来的准



确的事务个数。在 `trgt1_radm_pkt_halt[2:0]` 拉低和 `radm_grant_tlp_type[2:0]` 拉高至少要等待一个周期。

下图实现一个在 `radm_grant_tlp_type [2:0]` 上递增的计数器。当此计数器达到用户逻辑缓冲区中的可用 TLP 位置数时，请拉高 `trgt1_radm_pkt_halt`。使用完 Target LUT 中的所有条目后，RADM 会自动暂停 TRGT1 接口。

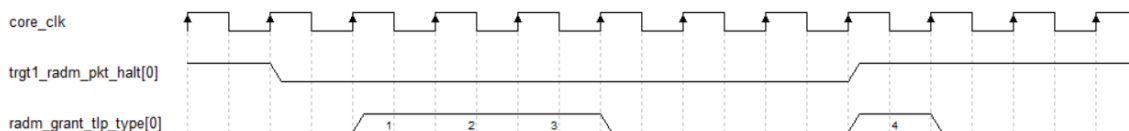


图 2-23 用 Grant/Halt 方式控制接收队列释放的示例

2.4.4 时序图

下图显示了一个示例，该示例在 TRGT1 接口上接收到来自上游的存储器写（MW_r）TLP。 `radm_trgt1_ecrc_err` 的置位表明 MW_r TLP 的 ECRC 错误； `radm_trgt1_abort` 的置位表明 MW_r TLP 格式错误。

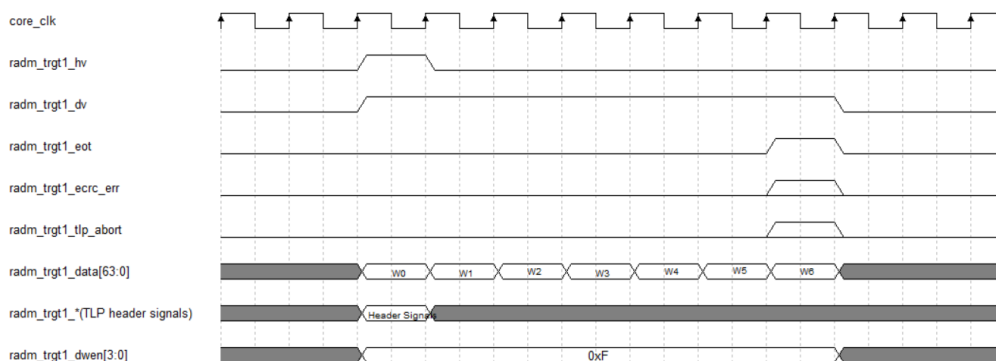


图 2-24 TRGT1 接口上的单个 MW_r Request TLP

下图显示了一个 TRGT1 接口时序示例，其中用户逻辑通过拉高 `trgt1_radm_halt` 插入等待状态。控制器通过拉高 `radm_trgt1_hv`、`radm_trgt1_dv` 和赋值 `radm_trgt1_data` 开始事务，然后将所有 TRGT1 输出保持为当前值，直到用户逻辑拉低 `trgt1_radm_halt`。在拉低 `trgt1_radm_halt` 的被采样后，控制器将继续执行 5-DWORD 报文。

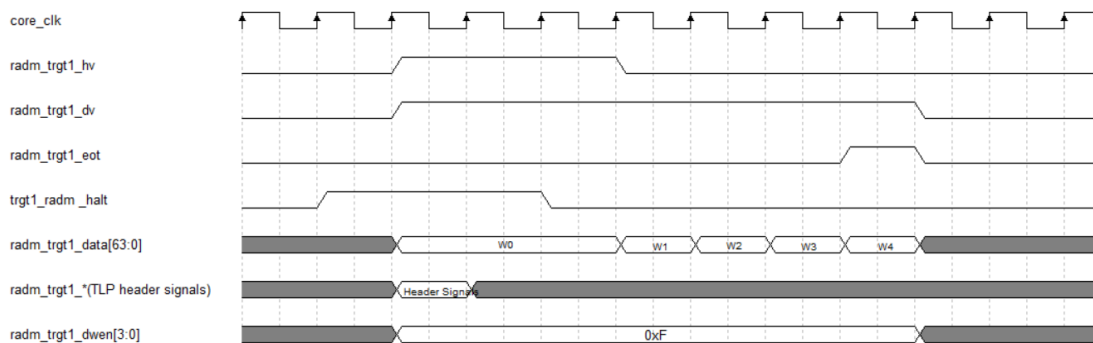


图 2-25 TRGT1 接口用户通过 halt 信号进行反压

下图表示接收 MWr 报文的具体 TLP Header 信号和多个 TLP 的波形示例。其中，radm_trgt1_hv/radm_trgt1_dv 分别指示 TLP Header/TLP Data 有效；radm_trgt1_fmt=0x2、radm_trgt1_type=0x00 表示收到 TLP 报文为 MWr；radm_trgt1_dw_len=0x1 表示该 MWr 数据负载 1 个 DW；radm_trgt1_dwen=0x1 表示数据负载在 radm_trgt1_data[31:0] 上有效；radm_trgt1_first_be=0xf 表示第一个 DW 所有字节有效。

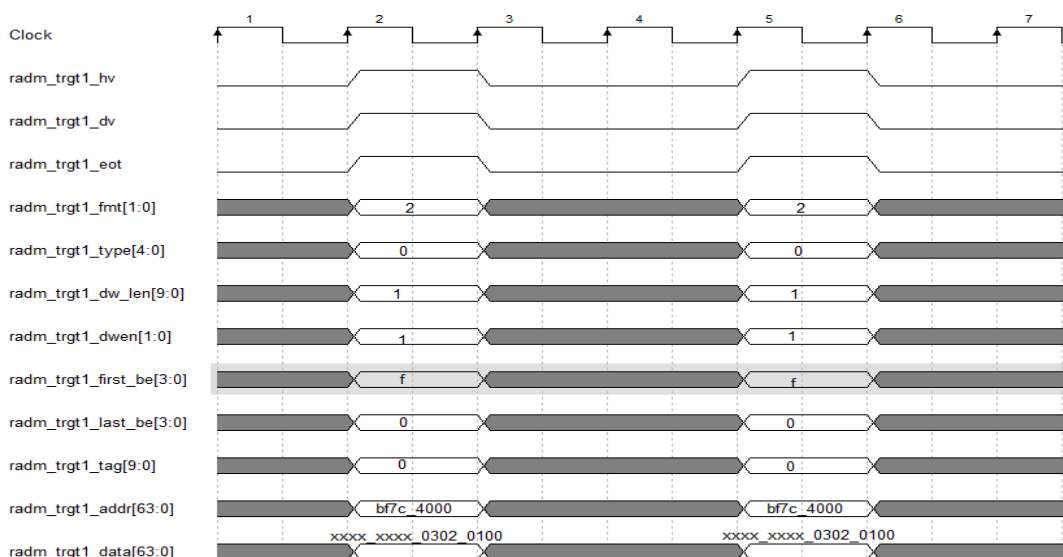


图 2-26 带有具体 Header 信号的多 MWr Request TLP

下图表示接收 MRd 报具体 TLP Header 信号和多个 TLP 波形示例。radm_trgt1_hv/radm_trgt1_dv 分别指示 TLP Header/TLP Data 有效；radm_trgt1_fmt=0x0、radm_trgt1_type=0x00 表示收到 TLP 报文为 memRd；radm_trgt1_dw_len=0x1 表示该 MRd 请求 1 个 DW 数据；radm_trgt1_tag 作为每个 MRd 报文的记号 (TAG)，一般为累加数；所以该 MRd 的含义为向地址 0xbf7c_4000 请求 1 个 DW 数据。

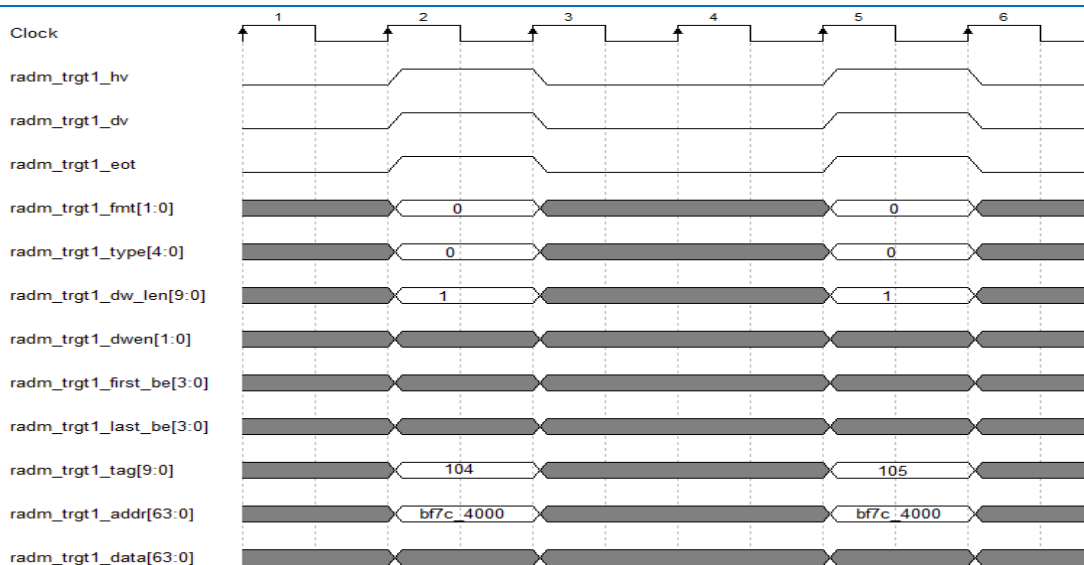


图 2-27 带有具体 Header 信号的多个 MRd Request TLP

2.4.4.1 Completer Atomic Operation

下图表示用户逻辑通过 TRGT1 Interface 接收原子(Atomic)操作，显示详细 TLP Header Signals。radm_trgt1_hv/radm_trgt1_dv 分别指示 TLP Header/TLP Data；radm_trgt1_fmt=0x2, radm_trgt1_type=0xc 表示 FetchAdd 操作；radm_trgt1_fmt=0x2, radm_trgt1_type=0xd 表示 Swap 总线事务；radm_trgt1_fmt=0x2, radm_trgt1_type=0xe 表示 CAS 总线事务；radm_trgt1_dw_len 表示读取原始数据长度；radm_trgt1_tag 表示 TLP 中的 TAG 字段。

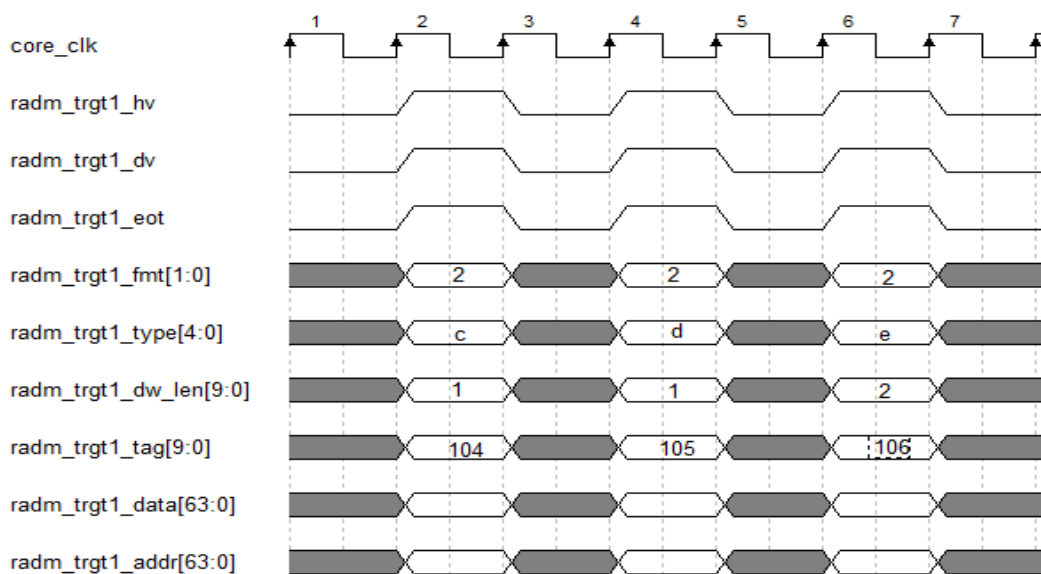


图 2-28 带有具体 Header 信号的多个 Atomic 报文接收



2.5 目标完成包查找表（Target Completion LUT）接口

2.5.1 功能简介

此接口在目标完成包查找表功能使能时，用于发送完成包。

2.5.2 端口

目标完成包查找表功能相关信号列表如下。

表 2-9 目标完成包查找表信号

Port Name	Width	I/O	Description
client0_cpl_lookup_id	10	I	XAL10 接口“目标完成包查找表”的查找 ID，必须和对应的请求报文的 trgt_lookup_id 值一致。当使用可选的“目标完成包查找表”功能时，控制器用存储在目标完成查找表里的 Byte Count, Tag, Attribute, Requester ID, Address 值去生成完成包 TLP 的 Header，而不使用相对应的输入端口上的值。这个信号只在发送完成包时有效。 时钟域: core_clk
client1_cpl_lookup_id	10	I	XAL11 接口“目标完成包查找表”的 Lookup ID，必须和对应的请求报文的 trgt_lookup_id 值一致。当使用可选的“目标完成包查找表”功能时，控制器用存储在目标完成查找表里的 Byte Count, Tag, Attribute, Requester ID, Address 值去生成完成包 TLP 的 Header，而不使用相对应的输入端口上的值。这个信号只在发送完成包时有效。 时钟域: core_clk
trgt_cpl_timeout	1	O	指示用户逻辑未能在规定时间内给接收到的请求生成完成包。关于超时的信息可以通过 trgt_timeout_* 输出端口体现。当发生完成包超时的情况，控制器不会清空查找表中相应的数据（entry）。默认的超时时间是 10ms。 时钟域: core_clk 高（脉冲信号）有效
trgt_timeout_cpl_func_num	1	O	发生完成包超时的 Function Number，起始值为 0。 时钟域: core_clk 高有效 trgt_cpl_timeout 拉高时有效
trgt_timeout_cpl_vfunc_num ¹	2	O	发生完成包超时的 VF 的 Function Number，起始值为 0 时钟域: core_clk trgt_cpl_timeout 和 trgt_timeout_cpl_vfunc_active 拉高时有效



Port Name	Width	I/O	Description
trgt_timeout_cpl_vfunc_active ¹	1	0	指示 timeout 来自于 VF (Virtual function)。 ■ 0 表示 timeout 不是来自于 VF 且 trgt_timeout_cpl_vfunc_num 无效。trgt_timeout_cpl_func_num 指向的 PF 有效 ■ 1 表示 trgt_timeout_cpl_vfunc_num 指向的 VF 有效 时钟域: core_clk 高有效: trgt_cpl_timeout 拉高时有效
trgt_timeout_cpl_tc	3	0	超时完成包的 TC 字段 时钟域: core_clk trgt_cpl_timeout 拉高时有效
trgt_timeout_cpl_attr	2	0	超时完成包的 Attribute 字段 时钟域: core_clk 高有效 trgt_cpl_timeout 拉高时有效
trgt_timeout_cpl_len	12	0	超时完成包的 Length 字段 时钟域: core_clk trgt_cpl_timeout 拉高时有效
trgt_timeout_lookup_id	10	0	超时完成包的 lookup ID 时钟域: core_clk trgt_cpl_timeout 拉高时有效
trgt_lookup_id	10	0	接收到的 Request TLP 的目标完成包查找表的 Lookup ID。当使用了可选的“目标完成包查找表”功能时，用户逻辑必须保存 Lookup ID 并在生成完成包时赋值到 client0/1_cpl_lookup_id 信号上。 时钟域: core_clk 高有效
trgt_lookup_empty	1	0	当此信号和 radm_trgt1_hv 都拉高时，表示目标完成包查找表非满。用户逻辑可以用此信号决定反压机制，以便于在收到的 outstanding non-posted 报文超过预期数量时避免出现溢出的情况。用户必须用应用逻辑的“halt”信号和该信号取反后进行“或”运算，来驱动 trgt1_radm_halt 时钟域: core_clk 高（电平信号）有效

注：1. 该端口只在 EP 模式下使用，RC 模式无效。

2.6 接收旁路 BYPASS 接口（RBYP）

2.6.1 功能简介

RBYP(Receive Bypass Interface)是用来连接控制器和用户逻辑的接收接口。它用的是旁路模式(Bypass Mode)接收队列，没有 Buffer 缓冲。这个端口用于接收 CPL TLP。用户逻辑需确保能接收所有 CPL 包。

2.6.2 端口



RBYP 接口信号列表如下。

表 2-10 RBYP 接口信号列表

Port Name	Width	I/O	Description
radm_bypass_data	64	0	接收到的 CPL TLP 的 Payload 数据，格式为 little endian。第一个收到的字节出现在 Bit[7:0]。PH1A100 固定为 64bit。 时钟域: core_clk, {radm_clk_g} radm_bypass_dv 拉高时有效
radm_bypass_dwen	2	0	用于指示 radm_bypass_data 最后一个 Dword 的位置。 64-bit 位宽 Bit[1:0] 映射关系如下 ■ 01: 最后一个 Dword 在 radm_bypass_data[31:0]. ■ 11: 最后一个 Dword 在 radm_bypass_data[63:32]. 时钟域: core_clk radm_bypass_eot 拉高时有效
radm_bypass_dv	1	0	Payload 数据，即 radm_bypass_data，有效指示。 时钟域: core_clk 高有效
radm_bypass_hv	1	0	Header 信号有效指示信号。拉高单周期。 时钟域: core_clk 高有效
radm_bypass_eot	1	0	接收到的 TLP Payload 即 radm_bypass_data 最后一个周期指示。 时钟域: core_clk 高有效 当 radm_bypass_hv 或 radm_bypass_dv 拉高时有效
radm_bypass_dllp_abort	1	0	指示当前 TLP 发生数据链路层 (data link layer) 错误。用户逻辑应丢弃此包，根据协议可等待重传。 时钟域: core_clk 高有效 当 radm_bypass_eot 拉高时有效
radm_bypass_tlp_abort	1	0	指示当前 TLP 发生 Malformed TLP, ECRC error, 或 completion lookup failures (如 TAG 或 requester ID 发生不一致) 等错误。用户逻辑应丢弃此包，不会发生重传。 时钟域: core_clk 高有效 radm_bypass_eot 拉高时有效
radm_bypass_ecrc_err	1	0	指示当前 TLP 发生 ECRC Error, 用户逻辑应丢弃此包，不会重传。 时钟域: core_clk 高有效 radm_bypass_eot 拉高时有效
radm_bypass_fmt	2	0	TLP Header 的 Format 字段。 时钟域: core_clk radm_bypass_hv 拉高时有效



Port Name	Width	I/O	Description
radm_bypass_type	5	0	TLP Header 的 Type 字段。 时钟域: core_clk radm_bypass_hv 拉高时有效
radm_bypass_tc	3	0	TLP Header 的 Traffic Class (TC) 字段。 时钟域: core_clk radm_bypass_hv 拉高时有效
radm_bypass_attr	3	0	TLP Header 的 Attribute 字段。 时钟域: core_clk radm_bypass_hv 拉高时有效
radm_bypass_reqid	16	0	TLP Header 中的 Requester ID 字段。按位编码映射如下: ■ Bit[15:8]: Bus number ■ Bit [7:3]: Device number ■ Bit [2:0]: Function number 时钟域: core_clk radm_bypass_hv 拉高时有效
radm_bypass_tag	10	0	TLP Header 中的 TAG 字段。 时钟域: core_clk radm_bypass_hv 拉高时有效。
radm_bypass_func_num	1	0	TLP Header 中的 PF 的 Function Number, 起始值为 0。 时钟域: core_clk radm_bypass_hv 拉高时有效。
radm_bypass_vfunc_num ¹	2	0	指示目标 VF 的 Function Number, 起始值为 0, 例如 radm_bypass_vfunc_num=0 指的是 PF 中的第一个 VF。 时钟域: core_clk 高有效 radm_bypass_hv 和 radm_bypass_vfunc_active 拉高时有效
radm_bypass_vfunc_active ¹	1	0	指示接收到的 TLP 目标为 VF。 0: 表示目标不是 VF, 因此 radm_bypass_vfunc_num 无效。因此 radm_bypass_func_num 所指向的 PF 有效。 1: 表示目标为 radm_bypass_vfunc_num 指向的 VF。 时钟域: core_clk 高有效 radm_bypass_hv 拉高时有效
radm_bypass_td	1	0	TLP Header 中的 TD 字段。 时钟域 core_clk radm_bypass_hv 拉高时有效
radm_bypass_poisoned	1	0	TLP Header 中的 EP 字段。 时钟域: core_clk 高有效 radm_bypass_hv 拉高时有效



Port Name	Width	I/O	Description
radm_bypass_dw_len	10	0	TLP Header 中的 Length 字段。 时钟域: core_clk 高有效 radm_bypass_hv 拉高时有效
radm_bypass_first_be	4	0	TLP Header 中的 First DW Byte Enable 字段。 时钟域: core_clk 高有效 radm_bypass_hv 拉高时有效
radm_bypass_last_be	4	0	TLP Header 中的 Last DW Byte Enable 字段。 时钟域: core_clk 高有效 radm_bypass_hv 拉高时有效
radm_bypass_addr	64	0	TLP Header 中的 Address, 3DW Header 中最后一个 Dword (8-11byte) 对应[31:0], 4DW Header 中第三个 Dword (8-11byte) 对应[63:32], 第四个 Dword (12-15byte) 对应[31:0]。 时钟域 core_clk radm_bypass_hv 拉高时有效
radm_bypass_rom_in_range	1	0	指示接收到的 TLP 目标地址在 Expansion ROM 范围内。 时钟域: core_clk radm_bypass_hv 拉高时有效
radm_bypass_io_req_in_range	1	0	指示接收到的 TLP 目标地址在 IO 地址范围内。 时钟域: core_clk radm_bypass_hv 拉高时有效
radm_bypass_in_membar_range	3	0	指示接收到的 TLP request 的目标地址所对应 BAR 空间。 ■ 000b: BAR 0. ■ 001b: BAR 1. ■ 010b: BAR 2. ■ 011b: BAR 3. ■ 100b: BAR 4. ■ 101b: BAR 5. ■ 110b: 保留 ■ 111b: 目标地址不在任何一个 BAR 空间
radm_bypass_bcm	1	0	接收到 TLP 的 BCM (Byte Count Modified) 字段。 时钟域: core_clk radm_bypass_hv 拉高时有效
radm_bypass_cpl_last	1	0	若一个 Request 对应多个 CPL TLP, 则指示最后一个 CPL。若一个 request 只对应一个 CPL, 则每个 CPL 都拉高此信号。和 radm_bypass_hv 同时拉高。 时钟域: core_clk 高有效 radm_bypass_hv 拉高时有效



Port Name	Width	I/O	Description
radm_bypass_cmpltr_id	16	0	CPL TLP 中的 Completer ID。按位编码映射关系如下： ■ Bit[15:8]: Bus number ■ Bit [7:3]: Device number ■ Bit [2:0]: Function number 时钟域: core_clk radm_bypass_hv 拉高时有效
radm_bypass_byte_cnt	12	0	接收到 TLP 的 Byte Count 字段。 时钟域: core_clk radm_bypass_hv 拉高时有效
radm_bypass_cpl_status	3	0	CPL TLP 中的 Completion Status 字段。 时钟域: core_clk radm_bypass_hv 拉高时有效
radm_bypass_ats	2	0	TLP Header 中的 AT 字段。 时钟域: core_clk radm_bypass_hv 拉高时有效
radm_bypass_th	1	0	TLP Header 中的 TH 字段。 时钟域: core_clk radm_bypass_hv 拉高时有效
radm_bypass_ph	2	0	TLP Header 中的 PH 字段。 时钟域: core_clk radm_bypass_hv 拉高时有效
radm_bypass_st	8	0	TLP Header 中的 ST 字段。 时钟域: core_clk radm_bypass_hv 拉高时有效

注：1. 该端口只在 EP 模式下使用，RC 模式无效。

2.6.3 用法

该接口用于接收 CPL TLP 报文。如下图：RBYP 接口由以下几组信号组成：

- radm_bypass_hv/_dv/_eot: 该组信号指示当前传输 TLP 头和数据有效, 标志 TLP 之间的间隔, 以及使控制器与用户逻辑之间进行握手。
- radm_bypass_*head*/_data: 该组信号指示当前 TLP 传输的头与数据。
- radm_bypass_tlp_abort/_dlp_abort/_ecrc_err: 该组信号指示当前传输 TLP 的状态, 是否有 ECRC 错误或者用户逻辑是否需要丢弃当前 TLP。

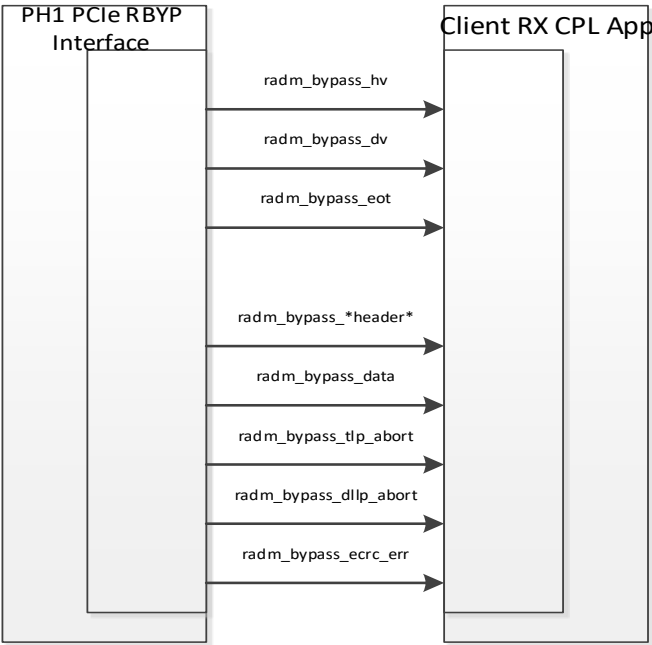


图 2-29 RBYP 接口信号

下表为 PCI-SIG 规定的完成包（completion）Header 格式和 PCIe 控制器的 BYPASS 接口信号的对应关系。该表格中斜体字为 BYPASS 接口信号名称。

表 2-11 Completion TLP Header 格式和 bypass 接口信号对应表

	+0								+1								+2								+3													
	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0						
Byte 0 >	Fmt [2]		Fmt [1:0] radm_bypass_fmt		Type radm_bypass_type				T9 radm_bypass_tag[9]		radm_bypass_tc		TC		T8 radm_bypass_tag[8]		Attr [2] radm_bypass_attr[2]		IN (unsupported)		TH radm_bypass_th		ID radm_bypass_id		EP radm_bypass_ep_reserved		Attr [1:0]		AT radm_bypass_ats		Length radm_bypass_dw_len							
Byte 4 >	Completer ID radm_bypass_cmpltr_id																radm_bypass_cpl_status		BCM radm_bypass_bcm		Compl. Status		Byte Count radm_bypass_byte_cnt															
Byte 8 >	Requester ID radm_bypass_reqid[15:0]																TAG radm_bypass_tag [7:0]								R		Lower Address radm_bypass_addr											



2.6.3.1 Bypass 接口协议规则

该控制器默认配置为旁路模式无缓存。

- 1) radm_bypass_hv 表示 TLP 包头信息有效的周期。
- 2) radm_bypass_dv 指示有效负载数据有效的周期。
- 3) 传输以 radm_bypass_hv 的置位开始，以 radm_bypass_eot 的置位结束。
- 4) radm_bypass_hv 和 radm_bypass_dv 都可以在同一周期内置位。
- 5) 仅当置位 radm_bypass_eot 时才可以置位 radm_bypass_tlp_abort 和 radm_bypass_dllp_abort。

2.6.4 时序图

下图表示 Bypass Interface 接收 CPLD 报文。radm_bypass_hv/radm_bypass_dv 分别指示 TLP Header/TLP Data 有效；radm_bypass_fmt=0x2, radm_bypass_type=0xa 表示当前传输的是 CPLD 报文；radm_bypass_byte_cnt=0x4 表示当前 CPLD 数据负载为 4bytes；radm_bypass_tag 为 0~3 与 MRd 报文的 TAG 相对应，用以区分 CPLD 顺序；radm_bypass_cpl_last 拉高指示 CPLD 报文传输的最后一个时钟周期。

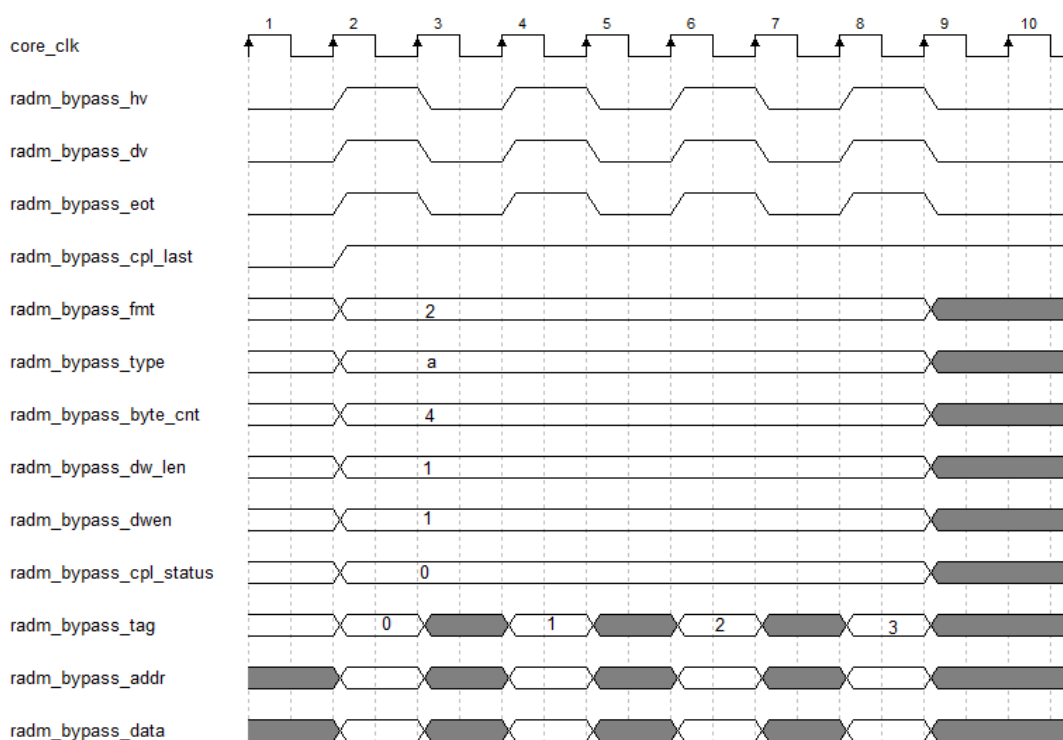


图 2-30 RBYP 接口上的完成包 Completion

2.6.4.1 Requester CPL Timeout

下图时序表示用户逻辑在通过 XAL11 发送 MRd 请求后，在超过寄存器空间指定的时间后没有接收到 CPL 报文，控制器会置位 radm_cpl_timeout 信号来表示 CPL 报文超时；当 radm_cpl_timeout 信号有效时，radm_timeout_cpl_len 表示该 timeout 的报文还需要传输多少字节才能结束；当 radm_cpl_timeout 信号有效时，radm_timeout_cpl_tag 表示该 timeout 报文的 TAG 字段，用以区分对



应哪个 memRd 的 CPL 报文超时。

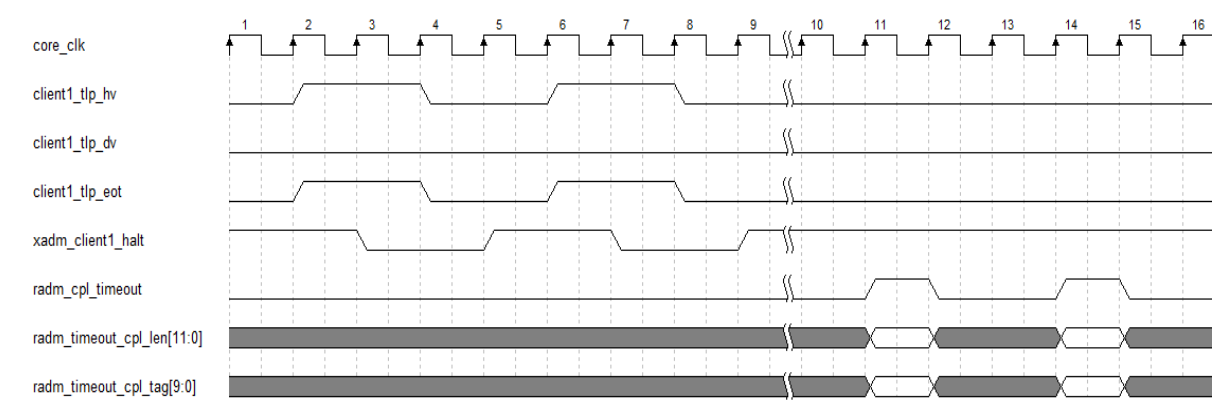


图 2-31 RBYP 接口上的完成包超时 Completion Timeout

2.7 DBI 接口

2.7.1 功能简介

DBI (Data Bus Interface) 是用户逻辑访问控制器内部 CDM 寄存器的接口，也可以访问 ELBI (External Local Bus Interface) 用户逻辑中的寄存器。可以接到本地 CPU 或者处理器。

2.7.2 端口

DBI 接口信号列表如下。

表 2-12 DBI 接口信号列表

Port Name	Width	I/O	Description
drp_dbi_addr	32	I	DBI 访问的 Config 寄存器地址。按位编码映射如下： ■ Bit[31:19]: 未用 ■ Bit [18:16]:未用 ■ Bit [15:12]: 未用 ■ Bit [11:2]: 寄存器地址，Dword 对齐 ■ Bit [1]: 未用 ■ Bit [0]: DBI 访问的目标， 0: 访问内部寄存器； 1: 访问外部寄存器 ELBI。当访问 ELBI 空间的时候，地址可以多于 12bit。 时钟域: core_clk dbi_cs 拉高时有效
drp_dbi_din	32	I	写数据总线。 时钟域: core_clk 高有效 dbi_cs 拉高时有效



Port Name	Width	I/O	Description
drp_dbi_cs	1	I	CDM 或 ELBI 的片选信号。 时钟域: core_clk 高有效
drp_dbi_cs2	1	I	如用户逻辑要写 BAR mask 寄存器, 需要的额外片选信号, 访问其他寄存器不需要拉高此信号。 时钟域: core_clk 高有效 dbi_cs 拉高时有效
drp_dbi_wr	4	I	读写选择信号。按位编码映射如下。 0000b: 读 0001b: 写 byte 0 0010b: 写 byte 1 0100b: 写 byte 2 1000b: 写 byte 3 1111b: 写全部 bytes 合并写多个字节也是有效的, 例如可以赋值为 0101b。 时钟域: core_clk 高有效 dbi_cs 拉高时有效
drp_lbc_dbi_ack	1	O	读写操作结束信号, 正常访问则拉高 2 个周期。如访问不存在的空间或者尝试写只读寄存器, 该信号只拉高一个周期。 时钟域: core_clk 高有效
drp_lbc_dbi_dout	32	O	读数据总线。 时钟域: core_clk 高有效 lbc_dbi_ack 拉高时有效
drp_dbi_vfunc_num ¹	2	I	指示被访问的 VF 号, 起始值为 0。 时钟域: core_clk 当 dbi_cs 和 dvi_vfunc_active 都拉高时有效
drp_dbi_vfunc_active ¹	1	I	表示 DBI 正在访问一个 VF, 0 表示没有访问 VF, 且 dbi_vfunc_num 无效, 此时 drp_dbi_func_num 选中的 PF 为有效; 1 表示 client0_tlp_vfunc_num 选中的 VF 有效。 时钟域: core_clk 高有效 dbi_cs 拉高时有效
drp_dbi_func_num	1	I	当 (CX_SRIOV_ENABLE or CX_ARI_ENABLE) = 1 时, 表示 DBI 正在访问的 PF 的 Function Number。按位编码映射如下。 00000b: Function 0 00001b: Function 1 时钟域: core_clk dbi_cs 拉高时有效

注: 1. 该端口只在 EP 模式下使用, RC 模式无效。



2.7.3 用法

■ DBI 协议规则

- 1) 拉高 `drp_dbi_cs` 表示一个活动的 CPU 周期。
- 2) `drp_dbi_wr` 输入指示启用字节的写访问。所有位为零表示读访问。
- 3) `drp_dbi_cs` 和 `lbc_dbi_ack` 信号形成同步握手。信号 `drp_dbi_cs` 必须保持置位状态，直到控制器拉高 `drp_lbc_dbi_ack` 作为响应。
- 4) 控制器拉高 `drp_lbc_dbi_ack` 持续两个时钟周期。
- 5) 当 `drp_dbi_addr[0] = 0` 时，将访问 CDM 中的内部配置寄存器。
- 6) 当 `drp_dbi_addr[0] = 1` 时，将访问 ELBI 上的外部寄存器，即用户逻辑中的寄存器。
- 7) DBI 第二个片选信号 (`drp_dbi_cs2`) 用于访问 shadow 和 iATU 配置寄存器。

用户访问 `capability` 4KB 空间只需要控制 `drp_dbi_addr[11:0]`, `drp_dbi_func_num`, `drp_dbi_vfunc_num`, `drp_dbi_vfunc_active`。

例如: `dbi address[11:0] = 12' h70; dbi_func_num = 1;`

`dbi_vfunc_num = 1;`

`dbi_vfunc_active = 1;`

表示用户正在访问 `pf1` 的 `vf1`, 地址是 `12' h70`, 即 `PCIE Capability Register`。

2.7.4 时序图

■ 写访问

下图描述了一个用户逻辑通过 DBI 对内部配置空间寄存器执行写访问的示例。

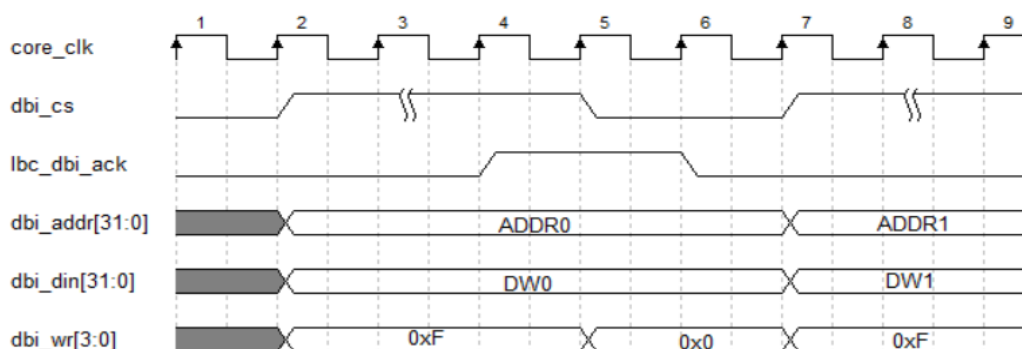


图 2-32 DBI 写寄存器时序图

■ 读访问

下图描述了一个用户逻辑通过 DBI 对配置空间寄存器执行读访问的示例。

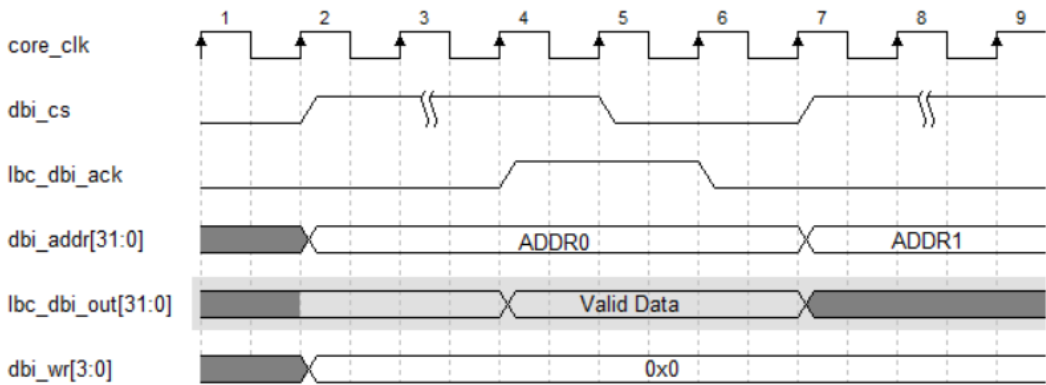


图 2-33 DBI 读寄存器时序图

2.8 ELBI 接口

2.8.1 功能简介

ELBI (External Local Bus Interface) 作用是，将控制器接收到的读或写请求 Request 报文，来访问控制器外部的用户逻辑寄存器的接口。控制器可以自动生成 CPL TLP。

2.8.2 端口

ELBI 接口信号列表如下。

表 2-13 ELBI 接口信号列表

Port Name	Width	I/O	Description
ext_lbc_ack ¹	2	I	表示控制器外部用户寄存器访问完成。也是 ext_lbc_din 的有效指示信号。从 lbc_ext_cs 拉高到 ext_lbc_ack 拉高的间隔没有要求。为了防止外部寄存器模块一直无法返回有效数据，用户逻辑可以实现一个超时监控电路以防止死锁。每个 bit 对应一个 function。即使 ELBI 不用，也不能把这个信号接为常 1。这个信号只能用作 lbc_ext_cs 拉高的响应。正常情况下这个信号拉高 2 个周期，发生异常时拉高一个周期。 异常情况包括：试图访问一个不存在的寄存器或者试图写一个只读的寄存器。 时钟域: core_clk 高有效 lbc_ext_cs 拉高时有效
ext_lbc_din ¹	64	I	外部寄存器模块数据总线。位宽为 64bit 时钟域: core_clk ext_lbc_ack 拉高时有效
lbc_ext_addr ¹	32	O	外部寄存器模块地址总线 时钟域: core_clk lbc_ext_cs 拉高时有效



Port Name	Width	I/O	Description
lbc_ext_dout ¹	32	0	写到外部寄存器模块的数据总线。 时钟域: core_clk lbc_ext_cs 拉高时有效
lbc_ext_cs ¹	2	0	当控制器接收到读写请求 TLP 的目标地址在 BAR 选择的外部寄存器模块范围内的时候, 拉高此信号。即 ELBI 接口的请求信号。每个 bit 对应一个 Function。控制器在接收到 ext_lbc_ack 完成信号后拉低这个信号。 时钟域: core_clk 高有效
lbc_ext_wr ¹	4	0	读或写指示信号。写操作时, 也可作为 byte enable 信号。按位编码映射如下: ■ 0000b: 读 ■ 0001b: 写 byte 0 ■ 0010b: 写 byte 1 ■ 0100b: 写 byte 2 ■ 1000b: 写 byte 3 ■ 1111b: 写 all bytes Byte Enable 也可以组合, 比如 0101b 也是合法的。 时钟: core_clk 高有效 lbc_ext_cs 拉高时有效
lbc_ext_rom_access ¹	1	0	指示当前 ELBI 访问的是 Expansion ROM 空间。 时钟域: core_clk 高有效 lbc_ext_cs 拉高时有效
lbc_ext_io_access ¹	1	0	指示当前 ELBI 是 IO 访问操作。 时钟域: core_clk 高有效 lbc_ext_cs 拉高时有效
lbc_ext_bar_num ¹	3	0	指示当前 ELBI 访问的 BAR 空间。按位编码映射如下: ■ 000b: BAR 0 ■ 001b: BAR 1 ■ 010b: BAR 2 ■ 011b: BAR 3 ■ 100b: BAR 4 ■ 101b: BAR 5 ■ 110b 未用 ■ 111b 指示配置请求 (CFG Request), 是用于用户自定义配置寄存器的访问, 比如 vendor capability 寄存器。所有的 DBI 访问都输出 111b。 时钟域: core_clk lbc_ext_cs 拉高时有效
lbc_ext_vfunc_num ¹	2	0	指示目标 VF 号, 从 0 起始。 时钟域: core_clk lbc_ext_cs 和 lbc_ext_vfunc_active 都拉高时有效



Port Name	Width	I/O	Description
lbc_ext_vfunc_active ¹	1	0	指示接收到的 TLP 访问目标是 VF。 时钟域: core_clk 高有效 lbc_ext_cs 拉高时有效

注：1. 该端口只在 EP 模式下使用，RC 模式无效。

2.8.3 用法

ELBI 协议规则

- 1) lbc_ext_cs 拉高表示该接口请求用户逻辑对数据包进行处理。
- 2) lbc_ext_wr 输出表示写访问字节使能 (Byte Enable)。对于读访问，控制器拉低所有的 bit 位。
- 3) 用户逻辑仅对指定 BAR Limit 的合法地址位进行解码。
- 4) lbc_ext_dout 输出仅在 lbc_ext_cs 拉高时有效。
- 5) lbc_ext_cs 和 ext_lbc_ack 信号形成同步握手。控制器保持 lbc_ext_cs 为高，直到用户逻辑拉高 ext_lbc_ack。lbc_ext_cs 和 ext_lbc_ack 之间的等待时间没有限制。
- 6) 用户逻辑必须返回一个 ack 信号，否则 transaction 会挂死。建议用户逻辑设计一个错误访问监控逻辑电路。当访问无法得到满足时，返回一个 ack 信号和 null 数据。
- 7) 在 ELBI 不用的时候，ext_lbc_ack 信号也不能被常接到 1' b1。也就是说 ext_lbc_ack 信号需跟随 lbc_ext_cs，当 lbc_ext_cs 处于非活跃状态时，ext_lbc_ack 也必须处于非活跃状态。
- 8) 用户逻辑在正常情况下必须拉高 ext_lbc_ack 两个时钟周期，除非发生 error，例如访问一个不存在的寄存器或写一个只读的寄存器，这时 ext_lbc_ack 必须拉高一个时钟周期。
- 9) 对于所有的 DBI 访问，控制器会将 lbc_ext_bar_num 输出为 3' b111。

2.8.4 时序图

下图为 ELBI 访问时序图。lbc_ext_cs 信号第一次拉高为写操作，4 个字节全部使能。lbc_ext_cs 信号第二次拉高为读操作。

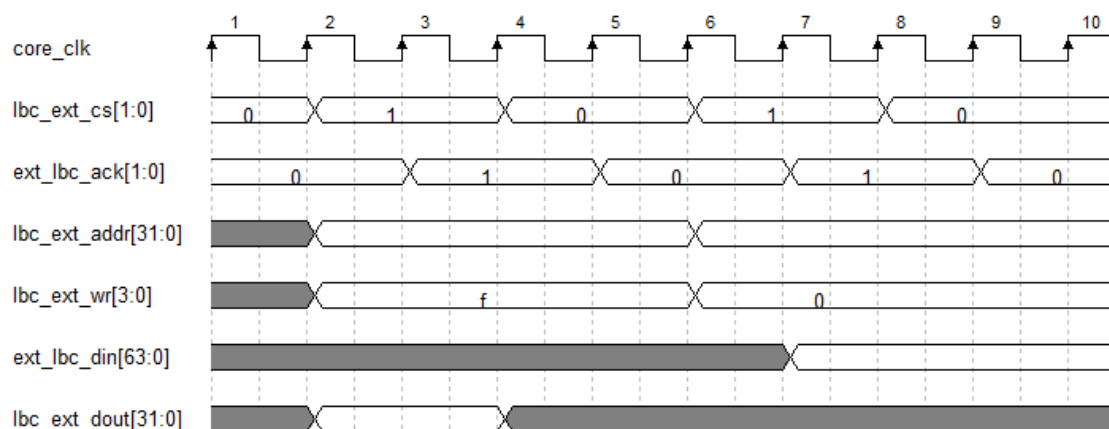


图 2-34 ELBI 访问时序图



2.9 SII: Legacy Interrupt 接口

2.9.1 功能简介

这个接口可以用来发送 Legacy Interrupt, 包括 Assert INTA, Assert INTB, Assert INTC, Assert INTD, De-assert INTA, De-assert INTB, De-assert INTC 和 De-assert INTD 这些 Message 报文。

2.9.2 端口

Legacy Interrupt 接口信号列表如下。

表 2-14 SII: Legacy Interrupt 接口信号列表

Port Name	Width	I/O	Description
sys_int ¹	2	I	当 sys_int 从低到高跳变, 控制器将会生成一个 Assert_INTx Message。当 sys_int 从高到低跳变, 控制器生成一个 Deassert_INTx Message。每个 function 对应一 bit 位。该 function 的 Interrupt Pin 寄存器决定了 INTx Message 是 INTA, INTB, INTC, 还是 INTD。 时钟域: core_clk
cfg_aer_rc_err_int ²	2	0	保留
cfg_aer_rc_err_msi ²	2	0	保留
cfg_aer_int_msg_num ²	10	0	保留
cfg_pme_int ²	2	0	保留
cfg_pme_msi ²	2	0	保留
cfg_pcie_cap_int_msg_num ²	10	0	保留
radm_inta_asserted ²	1	0	保留
radm_intb_asserted ²	1	0	保留
radm_intc_asserted ²	1	0	保留
radm_intd_asserted ²	1	0	保留
radm_inta_deasserted ²	1	0	保留
radm_intb_deasserted ²	1	0	保留
radm_intc_deasserted ²	1	0	保留
radm_intd_deasserted ²	1	0	保留
cfg_link_auto_bw_int ²	1	0	保留
cfg_link_auto_bw_msi ²	1	0	保留
cfg_bw_mgt_int ²	1	0	保留
cfg_bw_mgt_msi ²	1	0	保留
cfg_link_eq_req_int	1	0	告知用户逻辑 Link Status 2 寄存器的 Link Equalization Request 位为高, 且 Link Equalization Request Interrupt Enable (Link Control 3 Register bit 1) 为高。 时钟域: core_clk 高 (电平信号) 有效
assert_inta_grt	1	0	是单周期脉冲信号, 指示控制器发出了一个 Assert_INTA Message 给 UPS (upstream) 设备 时钟域: core_clk



Port Name	Width	I/O	Description
assert_intb_grt	1	0	是单周期脉冲信号，指示控制器发出了一个 Assert_INTB Message 给 UPS (upstream) 设备 时钟域: core_clk
assert_intc_grt	1	0	是单周期脉冲信号，指示控制器发出了一个 Assert_INTC Message 给 UPS (upstream) 设备 时钟域: core_clk
assert_intd_grt	1	0	是单周期脉冲信号，指示控制器发出了一个 Assert_INTD Message 给 UPS (upstream) 设备 时钟域: core_clk
deassert_inta_grt	1	0	是单周期脉冲信号，指示控制器发出了一个 Deassert_INTA Message 给 UPS (upstream) 设备 时钟域: core_clk
deassert_intb_grt	1	0	是单周期脉冲信号，指示控制器发出了一个 Deassert_INTB Message 给 UPS (upstream) 设备 时钟域: core_clk
deassert_intc_grt	1	0	是单周期脉冲信号，指示控制器发出了一个 Deassert_INTC Message 给 UPS (upstream) 设备 时钟域: core_clk
deassert_intd_grt	1	0	是单周期脉冲信号，指示控制器发出了一个 Deassert_INTD Message 给 UPS (upstream) 设备 时钟域: core_clk
cfg_int_pin	16	0	cfg_int_pin 指示 配置空间中 Interrupt Pin 寄存器的配置值。 时钟域: core_clk
cfg_int_disable	2	0	当此信号为高时，表示相应的 Fuction 没有生成 INTx messages 能力 时钟域: core_clk 高有效

注：1. 该端口只在 EP 模式下使用，RC 模式无效。

2. 该端口只在 RC 模式下使用，EP 模式无效。

2.9.3 用法

中断分为三种模式: Legacy Interrupt 模式, MSI 模式和 MSI-X 模式。本节描述 Legacy Interrupt。

■ Legacy Interrupt Mode

SII: Interrupt Interface 能使用户逻辑向控制器请求发送 Legacy Interrupt。该接口只能在 EP 端口进行使用。具体用法见时序图。

2.9.4 时序图

■ Legacy Interrupt Mode 时序图

如下图所示，当 PF 数量为 2 时，sys_int 位宽为 2bit，每个 bit 位对应一个 PF。控制器检测到 sys_int[0] 从低到高的过程后，会发送一个 Assert_INTx 的 message 报文；控制器检测到 sys_int[0] 从高到低的过程后，会发送一个 Deassert_INTx 的 message 报文。当控制器发送完 Assert_INTx 的 message 后，输出持续一个时钟周期的 assert_intx_grt 高脉冲作为提示；当控制器发送完 Deassert_INTx 的 message 后，输出持续一个时钟周期的 deassert_intx_grt 高脉冲作为提示。控制器输出 cfg_int_pin 信号指示配置空间中 Interrupt Pin 寄存器的值。

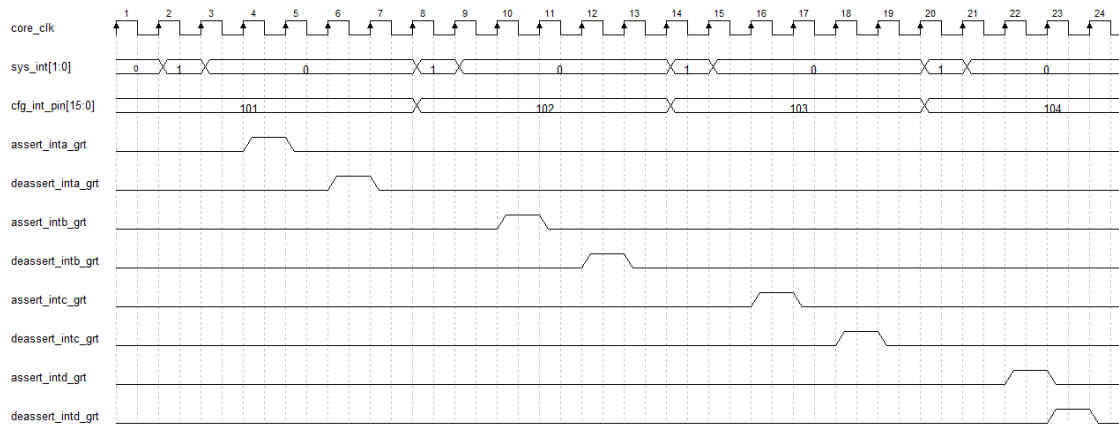


图 2-35 Legacy 中断请求时序图

2.10 MSI 接口

2.10.1 功能简介

MSI 接口是用户逻辑发送 MSI Request 给控制器的接口，独立于 XALI 接口。MSI 接口只用于 EP 模式。

2.10.2 端口

MSI 接口信号列表如下。

表 2-15 MSI 接口信号列表

Port Name	Width	I/O	Description
ven_msi_req	1	I	当 MSI 使能时，用户逻辑请求控制器发出 MSI 报文；当 MSI 没有使能时，控制器将发出 MSI-X 报文。一旦拉高必须保持高电平直到控制器拉高 ven_msi_grant 信号。 时钟域： core_clk 高有效
ven_msi_func_num	1	I	MSI 的 Function Number。起始值为 0。时钟域： core_clk ven_msi_req 拉高时有效
ven_msi_vfunc_num	2	I	指示 MSI 的 VF 号，起始值为 0。 时钟域： core_clk ven_msi_vfunc_active 拉高时有效
ven_msi_vfunc_active	1	I	指示 MSI 请求来自一个 VF。0：没有来自于 VF 的 MSI，因此 ven_msi_vfunc_num 无效。而 ven_msi_func_num 指示的 PF 有效。 1： MSI 来自于 VF，由 ven_msi_vfunc_num 指示。 时钟域： core_clk 高有效



Port Name	Width	I/O	Description
ven_msi_tc	3	I	MSI 的 Traffic Class (TX) 字段。 时钟域: core_clk 高有效 ven_msi_req 拉高时有效
ven_msi_vector	5	I	在 multiple message 模式下, MSI Data 后 5 位。 时钟域: core_clk 高有效 ven_msi_req 拉高, 并且 MSI Control 寄存器的 multiple message 模式使能的情况下才有效
ven_msi_grant	1	O	单周期脉冲, 表示控制器接受了发送 MSI 的请求。如果 ven_msi_grant 拉低后, ven_msi_req 继续拉高一个周期, 那么控制器会发出另一个 MSI。 时钟域: core_clk 高有效
cfg_msi_en	2	O	MSI 使能指示, INTx 不使能, 每个 function 对应 1 bit 位。 时钟域: core_clk
cfg_msi_mask	64	O	MSI Capability 中 Per Vector Mask 寄存器的内容。 时钟域: core_clk
cfg_msi_pending	64	I	用户逻辑告知控制器哪些 functions 有正在 pending 的 message, 被用来决定 MSI Capability 中的 Vector Interrupt Pending 寄存器。 时钟域: core_clk

2.10.3 用法

中断分为三种模式: Legacy Interrupt 模式, MSI 模式和 MSI-X 模式。本节描述 MSI 模式。

■ MSI Mode

MSI Interface 能使用户逻辑向控制器请求发送 MSI 中断。该接口只能在 EP 端口进行使用。

MSI 协议规则:

- 1) MSI 接口协议是简单的同步请求/响应握手。
- 2) 用户逻辑使能 ven_msi_req 输入时, 必须保持有效, 直到控制器输出 ven_msi_grant 有效。
- 3) 用户逻辑通过 MSI 接口请求 MSI。然后, 控制器生成相应的 MWr 操作。
- 4) 以下 MSI 接口信号也通用于 “MSI-X 协议事务”:
 - ven_msi_req
 - ven_msi_func_num
 - ven_msi_tc
 - ven_msi_grant
- 5) 控制器支持 PVM (每个向量掩码)。

2.10.4 时序图



■ MSI Mode 时序图

MSI 中断请求的产生:

下图描述了一个示例，用户逻辑在 `cfg_msi_en` 拉高时，请求控制器向上游发送 MSI 消息。
`ven_msi_req` 保持拉高状态，直到控制器拉高 `ven_msi_grant`。

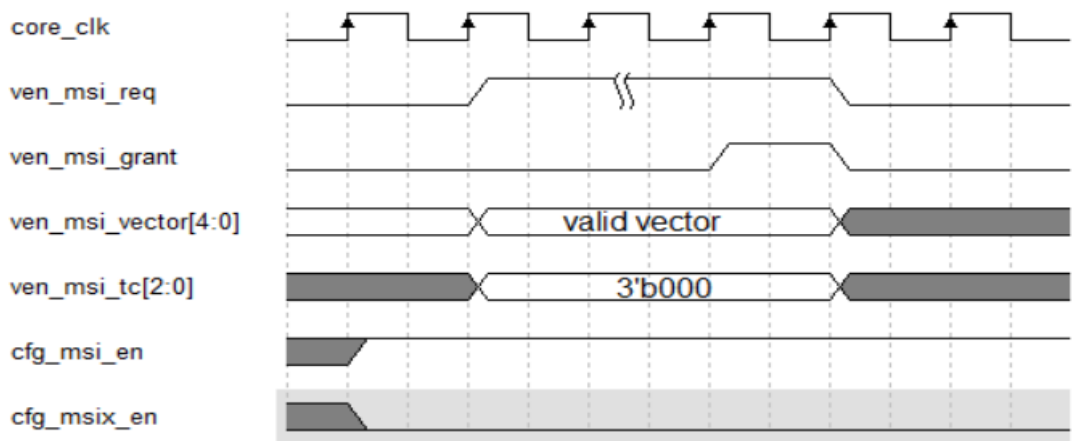


图 2-36 MSI 中断请求时序波形

内部 MSI 仲裁器对来自不同功能的 MSI 请求执行仲裁。 `ven_msi_grant` 是确认 `ven_msi_req` 的单周期脉冲。无需拉低 `ven_msi_req`，然后再将其拉高。当 `ven_msi_req` 保持拉高时，控制器将生成另一个 MSI。

2.11 MSI-X 接口

2.11.1 功能简介

MSI-X 接口是用户逻辑发送 MSI-X Request 给控制器的接口，独立于 XALI 接口。MSI-X 接口只用于 EP 模式。

2.11.2 端口

MSI-X 接口信号列表如下。

表 2-16 MSI-X 接口信号列表

Port Name	Width	I/O	Description
msix_addr	64	I	MSI-X 地址。 时钟域: core_clk ven_msi_req 拉高时有效



Port Name	Width	I/O	Description
msix_data	32	I	MSI-X 数据。 时钟域: core_clk ven_msi_req 拉高时有效
cfg_msix_en	2	O	MSI-X Control 寄存器中 MSI-X Enable 位。每个 Function 对应 1 位。 时钟域: core_clk 高有效
cfg_msix_func_mask	2	O	MSI-X Control 寄存器的 Mask 位。每个 Function 对应 1 位。 时钟域: core_clk 高有效
cfg_vf_msix_en	4	O	每个 VF 的 MSI-X Capability 结构的 Message Control Register 中 MSIX_EN 位。在 VF 发送 MSI-X 之前, 用户逻辑需确保相应的位已经置高。每一位对应一个 VF。 时钟域: core_clk 高有效
cfg_vf_msix_func_mask	4	O	每个 VF 的 MSI-X Capability 结构中 Message Control Register 的 Mask 位。每一位对应一个 VF。 时钟域: core_clk
cfg_vf_msix_table_size	22	O	VF 的 MSI-X Capability 结构 MSI-X Control 寄存器的 MSI-X Table Size 字段。每个 function 对应 11 位。 时钟域: core_clk
cfg_vf_msix_table_bir	6	O	VF MSI-X Capability 结构 MSI-X Table Offset and BIR 寄存器的 Table BAR Indicator Register (BIR) 字段。每个 function 对应 3 位。 时钟域: core_clk
cfg_vf_msix_table_offset	58	O	VF MSI-X Capability 结构 MSI-X Table Offset and BIR 寄存器的 Table Offset 字段。每个 function 对应 29 位。 时钟域: core_clk
cfg_vf_msix_pba_bir	6	O	VF MSI-X Capability 结构 MSI-X Table Offset and BIR 寄存器的 PBA BAR Indicator Register (BIR) 字段。每个 function 对应 3 位。 时钟域: core_clk
cfg_vf_msix_pba_offset	58	O	VF MSI-X Capability 结构 MSI-X Table Offset and BIR 寄存器的 PBA Offset 字段。每个 function 对应 29 位。 时钟域: core_clk
cfg_msix_table_size	22	O	MSI-X Capability 结构 MSI-X Table Offset and BIR 寄存器的 MSI-X Table Size 字段。每个 function 对应 11 位。 时钟域: core_clk
cfg_msix_table_bir	6	O	MSI-X Capability 结构 MSI-X Table Offset and BIR 寄存器的 Table BAR Indicator Register (BIR) 字段。每个 function 对应 3 位。 时钟域: core_clk
cfg_msix_table_offset	58	O	MSI-X Capability 结构 MSI-X Table Offset and BIR 寄存器的 Table Offset 字段。每个 function 对应 29 位。 时钟域: core_clk
cfg_msix_pba_bir	6	O	MSI-X Capability 结构 MSI-X Table Offset and BIR 寄存器的 PBA BIR 字段。每个 function 对应 3 位。 时钟域: core_clk



Port Name	Width	I/O	Description
cfg_msix_pba_offset	58	0	MSI-X Capability 结构 MSI-X Table Offset and BIR 寄存器的 PBA Offset 字段。每个 function 对应 29 位。 时钟域: core_clk

2.11.3 用法

中断分为三种模式: Legacy Interrupt 模式, MSI 模式和 MSI-X 模式。本节描述 MSI-X 模式。

■ MSI-X Mode

MSI-X Interface 能使用户逻辑向控制器请求发送 MSI-X 中断。该接口只能在 EP 端口进行使用。

MSI-X 协议规则:

- 1) 启用 MSI-X 时,以下 MSI 接口信号通用于 MSI-X: ven_msi_req, ven_msi_func_num, ven_msi_tc 和 ven_msi_grant。
- 2) 要请求控制器发送 MSI-X 消息,您的应用程序与发送 MSI 时方式相同,使用前面列出的信号。
- 3) 此外,您的应用程序必须分别在 msix_addr 和 msix_data 上提供 MSI-X 地址和数据。
- 4) 主机系统软件在系统初始化期间将 MSI-X 地址和数据值存储在位于应用程序存储空间中的 MSI-X 表中。

2.11.4 时序图

■ MSI-X Mode 时序图

MSI-X 中断请求的产生:

下图显示了 MSI-X 中断请求一个示例,该示例在 cfg_msix_en 声明时请求控制器向上游发送 MSI-X 消息。ven_msi_req 保持高电平状态,直到控制器拉高 ven_msi_grant。

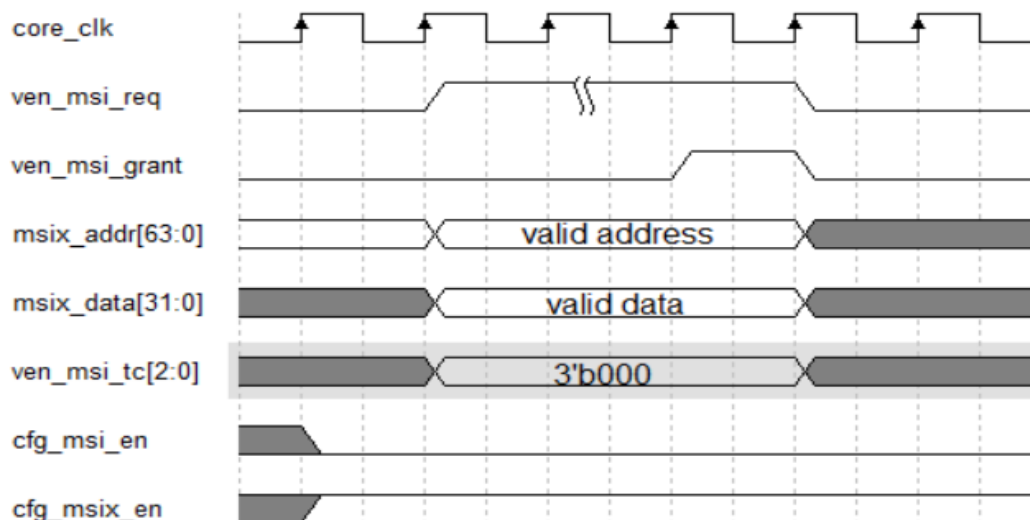


图 2-37 MSI-X 中断请求时序图

与 MSI 一样，内部仲裁器对来自不同功能的 MSI-X 请求执行仲裁。ven_msi_grant 是一个单周期脉冲，此后控制器不等待 ven_msi_req 无效。ven_msi_grant 单周期脉冲过后，如果 ven_msi_req 仍为高电平，控制器将生成另一个 MSI-X。

2.12 物理层（PHY）部分接口和数据链路层（Data Link）部分接口

2.12.1 功能简介

PCIe 控制器在上电后或复位之后，需进行建链（Link Training）的过程，建链成功后 LTSSM 进入 L0 状态。

2.12.2 端口

Link Training 相关信号列表如下。

表 2-17 Link Training 相关信号列表

Port Name	Width	I/O	Description
rdlh_link_up	1	0	Data link 层建链成功指示信号。 高有效
rtlh_rfc_upd ¹	1	0	指示控制器接收到了 Flow Control Update DLLP。用户逻辑可以在控制器外部实现流控。相应的 DLLP 出现在 rtlh_rfc_data[31:0]。
rtlh_rfc_data ¹	32	0	Flow Control Update DLLP 具体数据。 时钟域： core_clk 高有效



Port Name	Width	I/O	Description
			rtlh_rfc_upd 拉高时有效
radm_q_not_empty	1	0	高表示接收队列中有 TLP Header/Data
radm_qoverflow	1	0	脉冲指示 1 个或多个 P/NP/CPL 接收队列发生了溢出。 时钟域: core_clk 高脉冲表示溢出
cpl_debug_info	64	0	一些内部信号的状态, 用于 Debug。按位编码映射如下。 ■ [63]: smlh_scrambler_disable: 加扰关闭 ■ [62]: smlh_link_disable: LTSSM 在 DISABLE 状态, Link inoperable。 ■ [61]: smlh_link_in_training: LTSSM 正在 link training 过程中 ■ [60]: smlh_ltssm_in_pollconfig: LTSSM 在 Polling Configuration 状态 ■ [59]: smlh_training_rst_n: LTSSM-negotiated link reset ■ [58:55]: 0000b: 保留 ■ [54]: 保留 ■ [53]: mac_phy_txelecidle[0]: PIPE 发送电气空闲请求 ■ [52]: mac_phy_txcompliance[0]: PIPE 发送 compliance 请求 ■ [51]: app_init_rst: 应用请求开始训练复位 ■ [50:48]: 000b: 保留 ■ [47:40]: 保留 ■ [39:38]: 00b: 保留 ■ [37]: xmtbyte_skip_sent: 一个 skip ordered set 已被发送 ■ [36]: smlh_link_up: LTSSM 报告 PHY 建立链接或 Loopback Master LTSSM 在 Loop-back.Active 状态 ■ [35]: 保留 ■ [34]: 保留 ■ [33]: 保留 ■ [32]: 保留 ■ [31:28]: 保留 ■ [27]: 保留 ■ [26]: 保留



Port Name	Width	I/O	Description
			■ [25]: rmlh_rcvd_idle[0]: 接收器收到 logical idle ■ [24]: rmlh_rcvd_idle[1]: 高字节收到 logic idle (16bit PHY 适用) ■ [23:8]: 保留 ■ [7:6]: 保留 ■ [5:0]: smlh_ltssm_state: LTSSM 当前状态。按位编码映射和 smlh_ltssm_state 相同。 时钟域: core_clk
cxpl_debug_info_ei	16	0	接收器电气空闲 (Electrical Idle, EI) 内部状态。按位编码映射如下: 第一组 (脉冲) - 接收 Ordered Sets 信息 ■ [0]: 检测到 EIOS 第二组 (电平) - LTSSM 状态为依赖于 rxelecidle =0 的下列某一状态: ■ [1]: L1 ■ [2]: L2 ■ [3]: RxL0s ■ [4]: Disabled ■ [5]: Detect.Quiet ■ [6]: 保留 ■ [7]: 保留 第三组 (电平) - LTSSM 状态为依赖于 rxelecidle =1 的下列某一状态: ■ [8]: LTSSM 在 L1 或 L2 之前的临时状态 ■ [9]: LTSSM 在 Disabled 之前的临时状态 ■ [10]: LTSSM 在 Gen1 Slave 的 Loopback.Active 状态 ■ [11]: 保留 第四组 (脉冲) - LTSSM 因 EI inferred 发生跳变 ■ [12]: LTSSM 因收到 EI inferred 从 L0 进入 Recovery ■ [13]: 保留 ■ [14]: 保留 ■ [15]: 保留 时钟域: core_clk



Port Name	Width	I/O	Description
smlh_link_up	1	0	物理层建链成功信号。 时钟域: core_clk 高有效
smlh_req_rst_not	1	0	link_req_rst_not 的 early version
link_req_rst_not	1	0	指示复位请求。低电平指示控制器由于某种原因请求外部用户逻辑复位控制器。一般是物理层断链或控制器接收到一个 Hot Reset 请求。 时钟域: core_clk 低有效。
smlh_ltssm_state	6	0	当前 LTSSM 状态。按位编码映射如下。 ■ 6'h00: S_DETECT_QUIET ■ 6'h01: S_DETECT_ACT ■ 6'h02: S_POLL_ACTIVE ■ 6'h03: S_POLL_COMPLIANCE ■ 6'h04: S_POLL_CONFIG ■ 6'h05: S_PRE_DETECT_QUIET ■ 6'h06: S_DETECT_WAIT ■ 6'h07: S_CFG_LINKWD_START ■ 6'h08: S_CFG_LINKWD_ACCEPT ■ 6'h09: S_CFG_LANENUM_WAIT ■ 6'h0A: S_CFG_LANENUM_ACCEPT ■ 6'h0B: S_CFG_COMPLETE ■ 6'h0C: S_CFG_IDLE ■ 6'h0D: S_RCVRY_LOCK ■ 6'h0E: S_RCVRY_SPEED ■ 6'h0F: S_RCVRY_RCVRCFG ■ 6'h10: S_RCVRY_IDLE ■ 6'h11: S_L0 ■ 6'h12: S_L0S ■ 6'h13: S_L123_SEND_IDLE ■ 6'h14: S_L1_IDLE ■ 6'h15: S_L2_IDLE



Port Name	Width	I/O	Description
			■ 6'h16: S_L2_WAKE ■ 6'h17: S_DISABLED_ENTRY ■ 6'h18: S_DISABLED_IDLE ■ 6'h19: S_DISABLED ■ 6'h1A: S_LPBK_ENTRY ■ 6'h1B: S_LPBK_ACTIVE ■ 6'h1C: S_LPBK_EXIT ■ 6'h1D: S_LPBK_EXIT_TIMEOUT ■ 6'h1E: S_HOT_RESET_ENTRY ■ 6'h1F: S_HOT_RESET ■ 6'h20: S_RCVRY_EQ0 ■ 6'h21: S_RCVRY_EQ1 ■ 6'h22: S_RCVRY_EQ2 ■ 6'h23: S_RCVRY_EQ3

注：1. 该端口只在 RC 模式下使用，EP 模式无效。

2.13 VMI 接口

2.13.1 功能简介

用户逻辑可以使用 VMI 来发送 Vendor-Defined Messages。可以独立于 XALIO/1 接口发送 Message。

2.13.2 端口

VMI 接口信号列表如下。

表 2-18 VMI 接口信号列表

Port Name	Width	I/O	Description
ven_msg_fmt	2	I	vendor-defined Message TLP 的 Format 字段。 时钟域: core_clk ven_msg_req 拉高时有效
ven_msg_type	5	I	vendor-defined Message TLP 的 Type 字段。 时钟域: core_clk ven_msg_req 拉高时有效



Port Name	Width	I/O	Description
ven_msg_tc	3	I	vendor-defined Message TLP 的 Traffic Class (TC) 字段。 时钟域: core_clk ven_msg_req 拉高时有效
ven_msg_td	1	I	vendor-defined Message TLP 的 TLP Digest (TD) 字段。 时钟域: core_clk ven_msg_req 拉高时有效
ven_msg_ep	1	I	vendor-defined Message TLP 的 Poisoned TLP (EP) 字段。 时钟域: core_clk ven_msg_req 拉高时有效
ven_msg_attr	3	I	vendor-defined Message TLP 的 Attributes 字段。 时钟域: core_clk ven_msg_req 拉高时有效
ven_msg_len	10	I	vendor-defined Message TLP 的 Length 字段。指示 Dword 数量。 时钟域: core_clk ven_msg_req 拉高时有效
ven_msg_func_num	1	I	vendor-defined Message TLP 的 Function Number。起始值为 0。 时钟域: core_clk ven_msg_req 拉高时有效
ven_msg_vfunc_num ¹	2	I	访问 VMI 接口的 VF 的 Function Number。起始值为 0。 时钟域: core_clk ven_msg_req 和 ven_msg_vfunc_active 拉高时有效
ven_msg_vfunc_active ¹	1	I	指示访问 VMI 接口的是 VF。 ■ 0: 访问 VMI 的不是 VF 且 ven_msg_vfunc_num 无效, ven_msg_func_num 指示的 PF 为有效。 ■ 1: 访问 VMI 的是 VF, ven_msg_vfunc_num 有效 时钟域: core_clk ven_msg_req 拉高时有效
ven_msg_tag	10	I	vendor-defined Message TLP 的 Tag 字段。 时钟域: core_clk ven_msg_req 拉高时有效
ven_msg_code	8	I	vendor-defined Message TLP 的 Message Code 字段。 时钟域: core_clk ven_msg_req 拉高时有效
ven_msg_data	64	I	Vendor Defined Message header 的第三个和第四个 Dword ■ 字节 8-11 (Header 第三个 Dword) =ven_msg_data[63:32] ■ 字节 12-15 (Header 第四个 Dword) =ven_msg_data[31:0], 其中 ven_msg_data[7:0] =byte 15 时钟域: core_clk ven_msg_req 拉高时有效



注：1. 端口只在 EP 模式下使用，RC 模式无效

VMI 是用于发送 vendor-defined message 的接口。VMI 的协议规则是一个简单的同步的请求 (request)+应答 (acknowledge) 的握手过程。VMI 只能用于发送没有 payload, 只有 header 的 vendor-defined message (Msg)。MsgD 是不支持的。若要发送 vendor-defined 的 MsgD, 用户逻辑需要使用发送客户端接口 XALIO 或 XALI1。

- 1) 如果 VMI 不用，那么 `ven_msg_req` pin 必须接 0。
- 2) 当 `ven_msg_req` 拉高以后，它必须保持为高，直到控制器拉高 `ven_msg_grant`。
- 3) `ven_msg_req` 和 `ven_msg_grant` 之间的时间间隔是没有定义的。
- 4) 用户逻辑必须在采样到 `ven msg grant` 为高以后在下一个周期拉低 `ven msg req`。

Timing diagram for the ven_msg_* signals. The diagram shows the relationship between the core_clk, ven_msg_req, ven_msg_grant, ven_msg_*(header), and ven_msg_data[63:0] signals. The ven_msg_req signal is active high and is followed by ven_msg_grant, which is also active high. The ven_msg_*(header) signal is active high and carries the header information. The ven_msg_data[63:0] signal is active high and carries the data. The diagram illustrates the sequence of events: a request is made, followed by a grant, then the header is transferred, and finally the data is transferred.

图 2-38 VMI 接口时序图

FLR 是允许特定的 Function 被复位而不影响其他 Function 的机制，整个链路 Link 作为一个整体



不受影响。FLR 适用于 PF 也适用于 VF。FLR 对于使能了 SR-IOV 的 Endpoint 来说是必选的；而对于未使能 SR-IOV 的 Endpoints 来说是可选的。

2.14.2 端口

FLR 相关的信号列表如下。

表 2-19 FLR 接口信号列表

Port Name	Width	I/O	Description
cfg_flr_pf_active	2	0	当软件对一个 PF 启动 FLR（写入 Initiate FLR），该端口拉高。这个信号会一直拉高，直到用户逻辑和控制器都完成了复位。 时钟域: core_clk 高有效
app_flr_pf_done	2	1	表示 PF 的 FLR 已经完成，意味着用户逻辑完成了该 PF 的数据结构的初始化，且对该 PF 的用户逻辑没有更多的 TLP 在发送队列中等待发送。这个信号必须一直拉高，直到控制器拉低对应 PF 的 cfg_flr_pf_active 信号。 时钟域: core_clk 高有效
cfg_flr_vf_active	4	0	当软件对一个 VF 启动 FLR（写入 Initiate FLR），该端口拉高。这个信号会一直拉高，直到用户逻辑和控制器都完成了复位。每一位对应一个 VF。同时可以有多个 VF 在 FLR 状态，因此控制器可以拉高多个位。 时钟域: core_clk 高有效
app_flr_vf_done	4	1	表示 VF 的 FLR 已经完成，意味着用户逻辑完成了该 VF 的数据结构的初始化，且对该 VF 的用户逻辑没有更多的 TLP 在发送队列中等待发送。这个信号必须一直拉高，直到控制器拉低对应 VF 的 cfg_flr_vf_active 信号。每一位对应一个 VF。同时可以有多个 VF 在 FLR 状态，因此控制器可以拉高多个位。 时钟域: core_clk 高有效

2.14.3 用法

PCI Express Base Specification, Revision 3.0, Version 1.0 要求整个复位过程在 100ms 内完成。控制器需要 (4 * 64 * clock_period) 去复位一个 function。

- 1) 上位机软件通过 CfgWr 请求写 PFx 的 Initiate FLR 位，也可以是本地的 DBI 写操作。控制器拉高 cfg_flr_pf_active[x] 输出信号作为响应。
- 2) 用户逻辑中断 TLP 发送
- 3) 当用户逻辑停止了 PF 的应用，它需要拉高 app_flr_pf_done[x]。
- 4) 当控制器探测到 app_flr_pf_done[x] 为 1 时，它开始 PFx 的 FLR。
- 5) 当 FLR 复位完成以后，控制器会拉低 cfg_flr_pf_active[x]

2.14.4 时序图

下图是 PF 的 FLR 时序。PF 和 VF 的时序是相同的，不同的是 PF 对应的信号是 cfg_flr_pf_active



和 app_flr_pf_done 而 VF 对应的信号是 cfg_flr_vf_active 和 app_flr_vf_done。

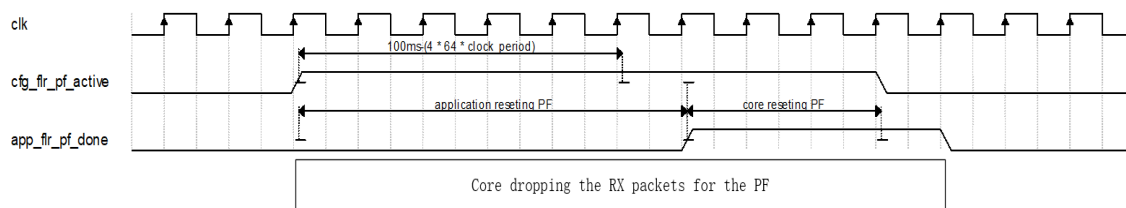


图 2-39 FLR 时序图

2.15 CRI (Control Register Interface) 接口

2.15.1 功能简介

该接口用于访问 PHY 内部寄存器。

2.15.2 端口

下表为 CRI 接口列表。

表 2-20 CRI 端口列表

Port Name	Width	I/O	Description
drp_phy0_cr_para_ack	1	0	参考 UG709_安路科技 PH1A100 系列 FPGA SERDES 用户手册中 cr_para_ack 信号
drp_phy0_cr_para_rd_data	16	0	参考 UG709_安路科技 PH1A100 系列 FPGA SERDES 用户手册中 cr_para_rd_data 信号
drp_phy0_cr_para_clk	1	1	参考 UG709_安路科技 PH1A100 系列 FPGA SERDES 用户手册中 cr_para_clk 信号
drp_phy0_cr_para_wr_en	1	1	参考 UG709_安路科技 PH1A100 系列 FPGA SERDES 用户手册中 cr_para_wr_en 信号
drp_phy0_cr_para_rd_en	1	1	参考 UG709_安路科技 PH1A100 系列 FPGA SERDES 用户手册中 cr_para_rd_en 信号
drp_phy0_cr_para_addr	16	1	参考 UG709_安路科技 PH1A100 系列 FPGA SERDES 用户手册中 cr_para_addr 信号
drp_phy0_cr_para_wr_data	16	1	参考 UG709_安路科技 PH1A100 系列 FPGA SERDES 用户手册中 cr_para_wr_data 信号

2.15.3 用法

该接口详细用法和时序图参考《UG709_安路科技 PH1A100 系列 FPGA SERDES 用户手册》的“CRI 接口”章节。



3 版本信息

日期	版本	修订记录
2020/05/09	0.1	安路初始版本。
2020/12/17	1.0	修正格式，更新为正式版本。
2022/02/15	1.1	1. 第二章和第三章合并为一章，每一套端口的端口列表和这套端口的时序图放在同一节； 2. 将全文 PCIe 协议描述中的 FPGA 修改为 CPLD； 3. 修改图 1-1 以及相应的文字说明，更新 SERDES DUAL 坐标值； 4. 表 1-3 去掉 BAR1，BAR3 和 BAR5 的相关描述； 5. 增加表 2-4，表 2-5，表 2-6，表 2-8，各个接口和 TLP Header 格式映射关系表格； 6. 删除 2.3.4.3 章节中 TRGT_CPL_LUT_EN 的说明，该寄存器固定为 1，用户无需控制； 7. 更新图 2-20，去掉 client0_tlp_dwen[3:0]信号； 8. 更新表 2-20 中参考的 serdes 文档名称； 9. 增加表 2-1 中复位会清空 PCIe 寄存器的描述；
2023/02/08	1.2	1. 表 2-1 及后续端口列表中标注 EP 和 RC 模式是否使用； 2. 表 2-1 增加了 app_power_up_rst_n 能够复位底层 PHY 的 PLL 的说明； 3. 表 2-3 中修改 client0_tlp_byte_len 信号描述。 4. 增加表 2-1 对 app_auxclk 的描述； 5. 增加 2.3.1 章节对 Client 接口的描述；



版权所有©2023 上海安路信息科技股份有限公司

未经本公司书面许可，任何单位和个人都不得擅自摘抄、复制、翻译本文档内容的部分或全部，并不得以任何形式传播。

免责声明

本文档并未授予任何知识产权的许可，并未以明示或暗示，或以禁止发言或其他方式授予任何知识产权许可；本文档仅为向用户提供使用器件的参考，协助用户正确地使用安路科技产品之用，其著作权归安路科技所有；本文档所展示的任何产品信息均不构成安路科技对所涉产品或服务作出任何明示或默示的声明或保证。

安路科技将不定期地对本文档进行更新、修订。用户如需获取最新版本的文档，可通过安路科技的官方网站（网址为：<https://www.anlogic.com>）自行查询下载，也可联系安路科技的销售人员咨询获取。